

DATA STRUCUTRES AND ALGORITHMS

23CSE203

LAB MANUAL

Field	Details
Submitted By	S. Aitezaz Kauser
Roll No	AV.SC.U4CSE25051
Year/Sem/Section	2nd Year / Sem-3 / CSE-A
Credits	05

Field	Details
Submitted To	Dr. RAJ KUMAR BATCHU
Department	School of Computing
Designation	Asst. Professor (SI Grade)

"3rd Sem CSE – A | DSA Lab Schedule"

DATA STRUCTURES AND ALGORITHMS – INDEX

S.No.	Title	Page No.	Date
1.	WEEK 1: 1a. Recursive countdown from n to Launch. 1b. Recursive calculation of compound growth factor. 1c. Recursive search for employee ID in list. 1d. Recursive factorial calculation for parcel arrangements. 1e. Recursive Fibonacci series generation.	4–13	23-07-2026
2.	WEEK 2: 2a. Python program to perform Linear Search. 2b. Python program to perform Binary Search on a sorted array. 2c. Python program to perform Binary Search on an unsorted array.	14–19	30-07-2026
3.	WEEK 3: 3a. Bubble Sort. 3b. Insertion Sort. 3c. Selection Sort. 3d. Merge Sort. 3e. Quick Sort.	20–33	06-08-2026
4.	WEEK 4: 4a. Implementation of Singly Linked List. 4b. Implementation of Doubly Linked List. 4c. Implementation of Circular Linked List with Display Head and Tail and Reverse Display operations.	34–76	13-08-2026
5.	WEEK 5: Implementation of Stack using: a) Array.	77–84	27-08-2026

S.No.	Title	Page No.	Date
	b) Linked List. **Operations: Push, Pop, Peek and Display.**		
6.	**WEEK 6:** **Implementation of Queue and its operations:** **6a. Using Array and Linked List.** **6b. Circular Queue.**	**85–104**	**03-09-2026**
7.	**WEEK 7:** **Implementation of Binary Tree:** **7a. Using Array.** **7b. Using Linked List with all operations.**	**105–124**	**24-09-2026**
8.	**WEEK 8:** **Implementation of Binary Search Tree (BST) and AVL Tree with their operations.**	**125–144**	**08-10-2026**
9.	**WEEK 9:** **Implementation of Hashing using Separate Chaining, Linear Probing, Quadratic Probing and Double Hashing.**	**145–160**	**22-10-2026**

Week-1

1)Aim:- A rocket launch system displays a count down before launch. Instead of using a loop implement the count down using a recursion ,write a recursive program to display a count down from n to 1 followed by "Launch"

Program:

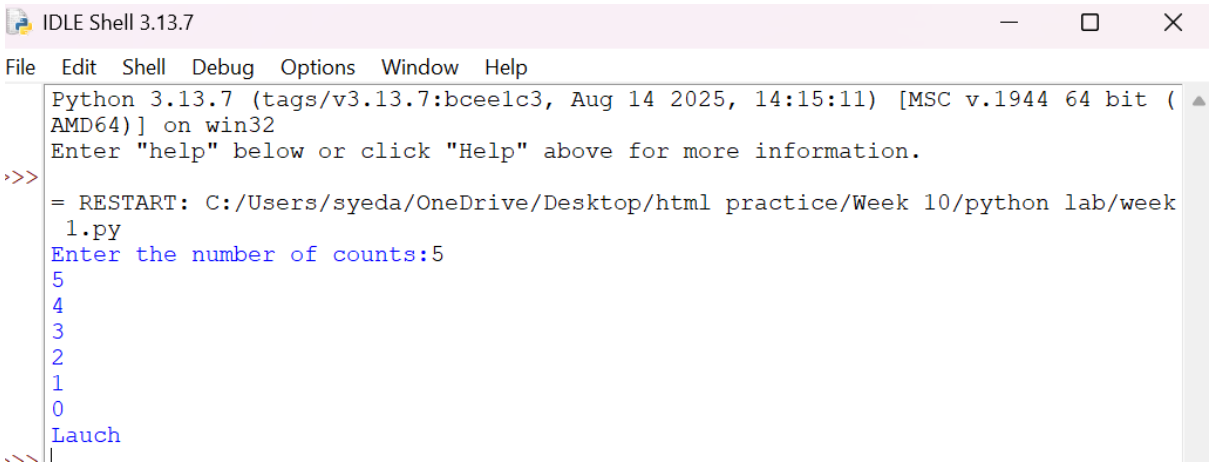
```
'''def countdown(n):
    print(n)
    if n>0:
        return countdown(n-1)
    elif n==0:
        return 0
```

n=int(input("Enter the number of counts:"))

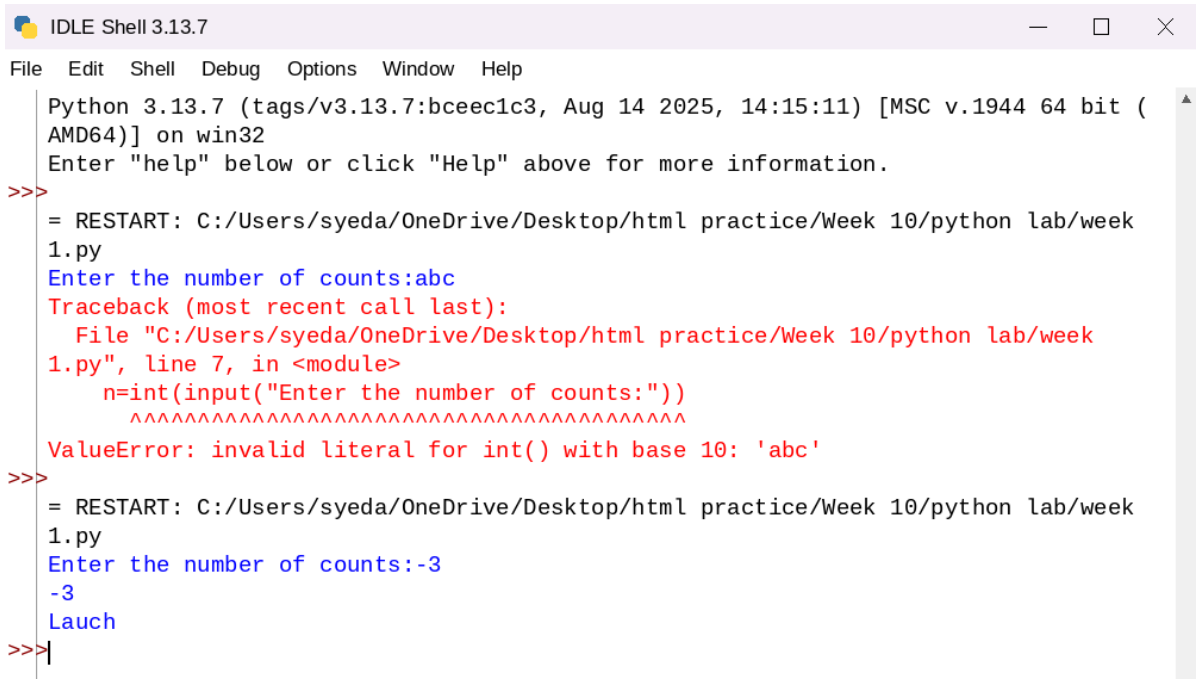
countdown(n)

print("Lauch")'"

Output:



Negative Case:



Explanation:

- The function launch(n) prints the current value of `n`.
- If `n` is greater than 0, it decreases `n` by `1` and calls itself recursively.

- The recursion continues until n becomes 0, which is the base case, and the function stops.

Error Table:

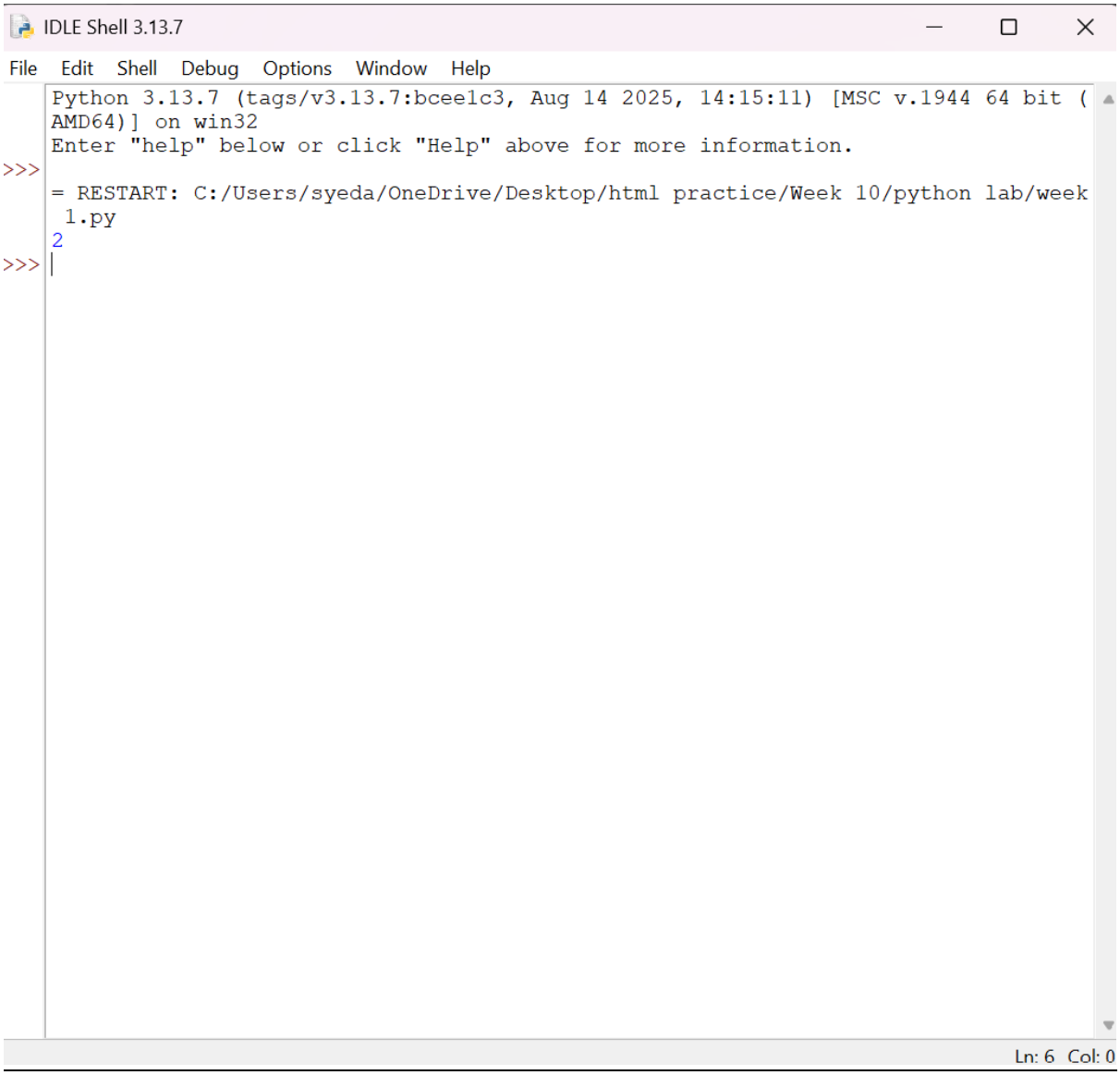
S.No	Error Obtained	Rectification
01.	Didn't write the print statement after definition.	After execution I found the mistake and added it.

2)Aim: An Hr department stores employee ID in a list . Develop a recursive program to search for a given employee ID write a recursive function to search for an employee ID . Write a recursive function to search for an employee ID in a list.

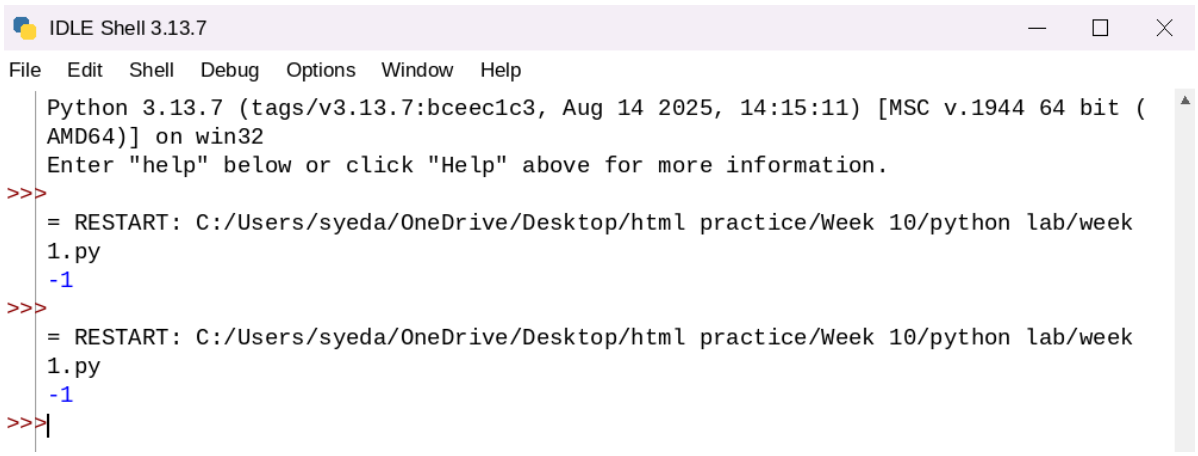
Program:

```
def search(arr, target, i):
    if i >= len(arr):
        return -1
    if arr[i] == target:
        return i + 1
    else:
        return search(arr, target, i + 1)
A = [1, 2, 3, 4]
print(search(A, 2, 0))  # Output: 2
```

Output:



Negative Case:



Explanation:

This program uses recursion to perform a linear search for an employee ID (or any target element) in a list.

1. The function empid_Search(arr, target, n=0) is defined with three parameters:
    - arr: The list in which the search is performed.
    - target: The element to be searched.
    - n: The current index, initialized to 0.
2. The statement if n == len(arr): return -1 is the base condition. It checks whether all elements in the list have been examined. If the end of the list is reached without finding the target, the function returns -1.
3. The statement if arr[n] == target: return n compares the current element with the target. If they are equal, the function returns the current index where the target is found.
4. If the current element does not match the target, the function calls itself recursively with the next index using return empid_Search(arr, target, n + 1).
5. The list arr = [10, 20, 30, 50, 607, 809, 506] is created, and the function is called to search for the element 50.
6. The returned value is stored in the variable result.
7. If result is not equal to -1, the program prints the index where the element is found. Otherwise, it prints "Element not found."

Error table:

S.No	Error Obtained	Rectification
01.	Recursion Error: maximum recursion depth exceeded	Increment the index in the recursive call: return empid_Search(arr, target, n + 1).

3)Aim: This Python program uses recursion **to calculate the value of** p^n (p raised to the power n)**.**

Program:

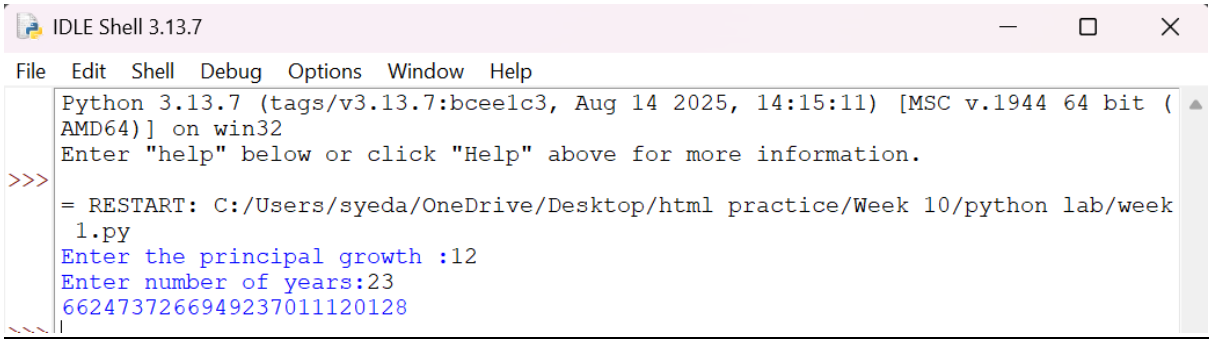
```
def CI(p,n):
    if n==0:
        return 1
    else:
        return (CI(p,n-1))*p

P=int(input("Enter the principal growth :"))
```

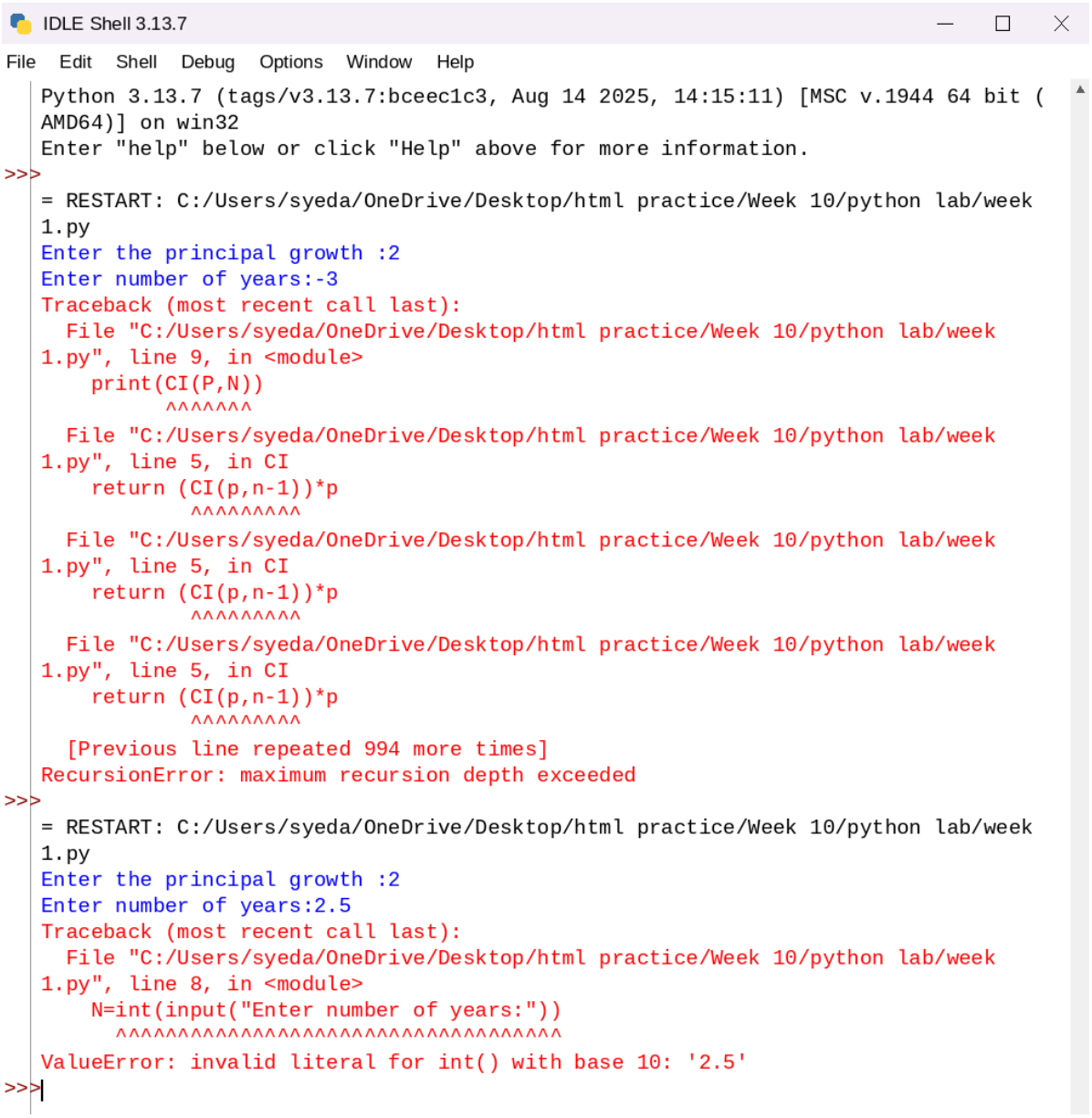
```
N=int(input("Enter number of years:"))

print(CI(P,N))
```

Output:



Negative Case:



Explanation:

- The program is used to find the power of a number (p^n) using recursion.
- It takes two inputs from the user:

  - p → Base number
  - n → Exponent (power)

- The function power(p, n) is called to calculate the result.
- If the exponent n is **0**, the function returns **1** because any number raised to the power 0 is
- If n is not 0, the function calls itself with n - 1 and multiplies the result by p.
- This recursive process continues until the exponent becomes **0**.
- After reaching the base case, each recursive call returns its result, and the final value of p^n is obtained.

- The result is stored in the variable result.
- Finally, the program prints the calculated power of the given number.

Error Table:

S.No	Error Obtained	Rectification
01.	Not a int value of n we can also take float in place of int, I have taken float value to check.	By replacing int by float.

4)Aim: Write a recursive python program for fibanocci series , print first n terms series.

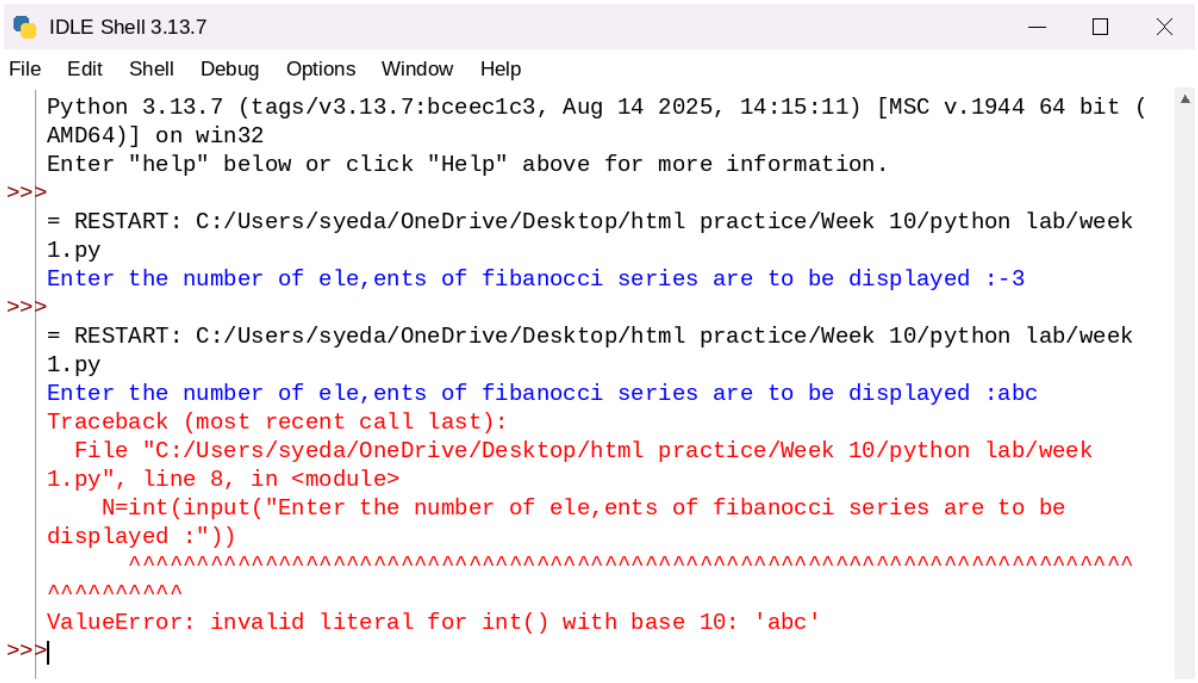
Program:

```
def fib(n):
    if n==0:
        return 0
    elif n==1:
        return 1
    else:
        return fib(n-1)+fib(n-2)
N=int(input("Enter the number of ele,ents of fibanocci series are to be displayed :"))
for i in range(N):
    print(fib(i), end=" ")
```

Output:

```
>>>
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week
 1.py
Enter the number of ele,ents of fibanocci series are to be displayed :3
0 1 1
>>>
```

Negative Case:



Explanation:

- The program uses a recursive function fib(n) to generate Fibonacci numbers.
- The base condition returns 0 when n = 0 and 1 when n = 1.
- For values greater than 1, the function returns the sum of the previous two Fibonacci numbers: fib(n-1) + fib(n-2).
- The user enters the number of terms (n) to be displayed.
- A for loop calls the fib() function repeatedly for each term from 0 to n-1.

- Finally, the program prints the Fibonacci series in sequence.

Error Table:

S.No	Error Obtained	Rectification
01.	No Errors obtained	----------

5)Aim: In a logistic company , the number of possible ways to arrange n different parcels is calculated using the factorial formula (n!) , develop a recursive program to calculate factorial of a given number.

Program:

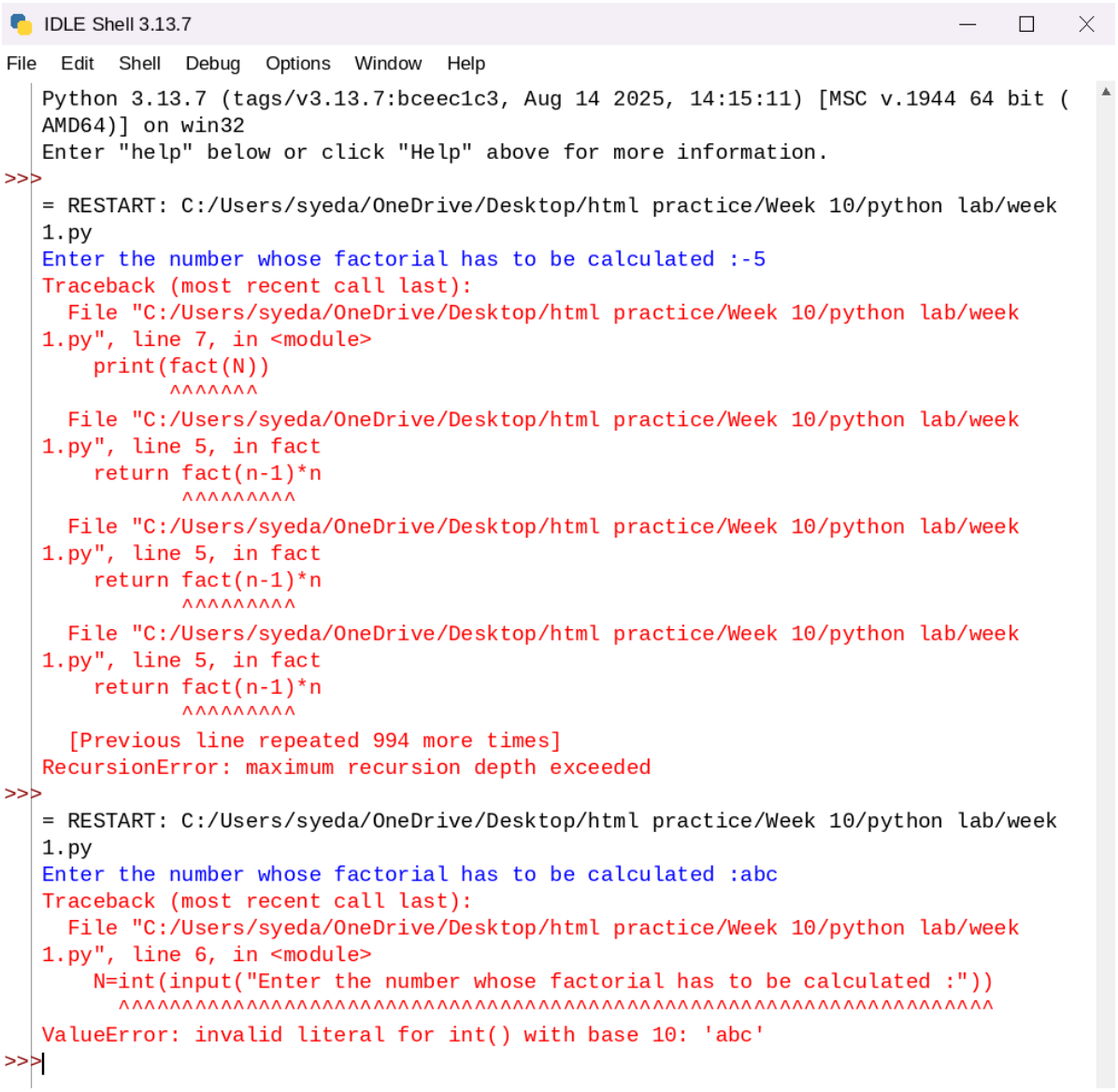
def fact(n):

```
    if n==0 or n==1:
        return 1
    else:
        return fact(n-1)*n
N=int(input("Enter the number whose factorial has to be calculated :"))
print(fact(N))
```

Output:

```
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week
 1.py
Enter the number whose factorial has to be calculated :5
120
```

Negative Case:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bceec1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py
Enter the number whose factorial has to be calculated :-5
Traceback (most recent call last):
File "C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py", line 7, in <module>
print(fact(N))
^^^^^^^
File "C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py", line 5, in fact
return fact(n-1)*n
^^^^^^^^^
File "C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py", line 5, in fact
return fact(n-1)*n
^^^^^^^^^
File "C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py", line 5, in fact
return fact(n-1)*n
^^^^^^^^^
[Previous line repeated 994 more times]
RecursionError: maximum recursion depth exceeded
>>>
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py
Enter the number whose factorial has to be calculated :abc
Traceback (most recent call last):
File "C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week 1.py", line 6, in <module>
N=int(input("Enter the number whose factorial has to be calculated :"))
^^^^^^^^^

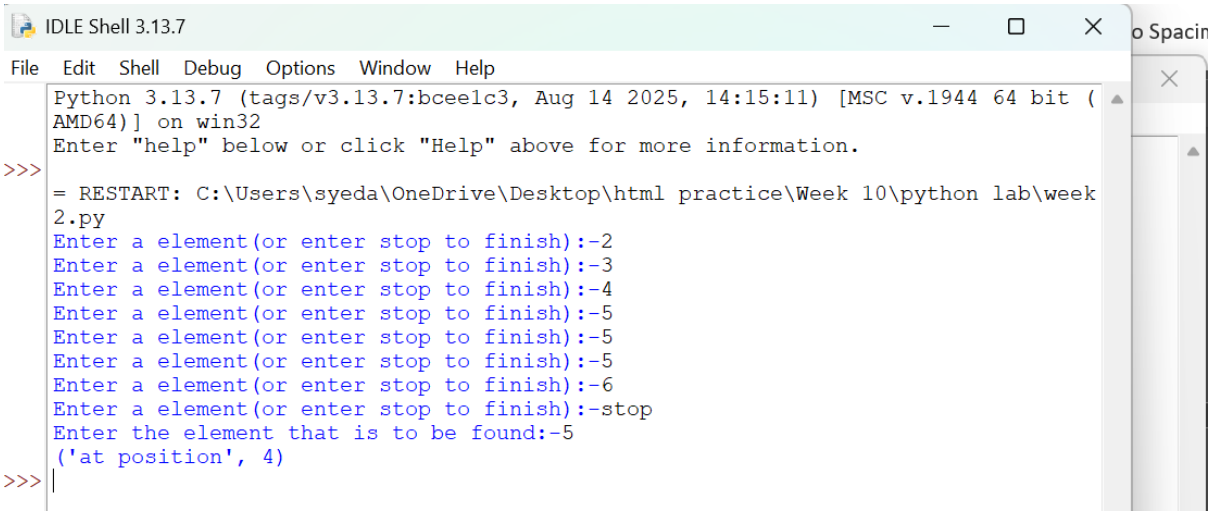
Week-2

1)Aim:- Write a program to perform linear search on the given input.

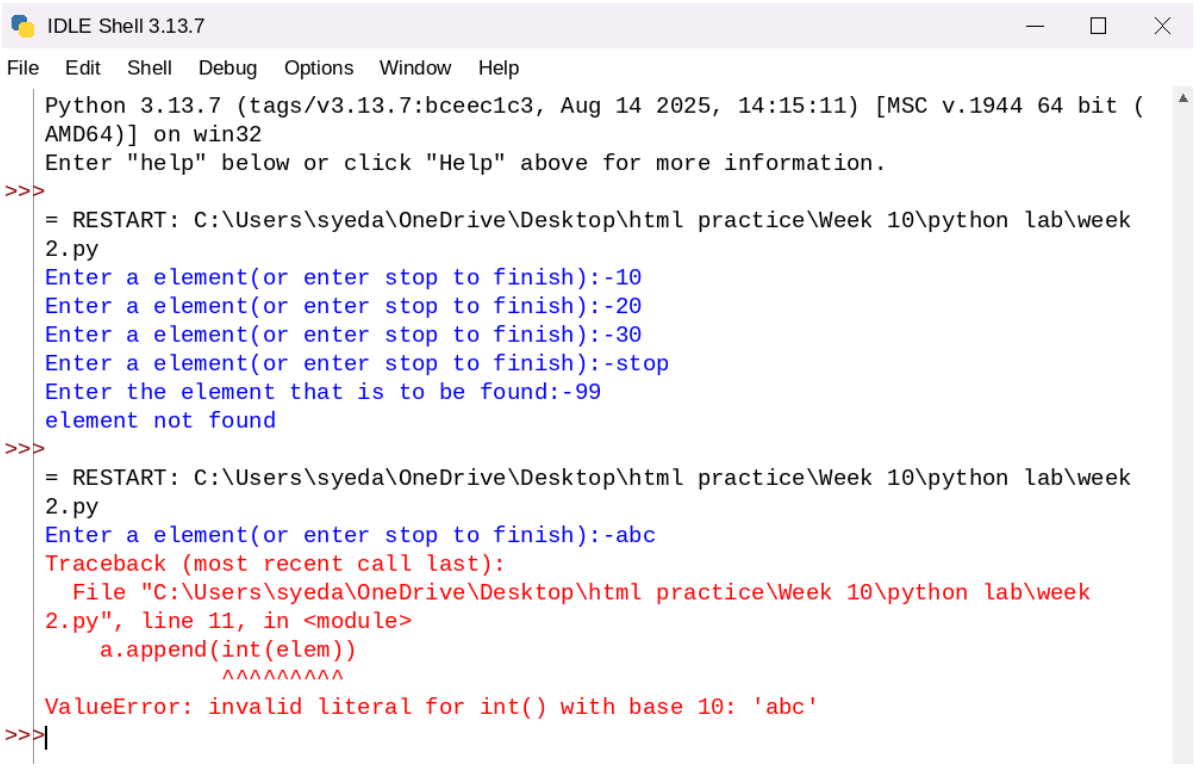
Program:-

```
def linear_search(arr,target):
    for i in range(len(arr)):
        if arr[i]==target:
            return "at position",i+1
    return "element not found"
a=[]
while True:
    elem=input("Enter a element(or enter stop to finish):-")
    if elem.lower()=="stop":
        break
    a.append(int(elem))
b=int(input("Enter the element that is to be found:-"))
print(linear_search(a,b))
```

Output:-



Negative Case:



Explanation:-

This program uses a linear search technique to find a target element in a list.

1. The function linear_search(arr, target) is defined with two parameters:
   - arr: The list in which the search is performed.
   - target: The element to be searched.
2. A for loop is used to traverse each element of the list from index 0 to len(arr) - 1.
3. The statement if arr[i] == target: compares each element of the list with the target element.
4. If the element is found, the function returns "at position", i + 1, which gives the position of the element (starting from 1).
5. If the loop completes without finding the element, the function returns "element not found".
6. An empty list a is created, and the user is asked to input elements continuously.
7. The loop continues until the user enters "stop", after which the input process ends.
8. The user then enters the element to be searched, which is stored in variable b.
9. The function is called using linear_search(a, b) and the result is printed.

Error Table and negatives where the code doesn't give the effective result:-

S.No	Error Obtained	Rectification

01	ValueError when entering non-integer input	Ensure only integer values are entered or handle using
02	Program crashes if empty input is given before converting to int	Validate input before conversion
03	Element not found even if list is empty	Check if list is empty before searching
04	If duplicate elements exist, only first occurrence is returned	Modify logic if all occurrences are needed
05	Program does not handle very large lists efficiently	Use better algorithms like binary search (for sorted lists)

2)Aim:-Write a python program to perform binary search on the given input verify by taking sorted array and unsorted array as a input:-

Program:-

```
def binary_search(arr,target):
    low=0
    high=(len(arr))-1

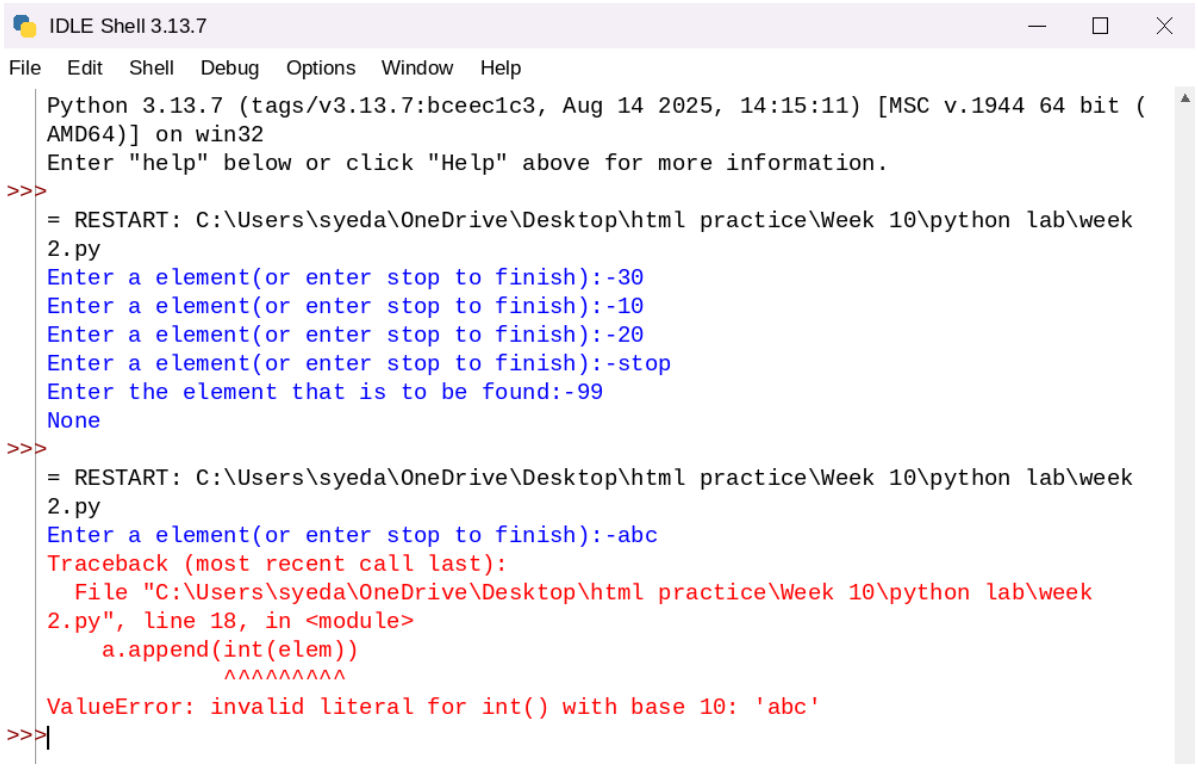
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==target:
            return "at position",mid
        elif target>arr[mid]:
            low,high=mid+1,high
        else:
            low,high=low,mid-1
a=[]
while True:
    elem=input("Enter a element(or enter stop to finish):-")
```

```
    if elem.lower()=="stop":
        break
    a.append(int(elem))
b=int(input("Enter the element that is to be found:-"))
a.sort()
print(binary_search(a,b))
```

Output:-

```
('at position', 4)
>>>
= RESTART: C:\Users\syeda\OneDrive\Desktop\html practice\Week 10\python lab\week
2.py
Enter a element(or enter stop to finish):-1
Enter a element(or enter stop to finish):-4
Enter a element(or enter stop to finish):-2
Enter a element(or enter stop to finish):-3
Enter a element(or enter stop to finish):-5
Enter a element(or enter stop to finish):-6
Enter a element(or enter stop to finish):-stop
Enter the element that is to be found:-5
('at position', 4)
>>>
= RESTART: C:\Users\syeda\OneDrive\Desktop\html practice\Week 10\python lab\week
2.py
Enter a element(or enter stop to finish):-1
Enter a element(or enter stop to finish):-2
Enter a element(or enter stop to finish):-3
Enter a element(or enter stop to finish):-4
Enter a element(or enter stop to finish):-5
Enter a element(or enter stop to finish):-6
Enter a element(or enter stop to finish):-stop
Enter the element that is to be found:-5
('at position', 4)
>>>
```

Negative Case:



Explanation:-

This program uses the binary search technique to find a target element in a list.

1. **The function binary_search(arr, target) is defined to perform the search operation.**
2. **Two variables are initialized:**
   - **low = 0: Represents the starting index of the list.**
   - **high = len(arr) - 1: Represents the last index of the list.**
3. **A while loop runs as long as low <= high, ensuring the search space is valid.**
4. **The middle index is calculated using mid = (low + high) // 2.**
5. **The statement if arr[mid] == target: checks whether the middle element is equal to the target.**
   - **If true, the function returns "at position", mid.**
6. **If the target is greater than the middle element (target > arr[mid]), the search continues in the right half by updating low = mid + 1.**
7. **Otherwise, the search continues in the left half by updating high = mid - 1.**
8. **The user inputs elements into the list a until "stop" is entered.**
9. **The list is sorted using a.sort().**
   - **This ensures that even if the user enters an unsorted array, the binary search will still work correctly.**
10. **The user enters the element to be searched, stored in variable b.**
11. **The function is called using binary_search(a, b) and the result is printed.**

Sno	Error Obtained	Rectification

01	**ValueError when non-integer input is entered**	**Ensure inputs are integers or use**
02	Incorrect position (0-based index returned)	Use mid + 1 if position should start from 1
03	Program fails if list is empty	Check if list is empty before searching
04	Binary search requires sorted array	Sorting is already handled using
05	Element not found but no explicit message returned	Add return "element not found" after loop
06	Output format shows tuple like ('at position', 2)	Use formatted print statement for better output
07	Duplicate elements return only one occurrence	Modify logic if all positions are needed
08	Sorting changes original order of elements	Inform user that positions are based on sorted list

Week-3

3a)Aim:- Write a Python program to perform Bubble Sort on a given list of elements.

Program:-

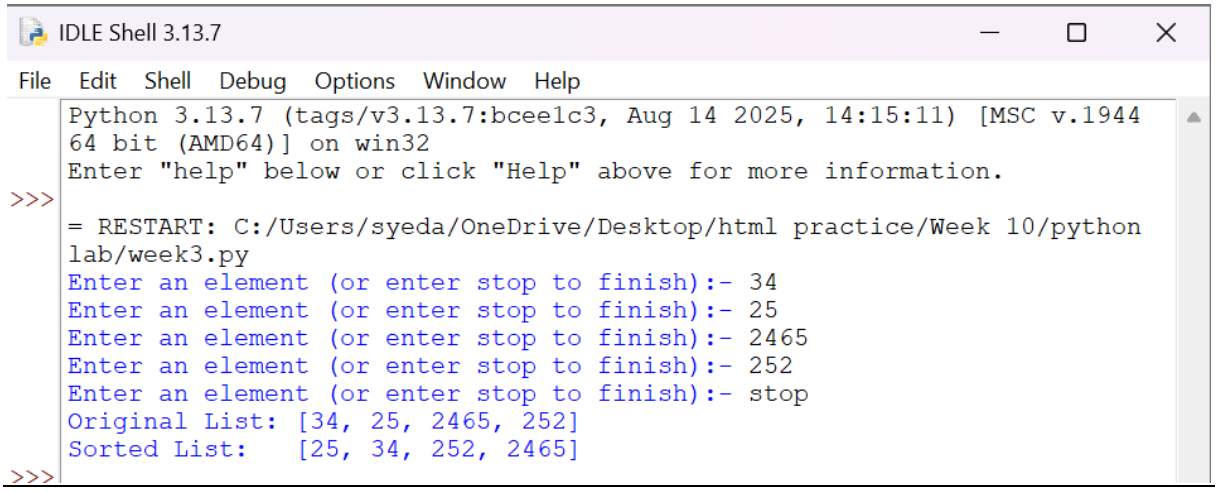
```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

a = []
```

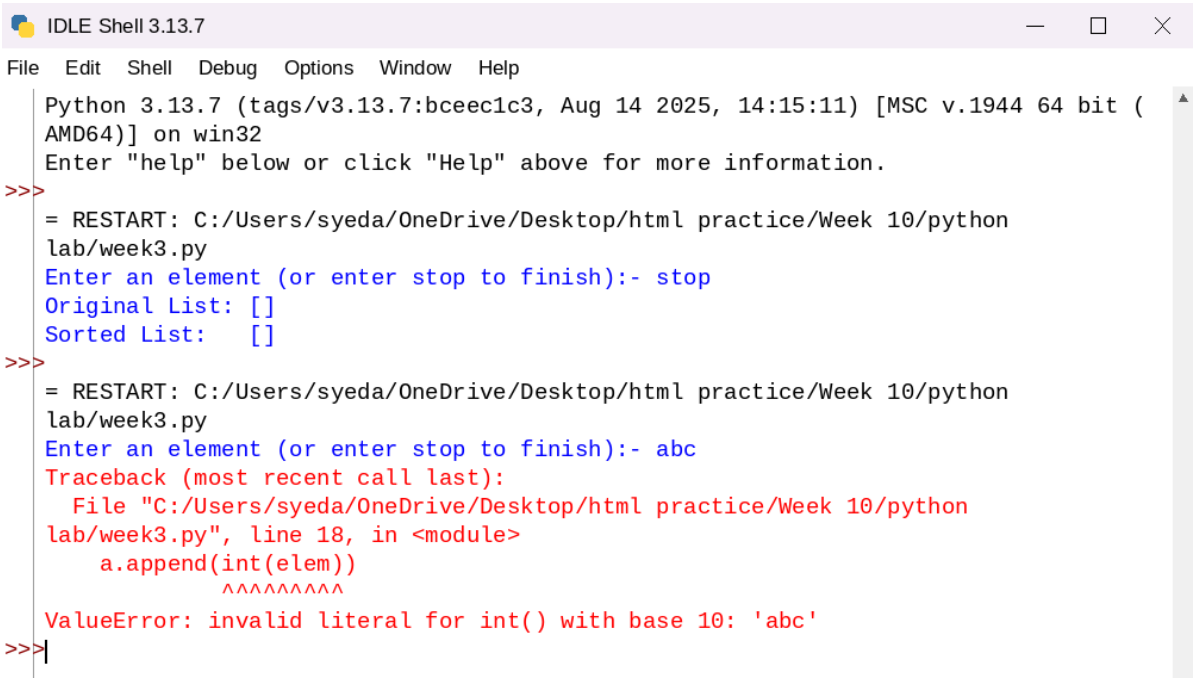
```
while True:
    elem = input("Enter an element (or enter stop to finish):- ")
    if elem.lower() == "stop":
        break
    a.append(int(elem))

print("Original List:", a)
sorted_list = bubble_sort(a)
print("Sorted List:  ", sorted_list)
```

OUTPUT:-



Negative Case:



Explanation:

This program uses the Bubble Sort technique to sort elements in a list.

1. The function bubble_sort(arr) is defined to sort the given list arr.
2. The variable n = len(arr) stores the number of elements in the list.
3. A loop runs from i = 0 to n-1, representing each pass of the sorting process.
4. A variable swapped is initialized as False at the beginning of each pass.
    - It is used to check whether any swapping occurred during the pass.
5. Another loop runs from j = 0 to n-i-2, comparing adjacent elements.
6. The condition if arr[j] > arr[j + 1]: checks if the current element is greater than the next element.
    - If true, the elements are swapped using arr[j], arr[j + 1] = arr[j + 1], arr[j].
    - The variable swapped is set to True.
7. After each pass, if no swapping occurs (swapped == False), the loop breaks early.
    - This indicates that the list is already sorted.
8. The function returns the sorted list.
9. The user inputs elements into the list a until "stop" is entered.
10. The original list is printed, followed by the sorted list after calling bubble_sort(a).

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Ensure input is integer or handle using try-except

S.No	Error Obtained	Rectification
02.	Program fails for empty list (no elements entered)	Check if list is empty before sorting
03.	Sorting modifies original list	Use a copy of list if original order must be preserved
04.	Inefficient for large inputs (high time complexity)	Use faster algorithms like merge sort or quick sort

3b)

Aim:-

Write a Python program to perform Insertion Sort on a given list of elements.

Program:-

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

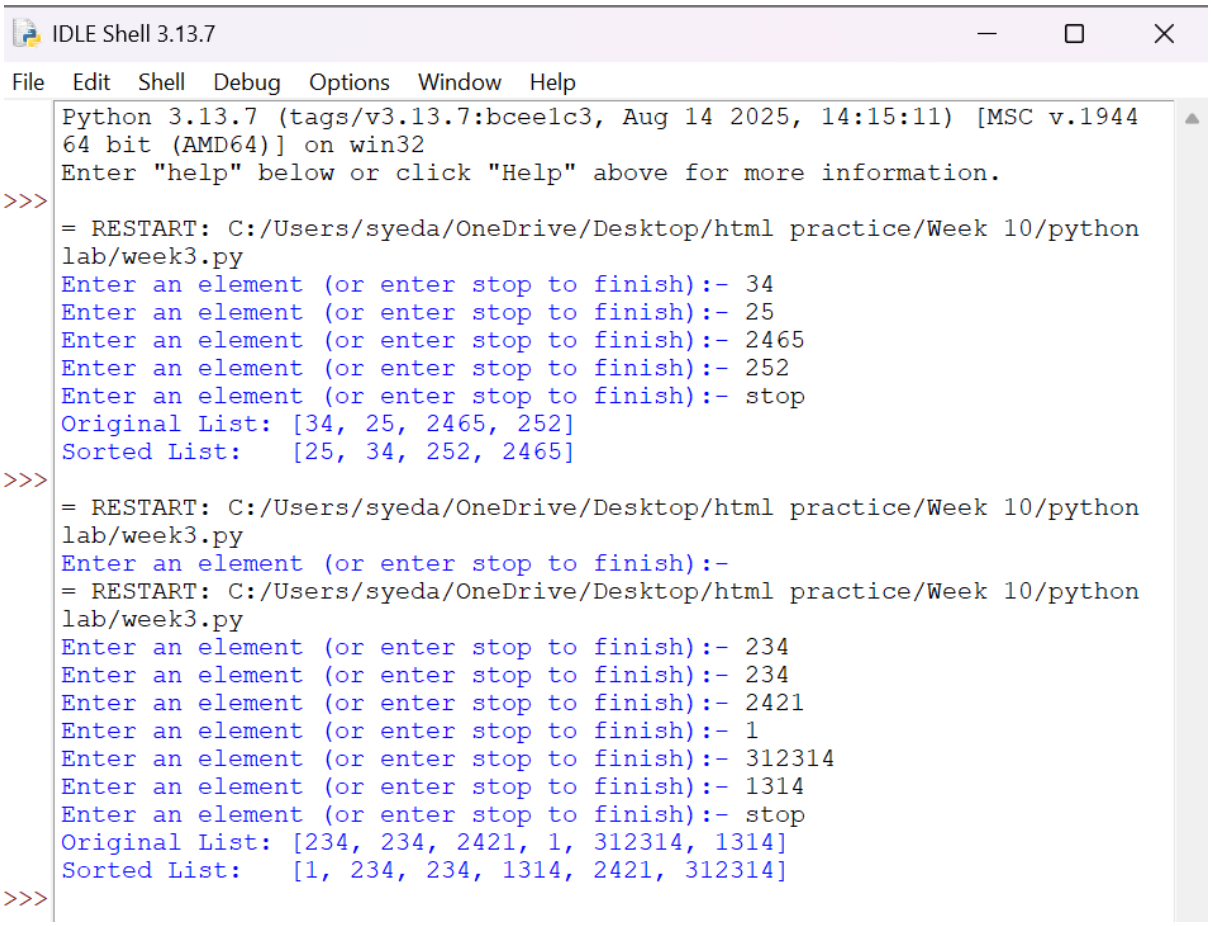

a = []
while True:
    elem = input("Enter an element (or enter stop to finish):- ")
    if elem.lower() == "stop":
        break
    a.append(int(elem))
```

```
print("Original List:", a)

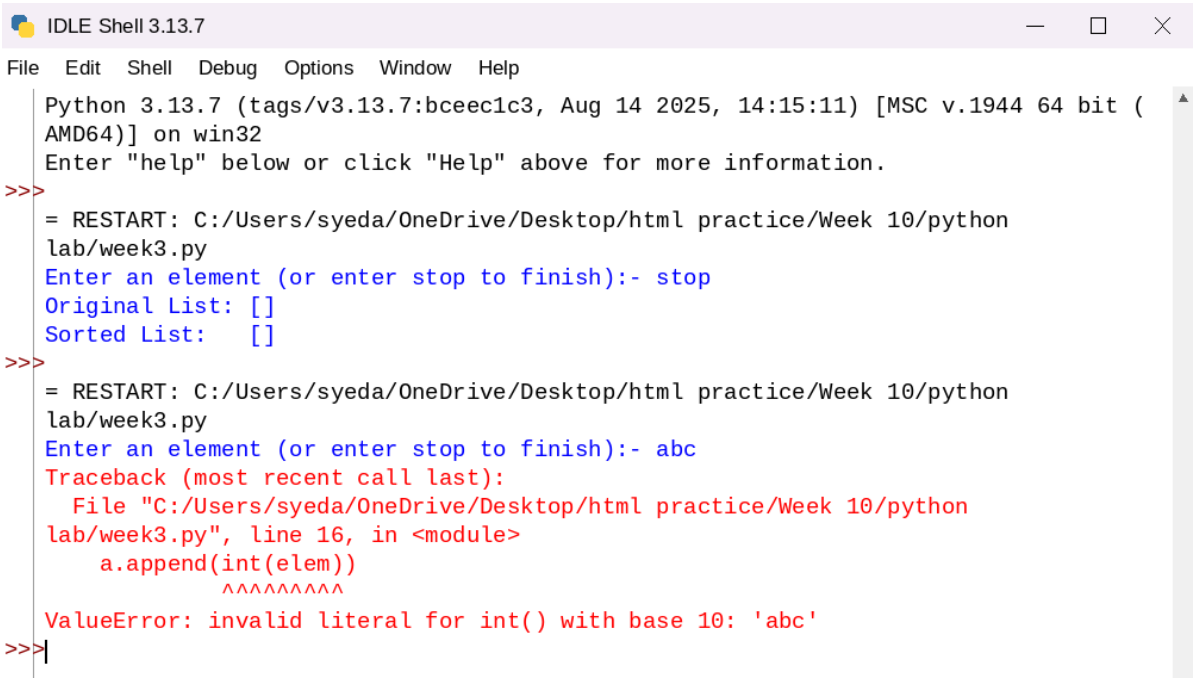
sorted_list = insertion_sort(a)

print("Sorted List:  ", sorted_list)
```

Output:-



Negative Case:



Explanation:

This program uses the Insertion Sort technique to sort elements in a list.

1. The function insertion_sort(arr) is defined to sort the given list arr.
2. A loop runs from i = 1 to len(arr) - 1, considering one element at a time as the key element.
3. The variable key = arr[i] stores the current element to be inserted into the sorted portion of the list.
4. The variable j = i - 1 points to the last element of the already sorted portion.
5. A while loop runs as long as j >= 0 and key < arr[j].
    - This shifts elements one position ahead using arr[j + 1] = arr[j].
    - The value of j is decreased by 1 after each shift.
6. After shifting all larger elements, the key is placed at the correct position using arr[j + 1] = key.
7. This process continues until the entire list is sorted.
8. The function returns the sorted list.
9. The user inputs elements into the list a until "stop" is entered.
10. The original list is printed, followed by the sorted list after calling insertion_sort(a).

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Ensure input is integer or handle using try-except

S.No	Error Obtained	Rectification
02.	Program fails for empty list	Check if list is empty before sorting
03.	Sorting modifies original list	Use a copy of list if original order must be preserved
04.	Incorrect output if comparison condition is wrong (key > arr[j])	Ensure correct condition key < arr[j] is used

3c)

Aim:-

Write a Python program to perform Selection Sort on a given list of elements.

Program:-

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

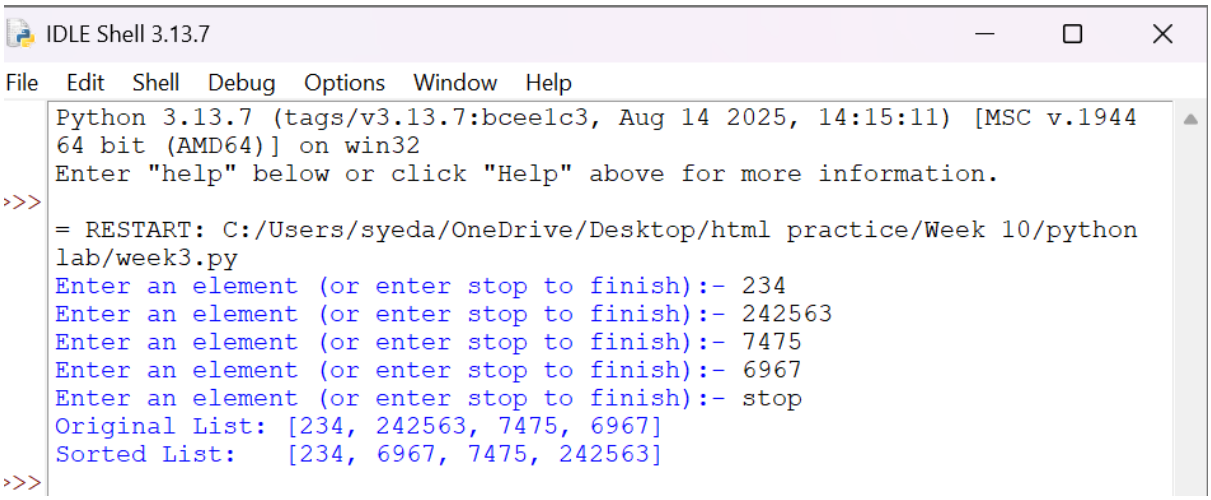

a = []
while True:
    elem = input("Enter an element (or enter stop to finish):- ")
    if elem.lower() == "stop":
        break
    a.append(int(elem))
```

```
print("Original List:", a)

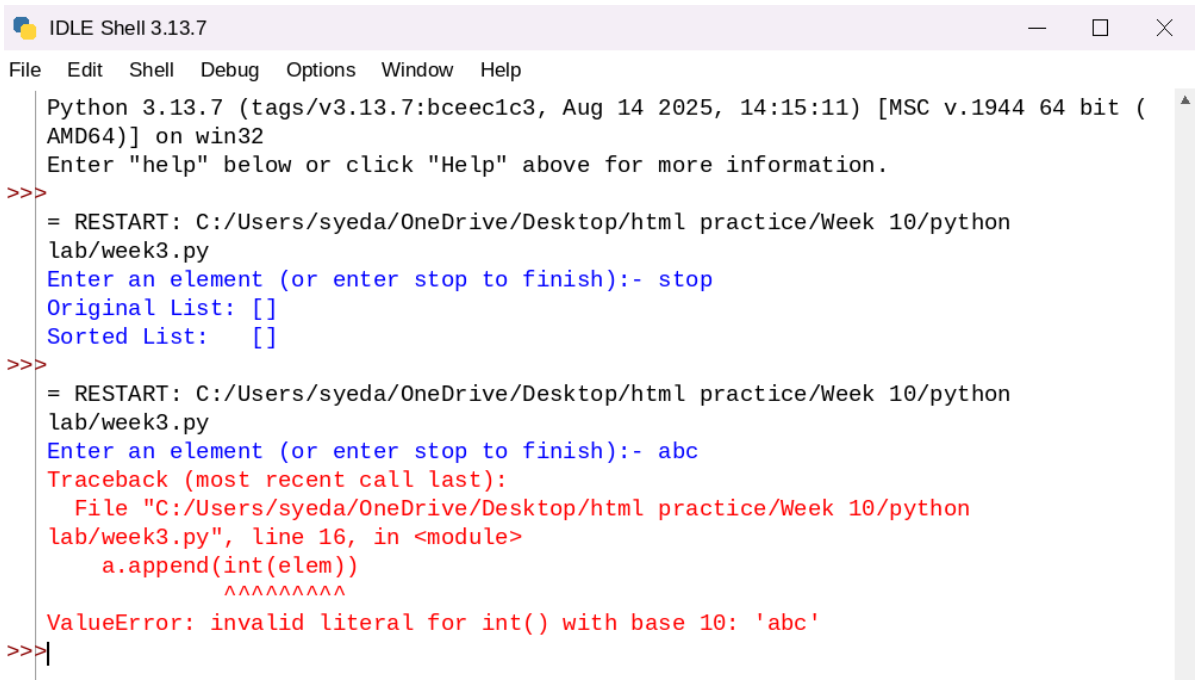
sorted_list = selection_sort(a)

print("Sorted List:  ", sorted_list)
```

Output:-



Negative Case:



Explanation:

This program uses the Selection Sort technique to sort elements in a list.

1. The function selection_sort(arr) is defined to sort the given list arr.
2. The variable n = len(arr) stores the number of elements in the list.
3. A loop runs from i = 0 to n-1, representing the position where the smallest element should be placed.
4. The variable min_idx = i is initialized to store the index of the minimum element in the unsorted portion.
5. Another loop runs from j = i + 1 to n-1, comparing elements in the unsorted part of the list.
6. The condition if arr[j] < arr[min_idx]: checks if a smaller element is found.
    - If true, min_idx is updated with the index of that smaller element.
7. After finding the smallest element, it is swapped with the element at position i using arr[i], arr[min_idx] = arr[min_idx], arr[i].
8. This process continues until the entire list is sorted.
9. The function returns the sorted list.
10. The user inputs elements into the list a until "stop" is entered.
11. The original list is printed, followed by the sorted list after calling selection_sort(a).

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Ensure input is integer or handle using try-except
02.	Program fails for empty list	Check if list is empty before sorting
03.	Sorting modifies original list	Use a copy of list if original order must be preserved
04.	Incorrect sorting if comparison condition is wrong (arr[j] > arr[min_idx])	Ensure correct condition arr[j] < arr[min_idx] is used

3d)

Aim:-

Write a Python program to perform Merge Sort on a given list of elements.

Program:-

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        L = arr[:mid]
```

```
R = arr[mid:]


merge_sort(L)
merge_sort(R)


i = j = k = 0


while i < len(L) and j < len(R):
    if L[i] <= R[j]:
        arr[k] = L[i]
        i += 1
    else:
        arr[k] = R[j]
        j += 1
    k += 1


while i < len(L):
    arr[k] = L[i]
    i += 1
    k += 1


while j < len(R):
    arr[k] = R[j]
    j += 1
    k += 1
```

```
    return arr

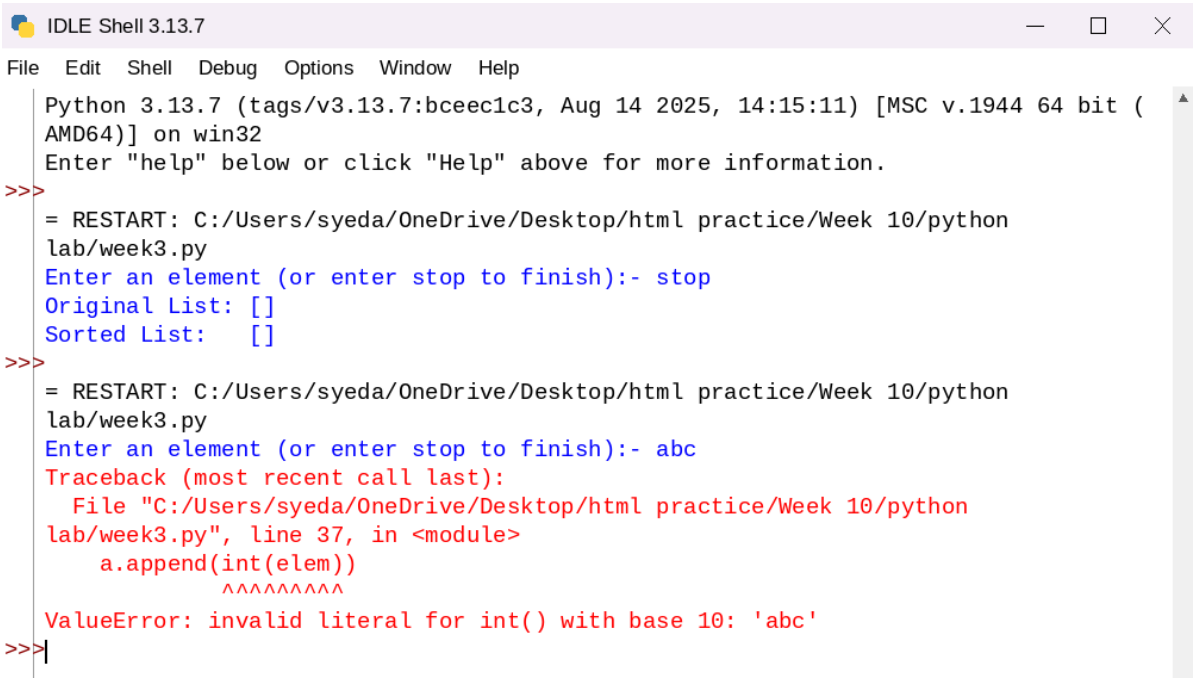

a = []
while True:
    elem = input("Enter an element (or enter stop to finish):- ")
    if elem.lower() == "stop":
        break
    a.append(int(elem))


print("Original List:", a)
sorted_list = merge_sort(a)
print("Sorted List:  ", sorted_list)
```

Output:-

```
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python
lab/week3.py
Enter an element (or enter stop to finish):- 242
Enter an element (or enter stop to finish):- 2425
Enter an element (or enter stop to finish):- 2534
Enter an element (or enter stop to finish):- 3635
Enter an element (or enter stop to finish):- 252
Enter an element (or enter stop to finish):- stop
Original List: [242, 2425, 2534, 3635, 252]
Sorted List:   [242, 252, 2425, 2534, 3635]
```

Negative Case:



Explanation:

This program uses the Merge Sort technique to sort elements in a list using recursion.

1. The function merge_sort(arr) is defined to sort the given list arr.
2. The condition if len(arr) > 1: checks whether the list has more than one element.
    - If not, the list is already sorted (base case).
3. The list is divided into two halves:
    - mid = len(arr) // 2 finds the middle index.
    - L = arr[:mid] stores the left half.
    - R = arr[mid:] stores the right half.
4. The function recursively calls itself on both halves using merge_sort(L) and merge_sort(R).
5. Three variables i, j, and k are initialized to 0:
    - i → index for left sublist
    - j → index for right sublist
    - k → index for merged list
6. A while loop compares elements of L and R:
    - The smaller element is placed into arr[k].
    - Corresponding indices are incremented.
7. After one sublist is exhausted, remaining elements of the other sublist are copied using two separate while loops.
8. The sorted elements are merged back into the original list arr.
9. The function returns the sorted list.
10. The user inputs elements into the list a until "stop" is entered.
11. The original list is printed, followed by the sorted list after calling merge_sort(a).

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Ensure input is integer or handle using try-except
02.	Program fails for empty list	Check if list is empty before sorting
03.	Incorrect merging if indices (i, j, k) are not updated properly	Ensure all indices are incremented correctly
04.	High memory usage due to creating sublists (L and R)	Use optimized in-place merge sort if required

3e)

Aim:- Write a Python program to perform Quick Sort on a given list of elements.

Program:-

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1

    if low < high:
        pivot = arr[high]
        i = low - 1

        for j in range(low, high):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        pi = i + 1
        quick_sort(arr, low, pi - 1)
```

```
        quick_sort(arr, pi + 1, high)


    return arr

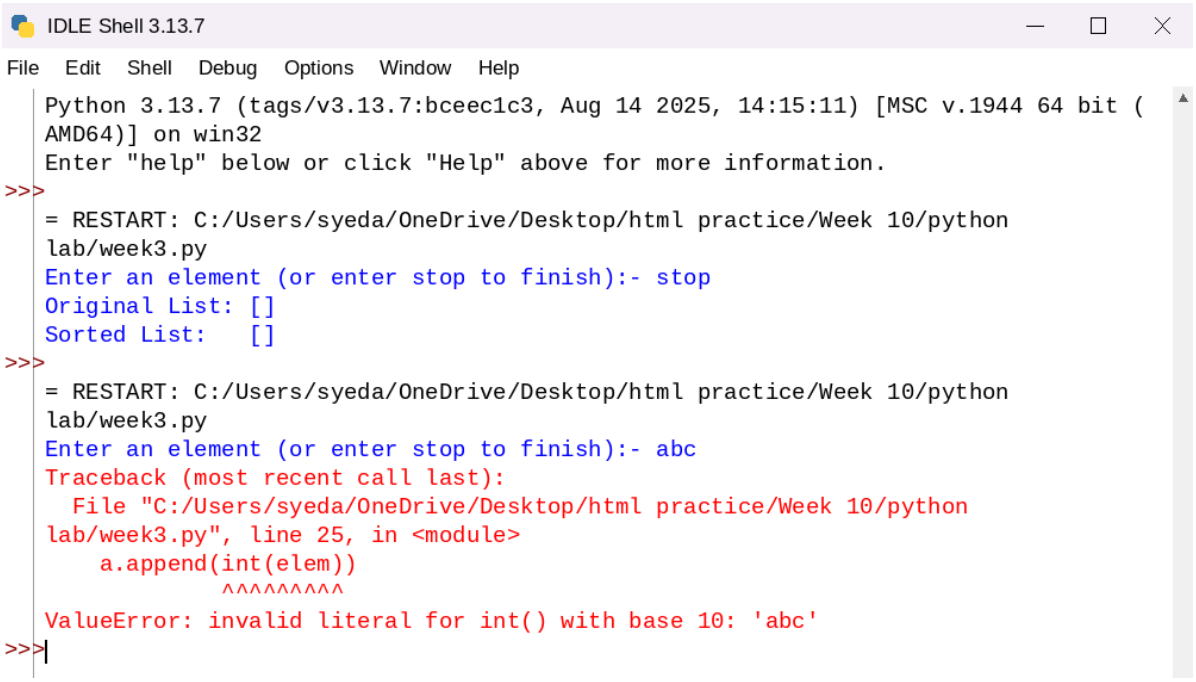

a = []
while True:
    elem = input("Enter an element (or enter stop to finish):- ")
    if elem.lower() == "stop":
        break
    a.append(int(elem))


print("Original List:", a)
sorted_list = quick_sort(a)
print("Sorted List:  ", sorted_list)
```

Output:-

```
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python
lab/week3.py
Enter an element (or enter stop to finish):- 2425
Enter an element (or enter stop to finish):- 34
Enter an element (or enter stop to finish):- 646
Enter an element (or enter stop to finish):- 6
Enter an element (or enter stop to finish):- 36
Enter an element (or enter stop to finish):- 3466
Enter an element (or enter stop to finish):- stop
Original List: [2425, 34, 646, 6, 36, 3466]
Sorted List:   [6, 34, 36, 646, 2425, 3466]
```

Negative Case:



Explanation:

This program uses the Quick Sort technique to sort elements in a list using recursion.

1. The function quick_sort(arr, low=0, high=None) is defined with parameters:
    - arr: The list to be sorted
    - low: Starting index (default 0)
    - high: Ending index (default last index of list)
2. If high is None, it is initialized as len(arr) - 1.
3. The condition if low < high: ensures that sorting is performed only when there are at least two elements.
4. The last element arr[high] is chosen as the **pivot**.
5. The variable i = low - 1 is used to track the position of smaller elements.
6. A loop runs from j = low to high - 1:
    - If arr[j] <= pivot, i is incremented and elements are swapped using arr[i], arr[j] = arr[j], arr[i].
7. After the loop, the pivot is placed at its correct position using arr[i + 1], arr[high] = arr[high], arr[i + 1].
8. The pivot index pi = i + 1 divides the array into two parts.
9. The function recursively sorts:
    - Left subarray → quick_sort(arr, low, pi - 1)
    - Right subarray → quick_sort(arr, pi + 1, high)
10. The function returns the sorted list.
11. The user inputs elements into list a until "stop" is entered.
12. The original list and sorted list are printed.

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Ensure input is integer or handle using try-except
02.	Program fails for empty list	Check if list is empty before sorting
03.	Worst-case performance ($O(n^2)$) for already sorted list	Use better pivot selection (e.g., middle or random pivot)
04.	Recursion depth error for very large lists	Increase recursion limit or use iterative approach

Week-4

4c)Aim:-write a python program to implement circular linked list which could perform below operations by taking input from the user

1)create linked list

2)Insert beginning

3)insert end

4)Insert at specific index

5)Delete by value

6)Delete first node

7)Delete last node

8)count number of nodes

9)Display/Traverse

10)display head to tail

11)Print the data from tail to head

10)exit

Program:

```
class Node:
    def __init__(self, data):
```

```
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.tail = None

    # Create list
    def create(self):
        n = int(input("Enter number of elements: "))
        for i in range(n):
            x = int(input("Enter value: "))
            self.insert_end(x)

    # Insert at beginning
    def insert_begin(self, data):
        new = Node(data)
        if self.tail is None:
            self.tail = new
            new.next = new
        else:
            new.next = self.tail.next
            self.tail.next = new
```

```
# Insert at end
def insert_end(self, data):
    new = Node(data)
    if self.tail is None:
        self.tail = new
        new.next = new
    else:
        new.next = self.tail.next
        self.tail.next = new
        self.tail = new

# Insert at index
def insert_index(self, index, data):
    if index < 0:
        print("Invalid Index")
        return
    if index == 0:
        self.insert_begin(data)
        return
    if self.tail is None:
        print("Invalid Index")
        return

    temp = self.tail.next
    for i in range(index - 1):
```

```
            temp = temp.next
            if temp == self.tail.next:
                print("Invalid Index")
                return

        if temp == self.tail:
            self.insert_end(data)
            return

        new = Node(data)
        new.next = temp.next
        temp.next = new

    # Delete from beginning
    def deleteAtBeg(self):
        if self.tail is None:
            print("No Data to delete")
            return

        if self.tail.next == self.tail:
            self.tail = None
            print("Deleted Value from beginning")
            return

        head = self.tail.next
```

```
    self.tail.next = head.next
    print("Deleted Value =", head.data)

# Delete from end
def deleteAtEnd(self):
    if self.tail is None:
        print("No Data to delete")
        return

    if self.tail.next == self.tail:
        self.tail = None
        print("Deleted Value from end")
        return

    temp = self.tail.next
    while temp.next != self.tail:
        temp = temp.next

    deleted = self.tail.data
    temp.next = self.tail.next
    self.tail = temp
    print("Deleted Value =", deleted)

# Delete by value
def delete(self, value):
```

```
if self.tail is None:
    print("No Data to delete")
    return

prev = self.tail
current = self.tail.next

while True:
    if current.data == value:
        if current == self.tail:
            if current.next == current:
                self.tail = None
            else:
                prev.next = current.next
                self.tail = prev
        else:
            prev.next = current.next
        print("Value deleted")
        return
    prev = current
    current = current.next
    if current == self.tail.next:
        break

print("Value not present")
```

```
# Search value
def search(self, value):
    if self.tail is None:
        print("No Data")
        return

    temp = self.tail.next
    index = 0
    while True:
        if temp.data == value:
            print("Value found at index", index)
            return
        temp = temp.next
        index += 1
        if temp == self.tail.next:
            break
    print("Value not found")

# Update value
def update(self, old_value, new_value):
    if self.tail is None:
        print("No Data")
        return
```

```
    temp = self.tail.next
    while True:
        if temp.data == old_value:
            temp.data = new_value
            print("Value updated")
            return
        temp = temp.next
        if temp == self.tail.next:
            break
    print("Value not found")


# Reverse list
def reverse(self):
    if self.tail is None:
        print("No Data")
        return


    values = []
    temp = self.tail.next
    while True:
        values.append(temp.data)
        temp = temp.next
        if temp == self.tail.next:
            break
```

```
    self.tail = None
    for value in reversed(values):
        self.insert_end(value)
    print("List reversed")

# Display list
def display(self):
    if self.tail is None:
        print("No Data")
        return

    temp = self.tail.next
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.tail.next:
            break
    print("head")

# Count nodes
def count(self):
    if self.tail is None:
        return 0

    c = 1
```

```
        temp = self.tail.next
        while temp != self.tail:
            c += 1
            temp = temp.next
        return c
# Display from head to tail (already similar to display)
def display_head_to_tail(self):
    if self.tail is None:
        print("No Data")
        return

    temp = self.tail.next   # head
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.tail.next:
            break
    print("head")


# Display from tail to head
def display_tail_to_head(self):
    if self.tail is None:
        print("No Data")
        return
```

```
    values = []
    temp = self.tail.next   # start from head

    while True:
        values.append(temp.data)
        temp = temp.next
        if temp == self.tail.next:
            break

    # print in reverse order
    for val in reversed(values):
        print(val, end=" -> ")
    print("tail")



# Menu-driven program
cll = CircularLinkedList()

while True:
    print("\n1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
```

```
print("6. Delete First Node")
print("7. Delete Last Node")
print("8. Count Nodes")
print("9. Display")
print("10. Search Value")
print("11. Update Value")
print("12. Reverse List")
print("13. Exit")
print("14. Display Head to Tail")
print("15. Display Tail to Head")

ch = int(input("Choice: "))

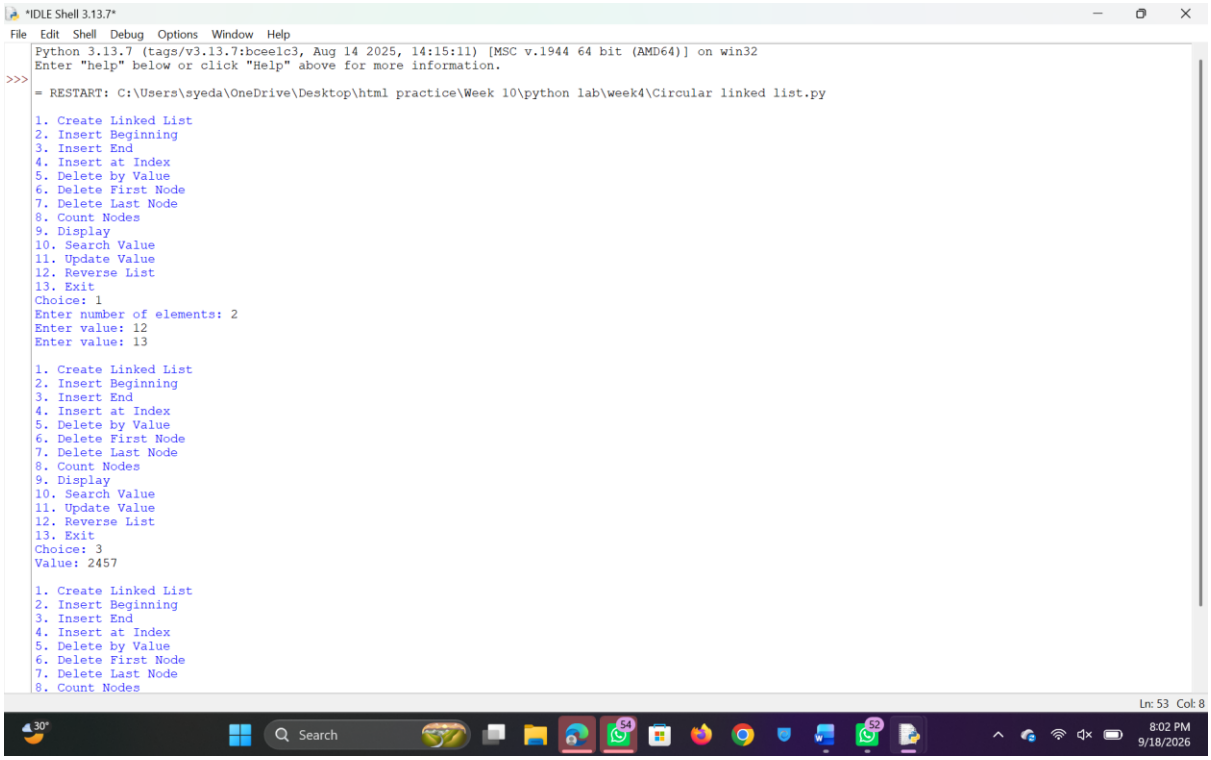
if ch == 1:
    cll.create()
elif ch == 2:
    x = int(input("Value: "))
    cll.insert_begin(x)
elif ch == 3:
    x = int(input("Value: "))
    cll.insert_end(x)
elif ch == 4:
    idx = int(input("Index: "))
    x = int(input("Value: "))
    cll.insert_index(idx, x)
```

```
elif ch == 5:
    x = int(input("Delete value: "))
    cll.delete(x)
elif ch == 6:
    cll.deleteAtBeg()
elif ch == 7:
    cll.deleteAtEnd()
elif ch == 8:
    print("Number of nodes =", cll.count())
elif ch == 9:
    cll.display()
elif ch == 10:
    x = int(input("Search value: "))
    cll.search(x)
elif ch == 11:
    old = int(input("Old value: "))
    new = int(input("New value: "))
    cll.update(old, new)
elif ch == 12:
    cll.reverse()
elif ch == 14:
cll.display_head_to_tail()
elif ch == 15:
cll.display_tail_to_head()
elif ch == 13:
```

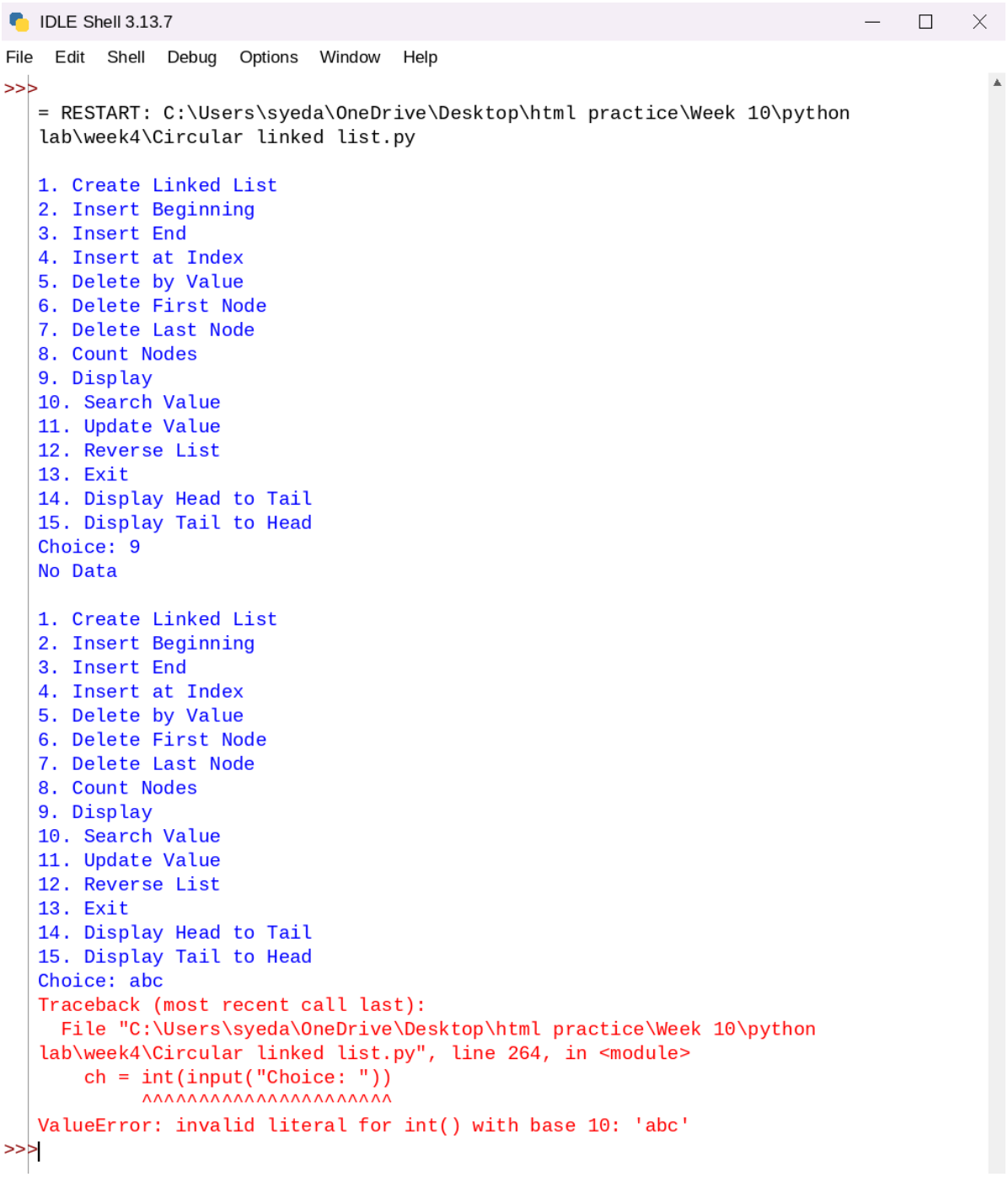
```
        break
    else:
        print("Invalid choice")


if __name__ == "__main__":
    pass
```

Output:



Negative Case:



Explanation:

This program implements a **Circular Linked List** using Python and performs various operations like insertion, deletion, searching, updating, reversing, and displaying elements using a menu-driven approach.

1. **Node Class Creation:**
   The class Node is defined with two attributes:
   - data → stores the value

- next → stores the reference to the next node

2. **Circular Linked List Initialization:**
   The class CircularLinkedList is defined with a tail pointer.
   - tail.next always points to the **head node**, maintaining circular nature.
3. **Creating the List:**
   The function create() takes input from the user and inserts elements at the end using insert_end().
4. **Insertion at Beginning:**
   - If list is empty → new node points to itself.
   - Otherwise → new node is inserted before head and tail is updated accordingly.
5. **Insertion at End:**
   - New node is added after tail.
   - Tail pointer is updated to the new node.
   - Circular link is maintained.
6. **Insertion at Index:**
   - Validates index.
   - Traverses list to required position.
   - Inserts node at specified position.
   - Handles edge cases like insertion at beginning or end.
7. **Deletion at Beginning:**
   - If only one node → list becomes empty.
   - Otherwise → head node is removed and tail points to new head.
8. **Deletion at End:**
   - Traverses list to find node before tail.
   - Updates tail and maintains circular link.
9. **Deletion by Value:**
   - Traverses list to find the given value.
   - Deletes node by adjusting previous node's pointer.
   - Handles deletion of tail and single-node list.
10. **Search Operation:**
    - Traverses the list and compares each node's data.
    - Displays index if value is found.
11. **Update Operation:**
    - Searches for given value.
    - Replaces it with new value if found.
12. **Reverse Operation:**
    - Stores all node values in a list.
    - Rebuilds the linked list in reverse order.
13. **Display Operation:**
    - Traverses from head to tail.
    - Prints elements in circular format ending with "head".
14. **Count Operation:**
    - Traverses the list and counts number of nodes.
15. **Menu-Driven Execution:**
    - User selects operations using menu choices.
    - Corresponding functions are called based on input.

This program adds two additional display operations for a Circular Linked List.

10. Display Head to Tail:

1. The function display_head_to_tail() prints elements starting from the head node.
2. The head node is accessed using self.tail.next.
3. A temporary pointer temp is used to traverse the list.
4. A loop runs until it reaches the head again (circular condition).
5. Each node's data is printed in sequence.
6. Finally, "head" is printed to indicate circular linkage.

---

11. Display Tail to Head:

1. The function display_tail_to_head() prints elements from tail to head.
2. Since singly circular linked list cannot be traversed backward directly, values are first stored in a list.
3. The list is traversed from head using a temporary pointer.
4. All node values are appended into a Python list values.
5. The list is then reversed using reversed() function.
6. Elements are printed in reverse order, representing tail-to-head traversal.
7. Finally, "tail" is printed to indicate reverse direction.

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Use try-except to handle invalid input
02.	Invalid index during insertion	Check index range before inserting
03.	Deletion on empty list	Add condition to check if list is empty
04.	Infinite loop if circular link is broken	Ensure tail.next always points to head
05.	Value not found during delete/search/update	Display proper message if value not present
06.	Incorrect tail update during deletion	Ensure tail is updated correctly when last node is removed
07.	High memory usage in reverse function	Use pointer-based reversal instead of extra list
08.	Program crashes on invalid menu choice	Handle invalid choice using else condition

4b)Aim:-write a python program to implement doubly linked list which could perform below operations by taking input from the user

1)create linked list

2)Insert beginning

3)insert end

4)Insert at specific index

5)Delete by value

6)Delete first node

7)Delete last node

8)count number of nodes

9)Display/Traverse

10)exit

Program:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.previous = None
        self.next = None



class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    # Create list
```

```
def create(self):
    n = int(input("Enter number of elements: "))
    for i in range(n):
        x = int(input("Enter value: "))
        self.insert_end(x)

# Insert at beginning
def insert_begin(self, data):
    new = Node(data)
    if self.head is None:
        self.head = self.tail = new
    else:
        new.next = self.head
        self.head.previous = new
        self.head = new

# Insert at end
def insert_end(self, data):
    new = Node(data)
    if self.head is None:
        self.head = self.tail = new
    else:
        new.previous = self.tail
        self.tail.next = new
        self.tail = new
```

```
# Insert at index
def insert_index(self, index, data):
    if index < 0:
        print("Invalid Index")
        return
    if index == 0:
        self.insert_begin(data)
        return

    if self.head is None:
        print("Invalid Index")
        return

    temp = self.head
    for i in range(index):
        if temp is None:
            print("Invalid Index")
            return
        temp = temp.next

    if temp is None:
        self.insert_end(data)
        return
```

```
    new = Node(data)
    new.previous = temp.previous
    new.next = temp
    if temp.previous is not None:
        temp.previous.next = new
    else:
        self.head = new
    temp.previous = new

# Delete from beginning
def deleteAtBeg(self):
    if self.head is None:
        print("No Data to delete")
        return

    if self.head == self.tail:
        self.head = self.tail = None
    else:
        self.head = self.head.next
        self.head.previous = None
    print("Deleted Value from beginning")

# Delete from end
def deleteAtEnd(self):
    if self.tail is None:
```

```
            print("No Data to delete")
            return

        if self.head == self.tail:
            self.head = self.tail = None
        else:
            self.tail = self.tail.previous
            self.tail.next = None
        print("Deleted Value from end")

    # Delete by value
    def delete(self, value):
        if self.head is None:
            print("No Data to delete")
            return

        temp = self.head
        while temp is not None and temp.data != value:
            temp = temp.next

        if temp is None:
            print("Value not present")
            return

        if temp == self.head:
```

```
            self.deleteAtBeg()
            return

        if temp == self.tail:
            self.deleteAtEnd()
            return

        temp.previous.next = temp.next
        temp.next.previous = temp.previous
        print("Value deleted")

    # Delete by index
    def delete_at_index(self, index):
        if self.head is None:
            print("No Data to delete")
            return

        if index < 0:
            print("Invalid Index")
            return

        temp = self.head
        for i in range(index):
            if temp is None:
                print("Invalid Index")
```

```
            return
        temp = temp.next

    if temp is None:
        print("Invalid Index")
        return

    if temp == self.head:
        self.deleteAtBeg()
        return

    if temp == self.tail:
        self.deleteAtEnd()
        return

    temp.previous.next = temp.next
    temp.next.previous = temp.previous
    print("Deleted Value at index", index)

# Search value
def search(self, value):
    temp = self.head
    index = 0
    while temp is not None:
        if temp.data == value:
```

```
            print("Value found at index", index)
            return
        temp = temp.next
        index += 1
    print("Value not found")

# Update value
def update(self, old_value, new_value):
    temp = self.head
    while temp is not None:
        if temp.data == old_value:
            temp.data = new_value
            print("Value updated")
            return
        temp = temp.next
    print("Value not found")

# Reverse list
def reverse(self):
    if self.head is None:
        print("No Data")
        return

    current = self.head
    while current is not None:
```

```
            current.previous, current.next = current.next, current.previous
            self.head = current
            current = current.previous

        self.head, self.tail = self.tail, self.head
        print("List reversed")

    # Display list
    def display(self):
        if self.head is None:
            print("No Data")
            return

        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")

    # Count nodes
    def count(self):
        c = 0
        temp = self.head
        while temp:
            c += 1
```

```
            temp = temp.next
        return c


# Menu-driven program
dll = DoublyLinkedList()

while True:
    print("\n1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Nodes")
    print("9. Display")
    print("10. Search Value")
    print("11. Update Value")
    print("12. Reverse List")
    print("13. Exit")

    ch = int(input("Choice: "))

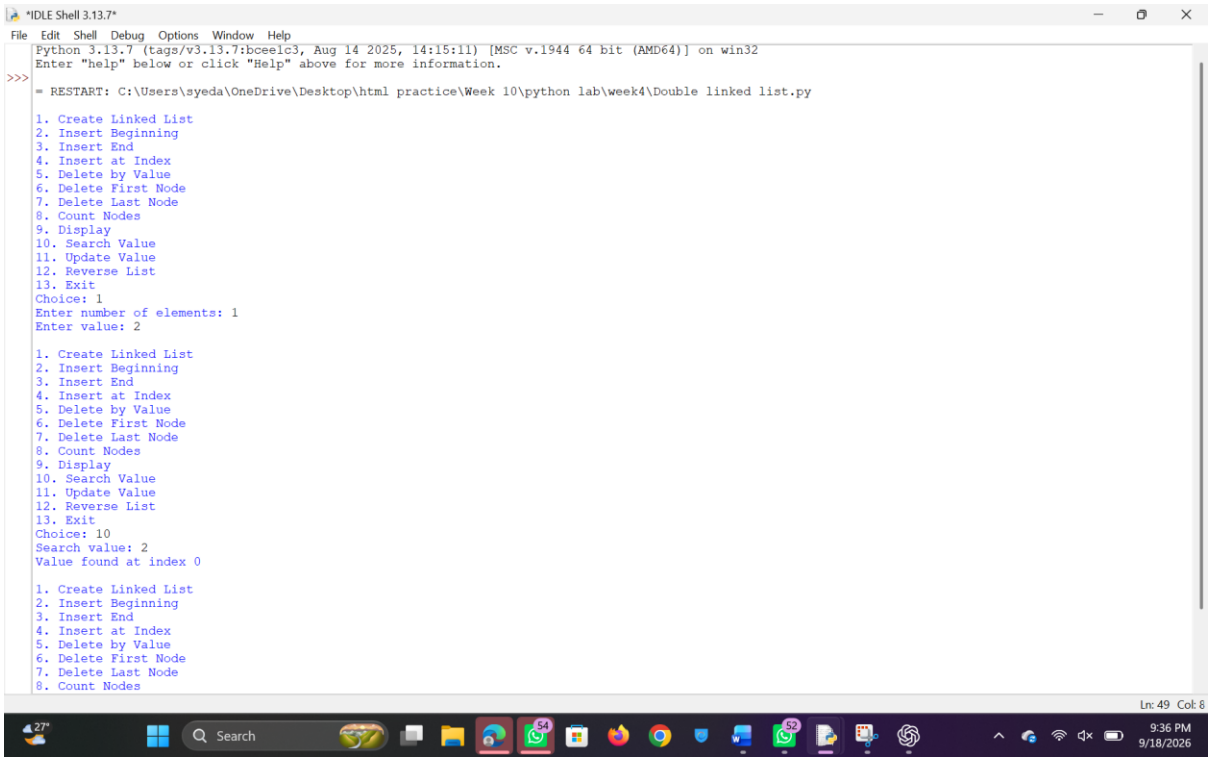
    if ch == 1:
```

```
        dll.create()
    elif ch == 2:
        x = int(input("Value: "))
        dll.insert_begin(x)
    elif ch == 3:
        x = int(input("Value: "))
        dll.insert_end(x)
    elif ch == 4:
        idx = int(input("Index: "))
        x = int(input("Value: "))
        dll.insert_index(idx, x)
    elif ch == 5:
        x = int(input("Delete value: "))
        dll.delete(x)
    elif ch == 6:
        dll.deleteAtBeg()
    elif ch == 7:
        dll.deleteAtEnd()
    elif ch == 8:
        print("Number of nodes =", dll.count())
    elif ch == 9:
        dll.display()
    elif ch == 10:
        x = int(input("Search value: "))
        dll.search(x)
```

```
    elif ch == 11:
        old = int(input("Old value: "))
        new = int(input("New value: "))
        dll.update(old, new)
    elif ch == 12:
        dll.reverse()
    elif ch == 13:
        break
    else:
        print("Invalid choice")


if __name__ == "__main__":
    pass
```

Output:-


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> = RESTART: C:\Users\syeda\OneDrive\Desktop\html practice\Week 10\python lab\week4\Double linked list.py
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Nodes
9. Display
10. Search Value
11. Update Value
12. Reverse List
13. Exit
Choice: 1
Enter number of elements: 1
Enter value: 2
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Nodes
9. Display
10. Search Value
11. Update Value
12. Reverse List
13. Exit
Choice: 10
Search value: 2
Value found at index 0
1. Create Linked List
2. Insert Beginning
3. Insert End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Nodes
Ln: 49 Col: 8
27°
Search
9:36 PM
9/18/2026

Negative Case:



Explanation:

This program implements a **Doubly Linked List** using Python and performs operations such as insertion, deletion, searching, updating, reversing, and displaying elements using a menu-driven approach.

1. **Node Class Creation:**
   The class Node contains:
   - data → stores value
   - previous → points to previous node
   - next → points to next node

2. **Doubly Linked List Initialization:**
   The class DoublyLinkedList maintains two pointers:
   - head → first node
   - tail → last node
3. **Creating the List:**
   The function create() takes input from the user and inserts elements at the end using insert_end().
4. **Insertion at Beginning:**
   - If list is empty → new node becomes head and tail.
   - Otherwise → new node is linked before head and head is updated.
5. **Insertion at End:**
   - New node is added after tail.
   - Tail pointer is updated accordingly.
6. **Insertion at Index:**
   - Validates index.
   - Traverses to required position.
   - Inserts node between two nodes by updating both previous and next pointers.
7. **Deletion at Beginning:**
   - If only one node → list becomes empty.
   - Otherwise → head is moved to next node and previous pointer is set to None.
8. **Deletion at End:**
   - If only one node → list becomes empty.
   - Otherwise → tail is moved to previous node and next pointer is set to None.
9. **Deletion by Value:**
   - Searches for the node containing the value.
   - Adjusts both previous and next pointers to remove the node.
10. **Deletion by Index:**
    - Traverses to given index.
    - Deletes node by updating adjacent node links.
11. **Search Operation:**
    - Traverses from head to tail.
    - Prints index if value is found.
12. **Update Operation:**
    - Searches for a value.
    - Replaces it with new value.
13. **Reverse Operation:**
    - Swaps previous and next pointers for each node.
    - Swaps head and tail pointers at the end.
14. **Display Operation:**
    - Traverses from head to tail.
    - Prints elements in forward direction.
15. **Count Operation:**
    - Traverses list and counts number of nodes.
16. **Menu-Driven Execution:**
    - User selects operation.
    - Corresponding function is executed.

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError for invalid input	Use `try-except` for input handling
02.	Invalid index during insertion/deletion	Validate index before operation
03.	Deletion on empty list	Check if `head is None`
04.	Incorrect pointer updates	Ensure both `next` and `previous` are updated
05.	Value not found during operations	Display proper message
06.	Infinite loop due to incorrect linking	Maintain proper `next` and `previous` connections
07.	Reverse not working correctly	Ensure pointer swapping is done properly
08.	Traversal error in reverse display	Use `previous` pointer correctly

4a)Aim:-write a python program to implement Single linked list which could perform below operations by taking input from the user

1)create linked list

2)Insert beginning

3)insert end

4)Insert at specific index

5)Delete by value

6)Delete first node

7)Delete last node

8)count number of nodes

9)Display/Traverse

10)exit

Program:-

```
class Node:
    def __init__(self, data):
        self.data = data
```

```
        self.next = None



class SinglyLinkedList:
    def __init__(self):
        self.head = None

    # Create list
    def create(self):
        n = int(input("Enter number of elements: "))
        for i in range(n):
            x = int(input("Enter value: "))
            new = Node(x)

            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new

    # Insert at beginning
    def insert_begin(self, data):
```

```
    new = Node(data)
    new.next = self.head
    self.head = new

# Insert at end
def insert_end(self, data):
    new = Node(data)

    if self.head is None:
        self.head = new
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new

# Insert at index
def insert_index(self, index, data):
    if index == 0:
        self.insert_begin(data)
        return
    elif index > self.count() or index < 0:
        print("Invalid Index")
        return
```

```
    new = Node(data)
    temp = self.head

    for i in range(index - 1):
        temp = temp.next

    new.next = temp.next
    temp.next = new

# Delete from beginning
def deleteAtBeg(self):
    if self.head is None:
        print("No Data to delete")
    else:
        temp = self.head
        self.head = temp.next
        print("Deleted Value =", temp.data)

# Delete from end
def deleteAtEnd(self):
    if self.head is None:
        print("No Data to delete")
    elif self.head.next is None:
        self.head = None
    else:
```

```
        temp = self.head
        while temp.next.next:
            temp = temp.next
        temp.next = None

# Delete by value
def delete(self, value):
    if self.head is None:
        print("No Data to delete")
        return

    temp = self.head

    if temp and temp.data == value:
        self.head = temp.next
        print("Value deleted")
        return

    while temp.next and temp.next.data != value:
        temp = temp.next

    if temp.next is None:
        print("Value not present")
    else:
        temp.next = temp.next.next
```

```
            print("Value deleted")

    # Display list
    def display(self):
        if self.head is None:
            print("No Data")
        else:
            temp = self.head
            while temp:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

    # Count nodes
    def count(self):
        c = 0
        temp = self.head
        while temp:
            c += 1
            temp = temp.next
        return c


# Menu-driven program
sll = SinglyLinkedList()
```

```
while True:
    print("\n1. Create Linked List")
    print("2. Insert Beginning")
    print("3. Insert End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Nodes")
    print("9. Display")
    print("10. Exit")

    ch = int(input("Choice: "))

    if ch == 1:
        sll.create()

    elif ch == 2:
        x = int(input("Value: "))
        sll.insert_begin(x)

    elif ch == 3:
        x = int(input("Value: "))
        sll.insert_end(x)
```

```
elif ch == 4:
    idx = int(input("Index: "))
    x = int(input("Value: "))
    sll.insert_index(idx, x)

elif ch == 5:
    x = int(input("Delete value: "))
    sll.delete(x)

elif ch == 6:
    sll.deleteAtBeg()

elif ch == 7:
    sll.deleteAtEnd()

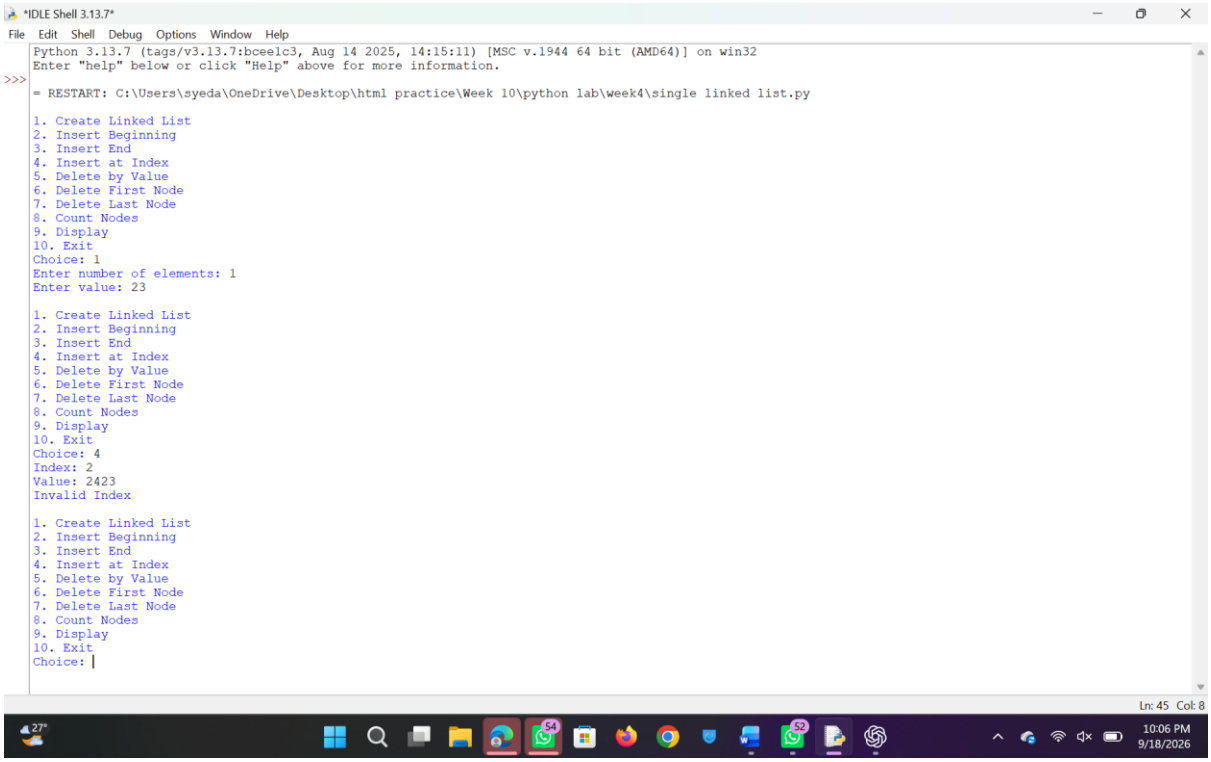
elif ch == 8:
    print("Number of nodes =", sll.count())

elif ch == 9:
    sll.display()

elif ch == 10:
    break
```

```
    else:
        print("Invalid choice")
```

Output:-



Negative Case:



Explanation:

This program implements a **Singly Linked List** using Python and performs operations such as insertion, deletion, and traversal using a menu-driven approach.

1. **Node Class Creation:**
   The class Node is defined with:
   - data → stores the value
   - next → stores reference to the next node
2. **Singly Linked List Initialization:**
   The class SinglyLinkedList contains a pointer head which points to the first node of the list.
3. **Creating the List:**
   The function create() takes input from the user and creates nodes dynamically.
   - If list is empty → new node becomes head
   - Otherwise → traverse till last node and attach new node
4. **Insertion at Beginning:**
   - A new node is created
   - Its next points to current head
   - Head is updated to new node
5. **Insertion at End:**
   - Traverse till last node
   - Attach new node at the end
6. **Insertion at Index:**
   - If index = 0 → insert at beginning
   - If index is invalid → display error
   - Otherwise → traverse to position and insert node
7. **Deletion at Beginning:**
   - If list is empty → display message
   - Otherwise → head is moved to next node
8. **Deletion at End:**
   - If only one node → list becomes empty
   - Otherwise → traverse till second last node and remove last node
9. **Deletion by Value:**
   - If value is at head → update head
   - Otherwise → traverse list to find value
   - Adjust links to remove node
10. **Display Operation:**
    - Traverse from head to end
    - Print each node's data
11. **Count Operation:**
    - Traverse list and count number of nodes
12. **Menu-Driven Execution:**
    - User selects operation
    - Corresponding function is executed

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError for invalid input	Use `try-except` for input
02.	Invalid index during insertion	Check index range before inserting
03.	Deletion on empty list	Check if `head is None`
04.	Value not found during deletion	Display proper message

S.No	Error Obtained	Rectification
05.	Infinite loop due to wrong linking	Ensure `next` pointers are correct
06.	Reverse display not possible directly	Use auxiliary list for reverse traversal
07.	Program crash for negative index	Validate index before operation

WEEK – 5

5)aim:-

write a python program to implement a stack using an array, linked list. Also

implement the stack operation such as push, pop, peep and also display the stack elements

Program:-

```
class StackArray:

def init(self, size):

 self.size = size self.stack = [None] * size self.top = -1

def push(self, item):
    if self.top == self.size - 1:
        print("Stack Overflow")
    else:
        self.top += 1
        self.stack[self.top] = item
        print(item, "pushed into the stack")

def pop(self):
    if self.top == -1:
        print("Stack Underflow")
    else:
        item = self.stack[self.top]
        self.stack[self.top] = None
        self.top -= 1
        print(item, "popped from the stack")

def peek(self):
    if self.top == -1:
        print("Stack is empty")
```

```
    else:
        print("Top element:", self.stack[self.top])

def display(self):
    if self.top == -1:
        print("Stack is empty")
    else:
        print("Stack elements (top to bottom):")
        for i in range(self.top, -1, -1):
            print(self.stack[i])

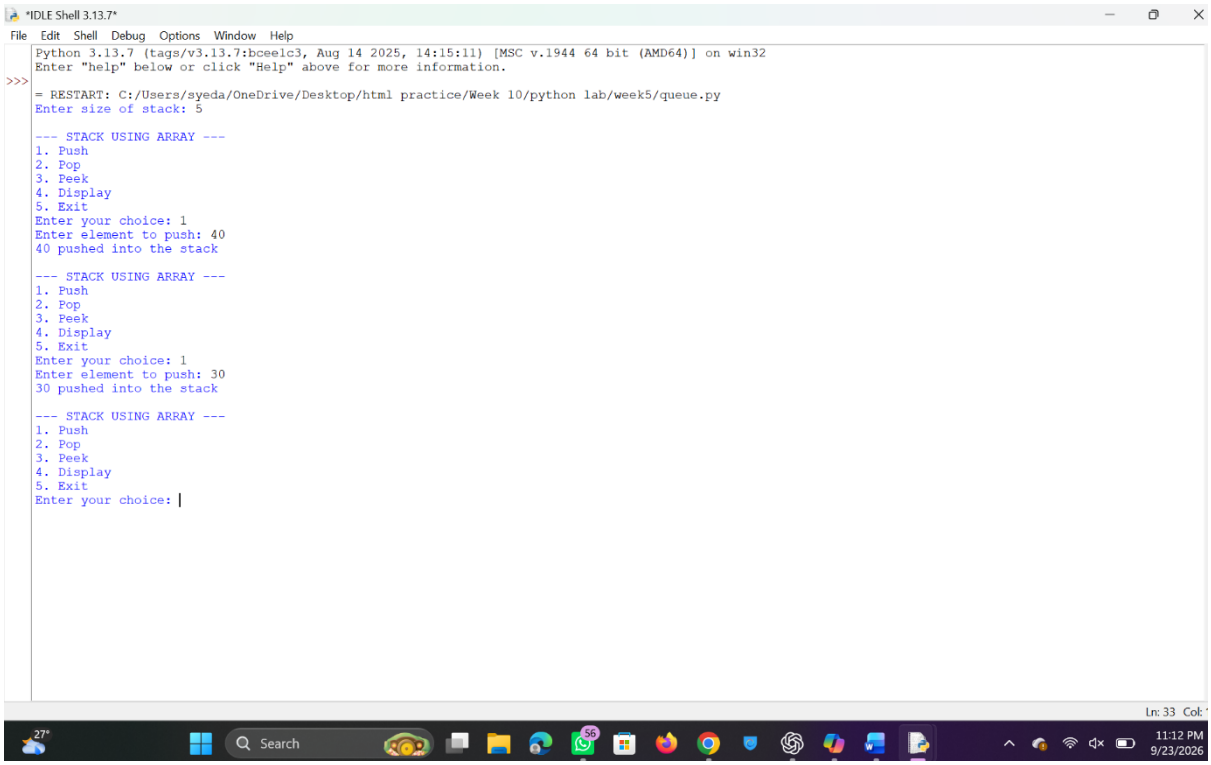

size = int(input("Enter size of stack: ")) s = StackArray(size)

while True: print("\n--- STACK USING ARRAY ---") print("1. Push") print("2. Pop")
print("3. Peek") print("4. Display") print("5. Exit")

choice = int(input("Enter your choice: "))

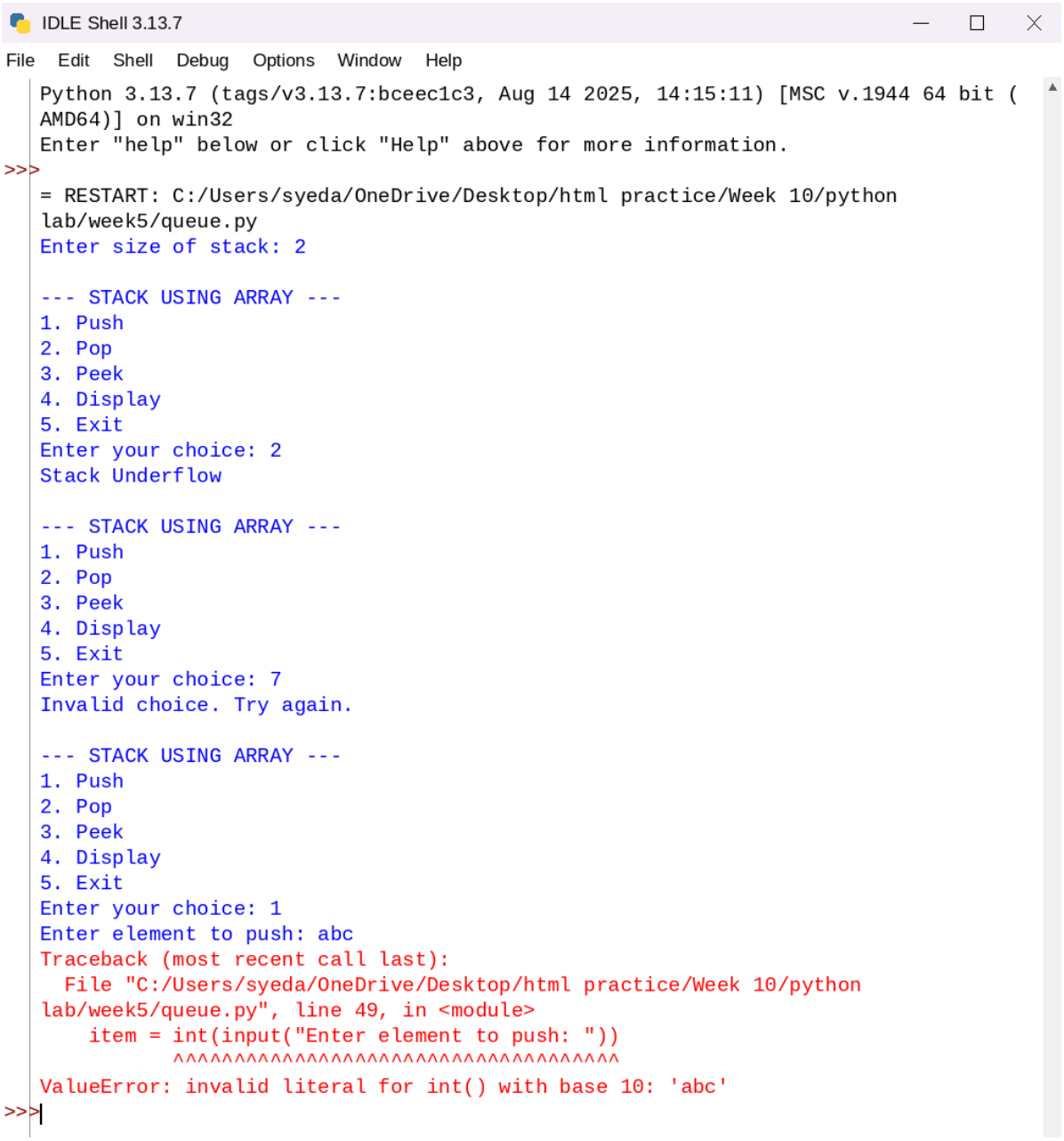
if choice == 1:
    item = int(input("Enter element to push: "))
    s.push(item)
elif choice == 2:
    s.pop()
elif choice == 3:
    s.peek()
elif choice == 4:
    s.display()
elif choice == 5:
    print("Exiting program...")
    break
else:
    print("Invalid choice. Try again.")
```

Output:



Stack using linkedlist

Negative Case:





Program:

class Node:

def init(self, data):

self.data = data self.next = None

class StackLinkedList: def init(self): self.top = None

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
    print(data, "pushed into the stack")
```

def pop(self):

```
    if self.top is None:
        print("Stack Underflow")
    else:
        popped = self.top.data
        self.top = self.top.next
        print(popped, "popped from the stack")

def peek(self):
    if self.top is None:
        print("Stack is empty")
    else:
        print("Top element:", self.top.data)

def display(self):
    if self.top is None:a
        print("Stack is empty")
    else:
        print("Stack elements (top to bottom):")
        temp = self.top
        while temp:
            print(temp.data)
            temp = temp.next

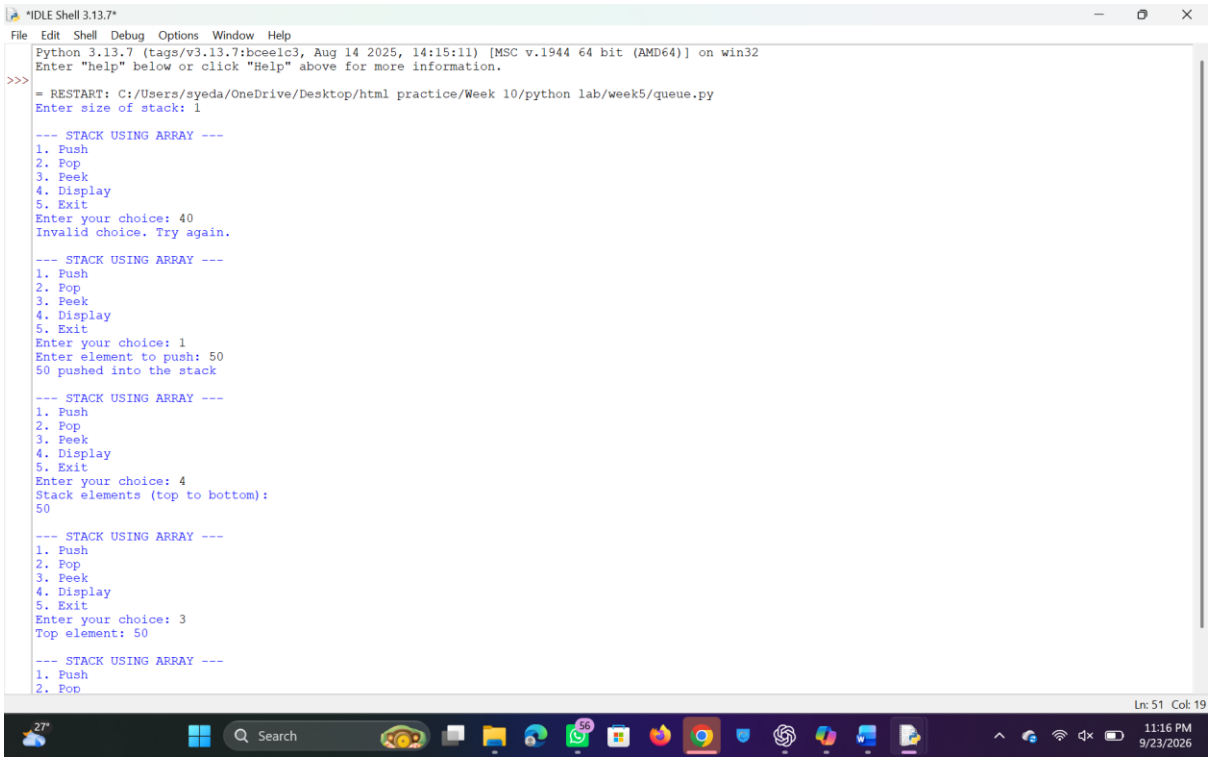

s = StackLinkedList()

while True: print("\n--- STACK USING LINKED LIST ---") print("1. Push") print("2. Pop")
print("3. Peek") print("4. Display") print("5. Exit")

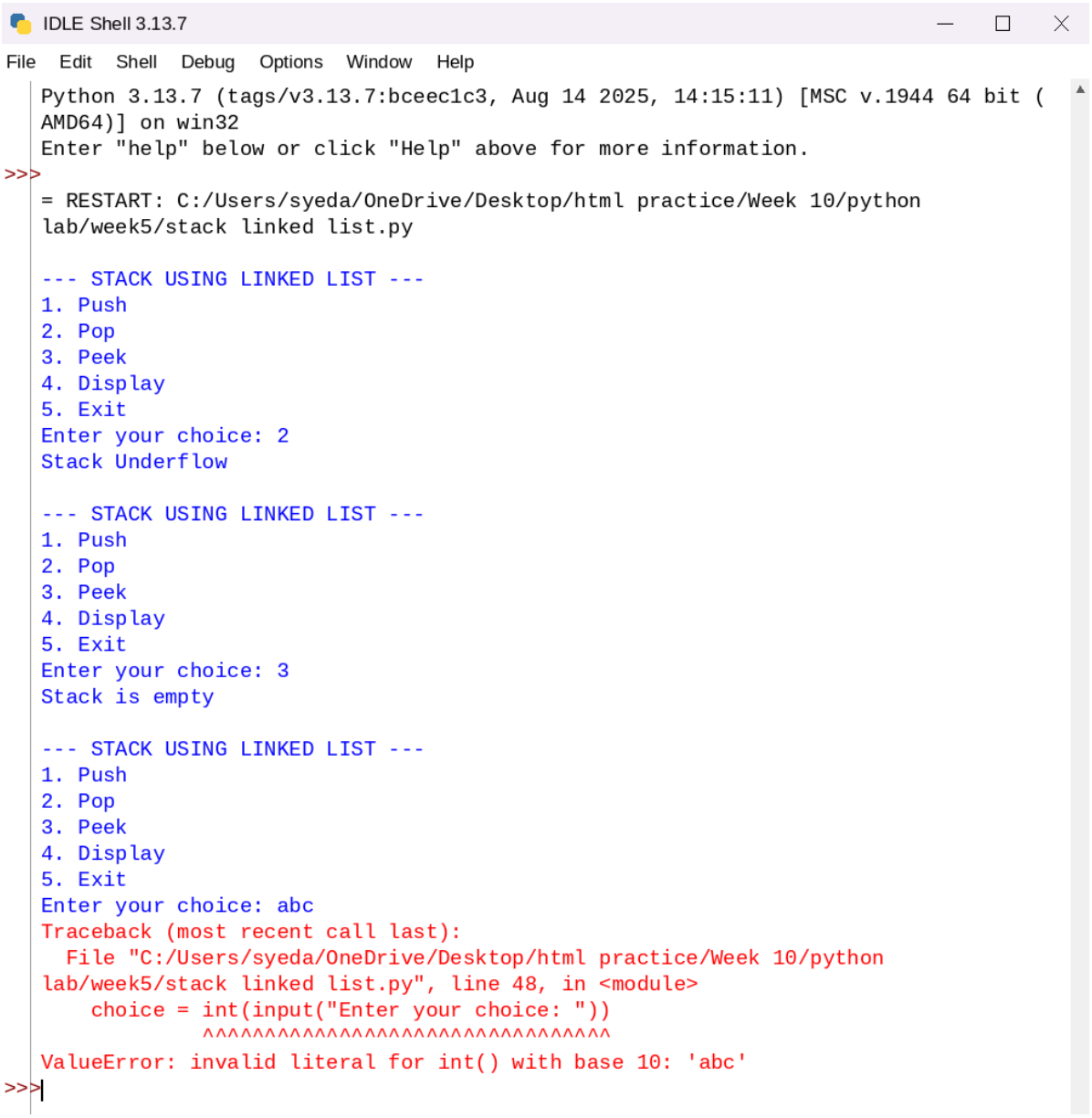
choice = int(input("Enter your choice: "))

if choice == 1:
    item = int(input("Enter element to push: "))
    s.push(item)
elif choice == 2:
    s.pop()
elif choice == 3:
    s.peek()
elif choice == 4:
    s.display()
elif choice == 5:
    print("Exiting program...")
    break
else:
    print("Invalid choice. Try again.")
```

Output:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python lab/week5/queue.py
Enter size of stack: 1
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 40
Invalid choice. Try again.
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 50
50 pushed into the stack
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements (top to bottom):
50
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 50
--- STACK USING ARRAY ---
1. Push
2. Pop
Ln: 51 Col: 19
27°
Search
11:16 PM
9/23/2026

Negative Case:



Explanation:

This program implements a stack using an array and provides basic stack operations.

1. A class StackArray is defined with a constructor __init__(self, size) to initialize:
    - size: Maximum size of the stack
    - stack: A list of fixed size initialized with None
    - top = -1: Indicates that the stack is initially empty
2. The push(self, item) method is used to insert an element into the stack:
    - It checks if top == size - 1 (stack full), then displays "Stack Overflow"
    - Otherwise, top is incremented and the element is inserted
3. The pop(self) method is used to remove an element from the stack:
    - It checks if top == -1 (stack empty), then displays "Stack Underflow"
    - Otherwise, the top element is removed and top is decremented

4. The peek(self) method displays the top element of the stack:
    - If the stack is empty, it displays "Stack is empty"
    - Otherwise, it prints the element at index top
5. The display(self) method shows all elements in the stack:
    - If the stack is empty, it displays "Stack is empty"
    - Otherwise, elements are printed from top to bottom using a loop
6. In the driver code:
    - The user enters the size of the stack
    - A menu-driven loop allows the user to perform operations: Push, Pop, Peek, Display, or Exit
7. Based on user choice, the corresponding stack operation is executed until the user chooses to exit.

Error Table:

S.No	Error Obtained	Rectification
01.	ValueError when non-integer input is entered	Ensure valid integer input or use try-except
02.	Stack Overflow when pushing beyond capacity	Check size before pushing (already handled)
03.	Stack Underflow when popping from empty stack	Check if stack is empty before popping (already handled)
04.	Invalid menu choice entered	Validate input and prompt user again

Week-6

6a):Aim:-Write a program to implement queue and its operations such as enque deque ,peck,display,with menu driven code,using array and linked list data structure.

Queue using Array

Program:-

```
class QueueArray:

    def __init__(self, size):

        self.queue = [None] * size

        self.front = -1

        self.rear = -1

        self.size = size
```

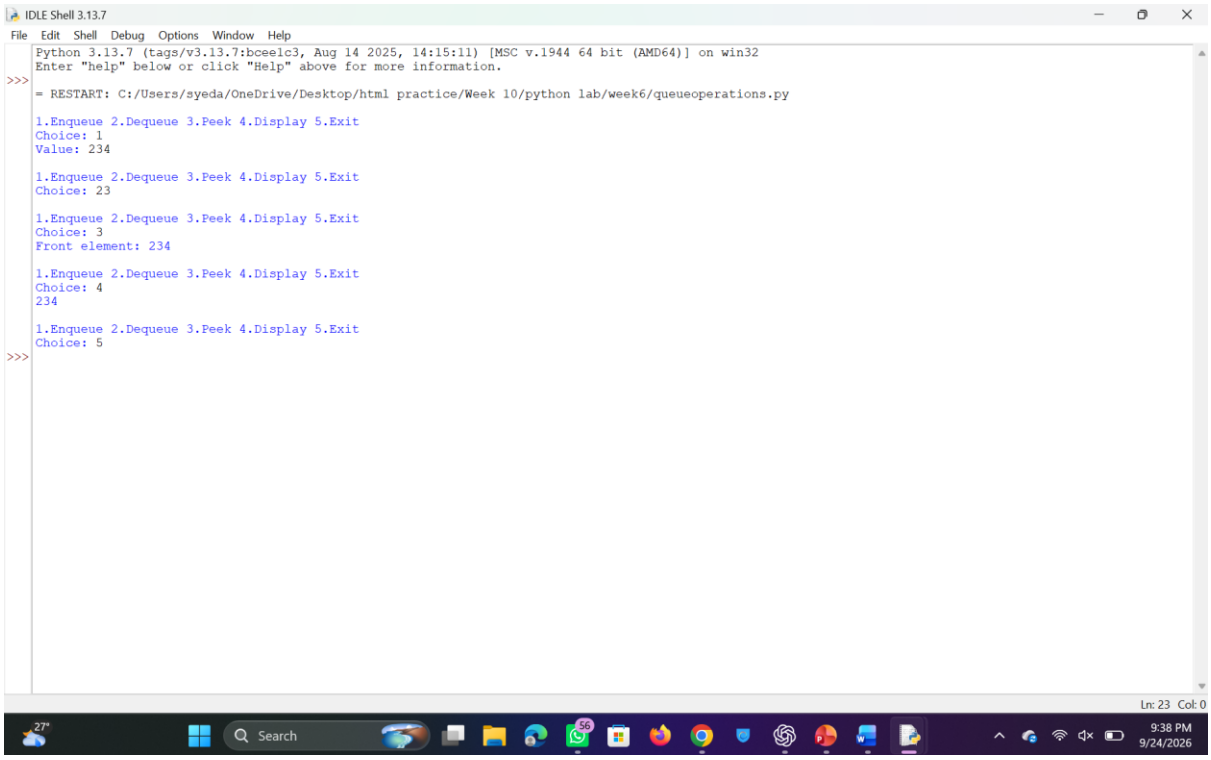
```
def enqueue(self, data):
    if self.rear == self.size - 1:
        print("Queue Overflow")
    else:
        if self.front == -1:
            self.front = 0
        self.rear += 1
        self.queue[self.rear] = data
def dequeue(self):
    if self.front == -1 or self.front > self.rear:
        print("Queue Underflow")
    else:
        print("Deleted:", self.queue[self.front])
        self.front += 1
def peek(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("Front element:", self.queue[self.front])
def display(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        for i in range(self.front, self.rear + 1):
```

```
            print(self.queue[i], end=" ")
        print()
# Menu
q = QueueArray(5)

while True:
 print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")
    ch = int(input("Choice: "))

    if ch == 1:
        x = int(input("Value: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
```

Output:-



Negative Case:

IDLE Shell 3.13.7

File Edit Shell Debug Options Window Help

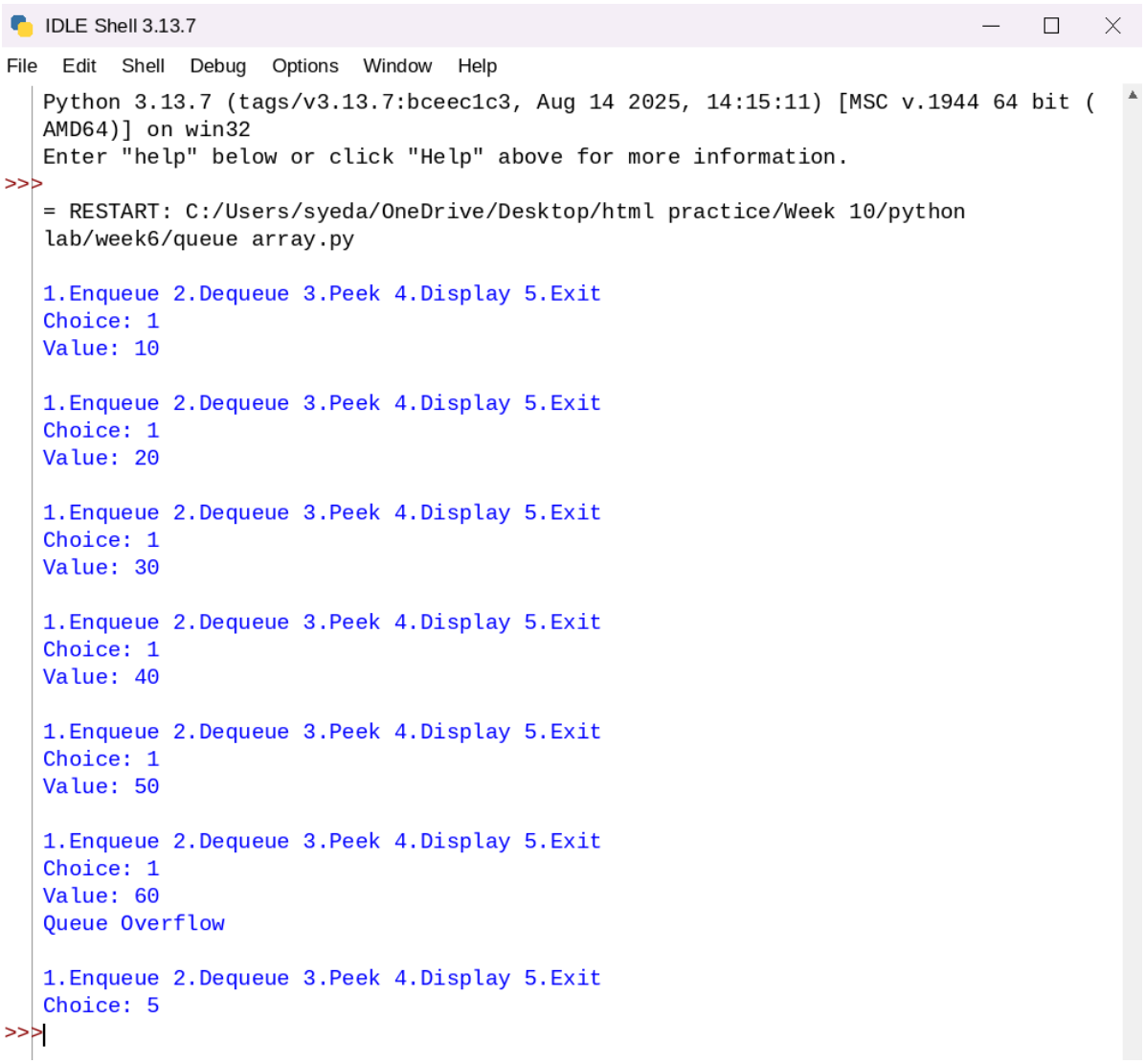
```
Python 3.13.7 (tags/v3.13.7:bceec1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python
lab/week6/queue array.py

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 2
Queue Underflow

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 3
Queue is empty

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 4
Queue is empty

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: abc
Traceback (most recent call last):
  File "C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python
lab/week6/queue array.py", line 39, in <module>
    ch = int(input("Choice: "))
         ^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'
>>>
```



Explanation:

This program implements a **Queue using Array** following FIFO principle.

1. The class QueueArray is defined with an array, front, and rear.
2. Initially, both front and rear are set to -1 indicating empty queue.
3. **Enqueue operation:**
    - Inserts element at rear.
    - If queue is full (rear == size-1), overflow occurs.
4. **Dequeue operation:**
    - Removes element from front.
    - If queue is empty, underflow occurs.
5. **Peek operation:**
    - Displays front element without removing it.
6. **Display operation:**
    - Prints elements from front to rear.
7. Menu-driven approach allows user interaction.

Error Table:

S.No	Error Obtained	Rectification
01	Queue Overflow	Check condition before enqueue
02	Queue Underflow	Check if queue is empty
03	Invalid input	Use try-except
04	Incorrect pointer update	Update front and rear properly

Queue using Linked List

Program:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class QueueLL:
    def __init__(self):
        self.front = None
        self.rear = None


    def enqueue(self, data):
        new = Node(data)
        if self.rear is None:
            self.front = self.rear = new
        else:
            self.rear.next = new
            self.rear = new
```

```
def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
    else:
        temp = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        print("Deleted:", temp.data)

def peek(self):
    if self.front is None:
        print("Queue is empty")
    else:
        print("Front element:", self.front.data)

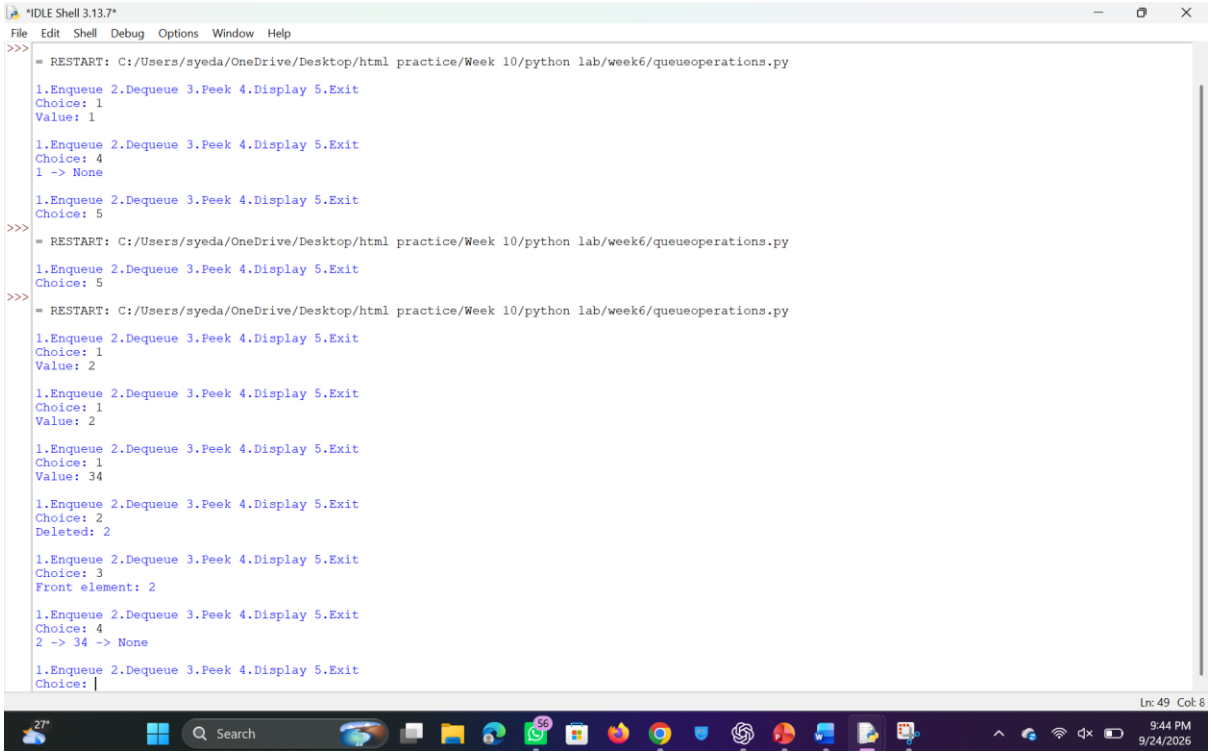
def display(self):
    temp = self.front
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

```
# Menu
q = QueueLL()

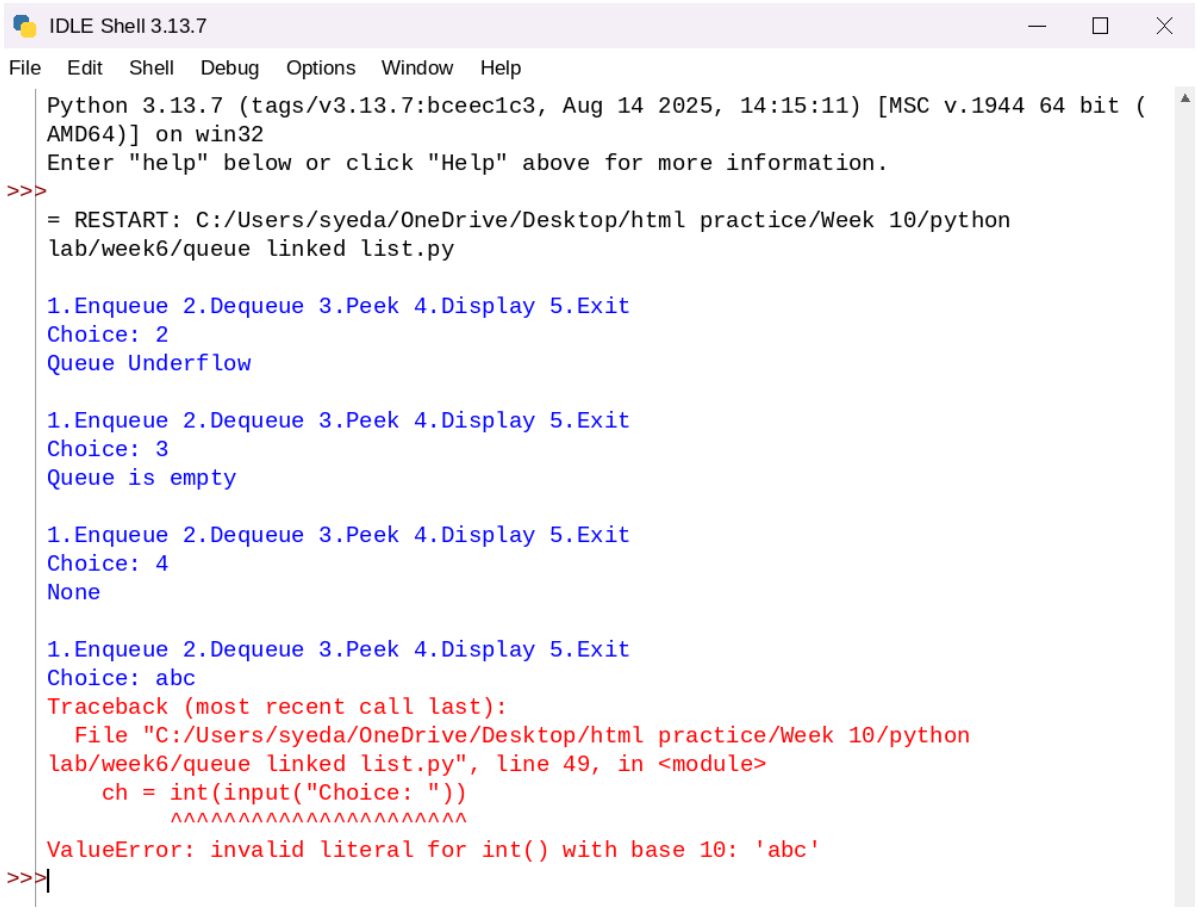

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")
    ch = int(input("Choice: "))


    if ch == 1:
        x = int(input("Value: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
  elif ch == 4:
        q.display()
    elif ch == 5:
        break
```

Output:-



Negative Case:



Explanation:

This program implements a **Queue using Linked List**.

1. A Node class is used to create elements.
2. front points to first node, rear to last node.
3. **Enqueue:** adds node at rear.
4. **Dequeue:** removes node from front.
5. No overflow condition unless memory is full.
6. Dynamic memory allocation is used.

Error Table:

S.No	Error Obtained	Rectification
01	Queue Underflow	Check if front is None
02	Memory issues	Ensure sufficient memory
03	Incorrect pointer linking	Update next pointers correctly
04	Empty queue display issue	Check before traversal

6b):-Write a program to implement Circular queue and its operations such as enque deque ,peck,display,with menu driven code,using array and linked list data structure.

Circular Queue using Array

Program:-

```
class CircularQueue:

    def __init__(self, size):

        self.queue = [None] * size

        self.front = -1

        self.rear = -1

        self.size = size



    def enqueue(self, data):

        if (self.rear + 1) % self.size == self.front:

            print("Queue Overflow")

        else:
```

```
        if self.front == -1:
            self.front = 0
        self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = data

def dequeue(self):
    if self.front == -1:
        print("Queue Underflow")
    else:
        print("Deleted:", self.queue[self.front])
        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

def peek(self):
    if self.front == -1:
        print("Queue is empty")
    else:
        print("Front element:", self.queue[self.front])

def display(self):
    if self.front == -1:
        print("Queue is empty")
    else:
```

```
        i = self.front
        while True:
            print(self.queue[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size
        print()

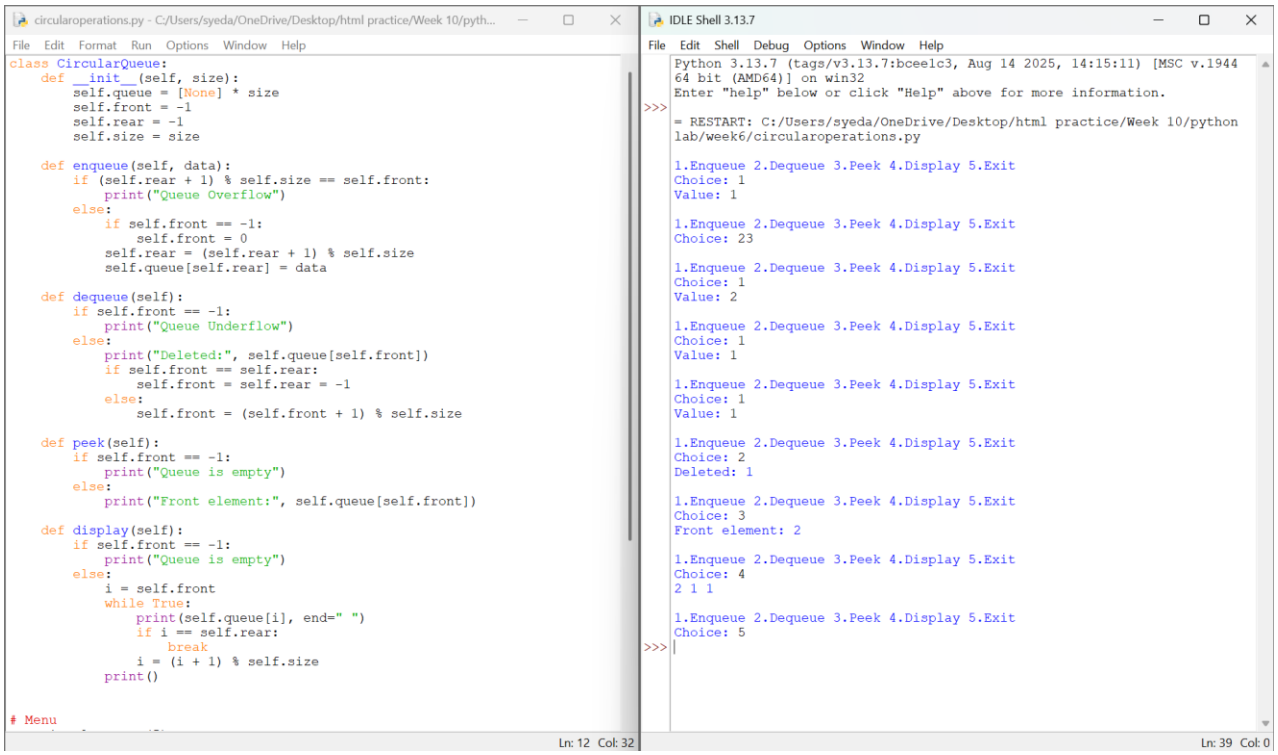

# Menu
q = CircularQueue(5)

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")
    ch = int(input("Choice: "))

    if ch == 1:
        x = int(input("Value: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
```

```
    elif ch == 5:
        break
```

output:-



```
class CircularQueue:
    def __init__(self, size):
        self.queue = [None] * size
        self.front = -1
        self.rear = -1
        self.size = size

    def enqueue(self, data):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = data

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            print("Deleted:", self.queue[self.front])
            if self.front == self.rear:
                self.front = self.rear = -1
            else:
                self.front = (self.front + 1) % self.size

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            i = self.front
            while True:
                print(self.queue[i], end=" ")
                if i == self.rear:
                    break
                i = (i + 1) % self.size
            print()


# Menu
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/syeda/OneDrive/Desktop/html practice/Week 10/python
lab/week6/circularoperations.py

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 1
Value: 1

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 23

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 1
Value: 2

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 1
Value: 1

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 1
Value: 1

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 2
Deleted: 1

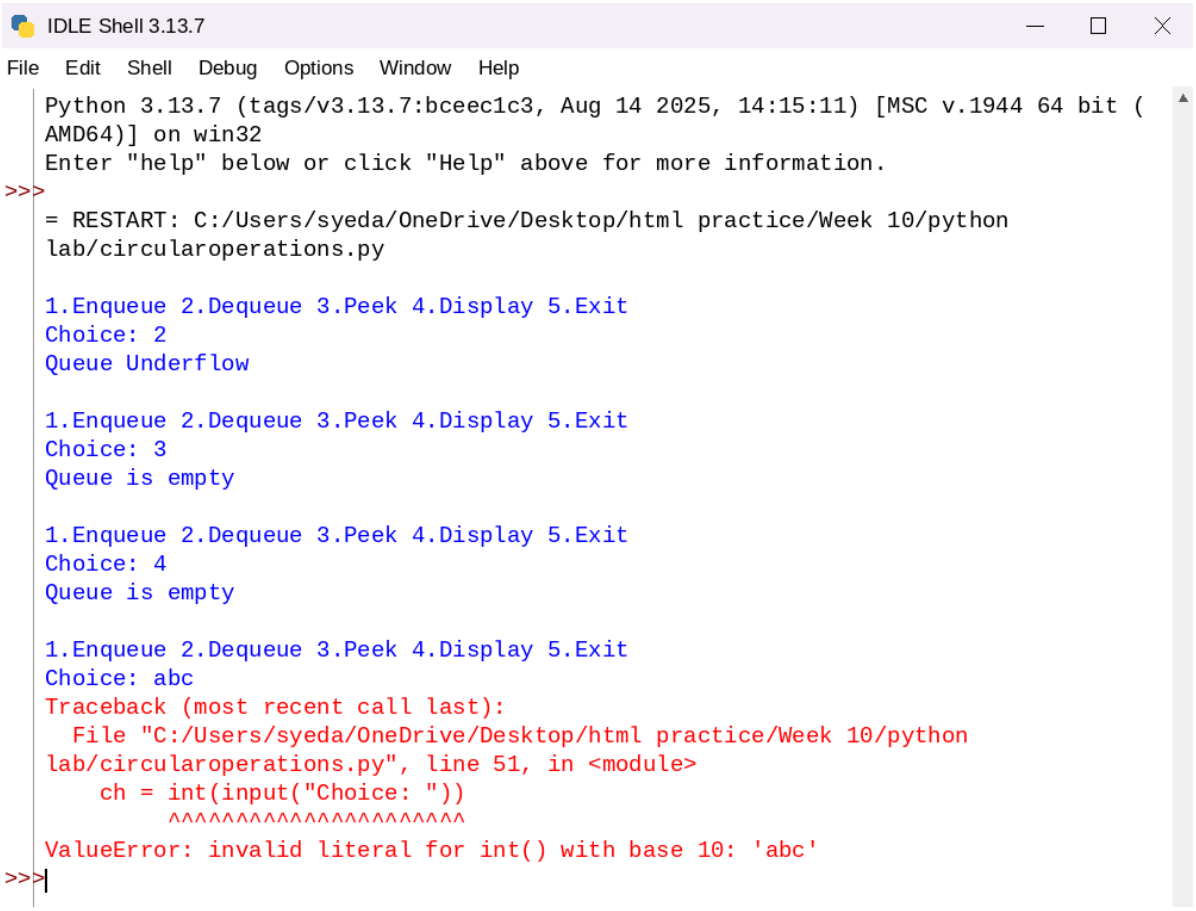
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 3
Front element: 2

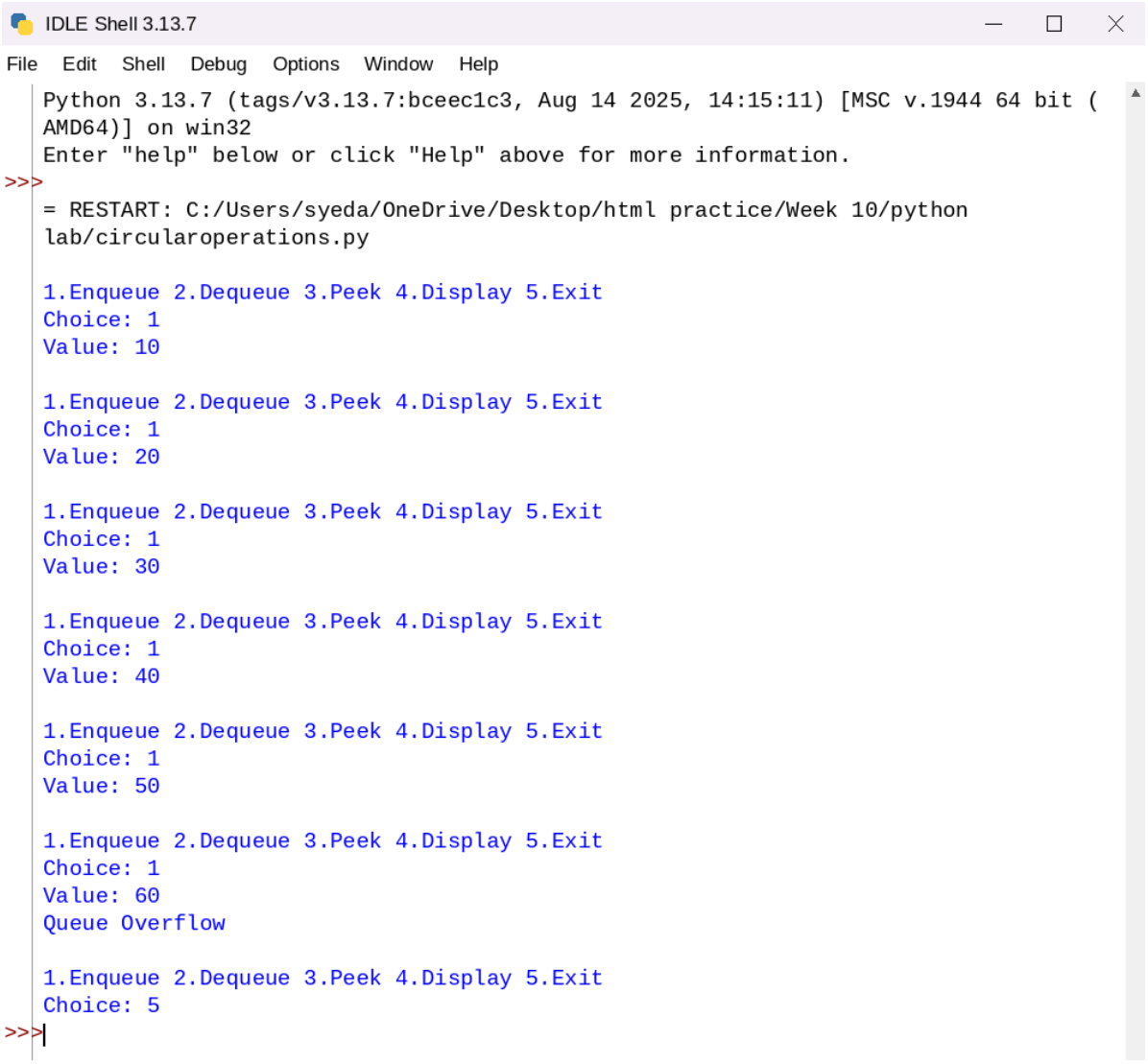
1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 4
2 1 1

1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit
Choice: 5
>>>
```

Explanation:

Negative Case:





Explanation:-

This program implements a Circular Queue using array.

1. Circular queue reuses empty space using modulo operation.
2. Enqueue: (rear + 1) % size
3. Dequeue: (front + 1) % size
4. Full condition → (rear + 1) % size == front
5. Empty condition → front == -1
6. Efficient memory utilization compared to linear queue.

Error Table:

S.No	Error Obtained	Rectification
01	Queue Overflow	Check circular condition
02	Queue Underflow	Check if queue is empty

S.No	Error Obtained	Rectification
03	Wrong modulo usage	Use correct formula
04	Traversal error	Use loop with break condition

Circular Queue using Linked List

Program:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None



class CircularQueueLL:
    def __init__(self):
        self.front = None
        self.rear = None

    # Enqueue
    def enqueue(self, data):
        new = Node(data)

        if self.front is None:
            self.front = self.rear = new
            self.rear.next = self.front
        else:
            self.rear.next = new
```

```
            self.rear = new
            self.rear.next = self.front

        print("Inserted:", data)

    # Dequeue
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
            return

        if self.front == self.rear:
            print("Deleted:", self.front.data)
            self.front = self.rear = None
        else:
            temp = self.front
            print("Deleted:", temp.data)
            self.front = self.front.next
            self.rear.next = self.front

    # Peek
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
```

```
            print("Front element:", self.front.data)

    # Display
    def display(self):
        if self.front is None:
            print("Queue is empty")
            return

        temp = self.front
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.front:
                break
        print("(front)")

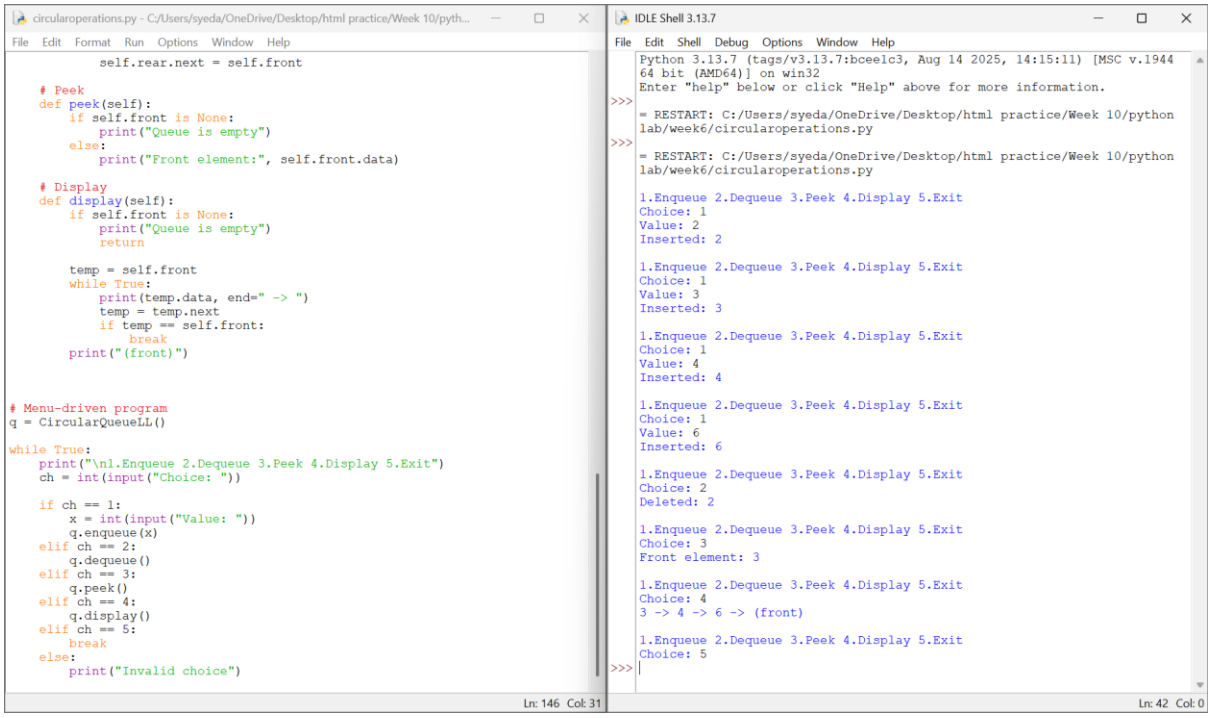


# Menu-driven program
q = CircularQueueLL()

while True:
    print("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit")
    ch = int(input("Choice: "))
```

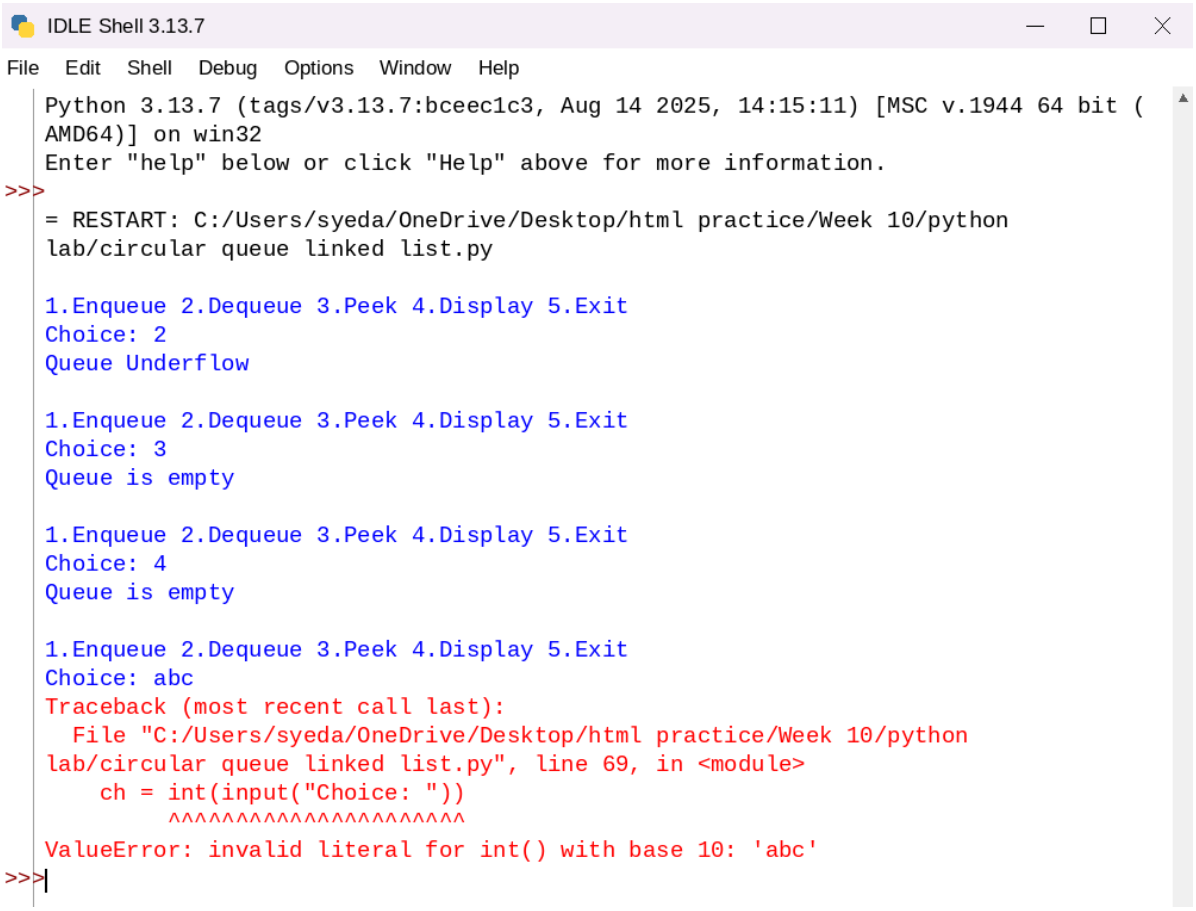
```
    if ch == 1:
        x = int(input("Value: "))
        q.enqueue(x)
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.peek()
    elif ch == 4:
        q.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

Output:-



Explanation:

Negative Case:



Explanation:-

This program implements a Circular Queue using Linked List.

1. A Node class is defined with data and next pointer.
2. The class CircularQueueLL maintains two pointers:
   - front → points to first node
   - rear → points to last node
3. Circular Nature:
   - The next pointer of rear always points to front.
   - This creates a circular structure.
4. Enqueue Operation:
   - A new node is created.
   - If queue is empty → both front and rear point to new node.
   - Otherwise → new node is added at rear and linked back to front.
5. Dequeue Operation:
   - If queue is empty → underflow condition.
   - If only one node → both pointers become None.
   - Otherwise → front is moved to next node and circular link is maintained.
6. Peek Operation:
   - Displays the value at front without removing it.
7. Display Operation:
   - Traverses from front until it reaches front again.

- Prints all elements in circular manner.

8. Menu-Driven Execution:
   - User selects operation and corresponding function is executed.

Error Table:

S.No	Error Obtained	Rectification
01.	Queue Underflow	Check if front is None before dequeue
02.	Infinite loop in display	Stop traversal when node reaches front again
03.	Incorrect linking	Ensure rear.next = front always
04.	Wrong update when single node	Set both front and rear to None
05.	Missing circular link after enqueue	Always update rear.next
06.	Display error when empty	Check before traversal

Week-7

7)Aim:- Write a program to implement binary tree using an array and linked with all of its operations (Insertion, deletion, searching , traversal)

Program:-

a)Using Array:-

```
class BinaryTreeArray:


    def __init__(self, size):
        self.tree = [None] * size
        self.size = size



    def set_root(self, data):
        if self.tree[0] is None:
            self.tree[0] = data
        else:
```

```
        print("Root already exists!")
    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1

        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")
    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2

        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")
    def insert(self, data):

        for i in range(self.size):

            if self.tree[i] is None:
                self.tree[i] = data
                print(data, "inserted at index", i)
                return

        print("Tree is full!")
```

```
def search(self, data):

    for i in range(self.size):

        if self.tree[i] == data:
            return i

    return -1
def delete(self, data):

    index = self.search(data)

    if index == -1:
        print(data, "not found!")
        return
    last_index = -1

    for i in range(self.size - 1, -1, -1):

        if self.tree[i] is not None:
            last_index = i
            break
    if last_index == index:
        self.tree[index] = None
```

```
        else:
            self.tree[index] = self.tree[last_index]
            self.tree[last_index] = None

        print(data, "deleted!")
    def preorder(self, index=0):

        if index >= self.size or self.tree[index] is None:
            return

        print(self.tree[index], end=" ")

        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)
    def inorder(self, index=0):

        if index >= self.size or self.tree[index] is None:
            return

        self.inorder(2 * index + 1)

        print(self.tree[index], end=" ")

        self.inorder(2 * index + 2)
    def postorder(self, index=0):
```

```
        if index >= self.size or self.tree[index] is None:
            return

        self.postorder(2 * index + 1)
        self.postorder(2 * index + 2)

        print(self.tree[index], end=" ")
    def level_order(self):

        for value in self.tree:

            if value is not None:
                print(value, end=" ")
    def display(self):

        print("\nArray Representation:")

        for i in range(self.size):
            print("Index", i, ":", self.tree[i])
tree = BinaryTreeArray(10)
tree.set_root('A')
tree.set_left(0, 'B')
tree.set_right(0, 'C')
tree.set_left(1, 'D')
```

```
tree.set_right(1, 'E')



tree.set_right(2, 'F')



tree.display()
print("\nPreorder: ", end="")
tree.preorder()

print("\nInorder: ", end="")
tree.inorder()

print("\nPostorder: ", end="")
tree.postorder()

print("\nLevel-order: ", end="")
tree.level_order()

print("\n\nSearching for E:")

position = tree.search('E')

if position != -1:
    print("Element found at index", position)
```

```
else:
    print("Element not found!")


print("\nInsertion:")
tree.insert('G')


tree.display()


print("\nDeletion:")
tree.delete('D')

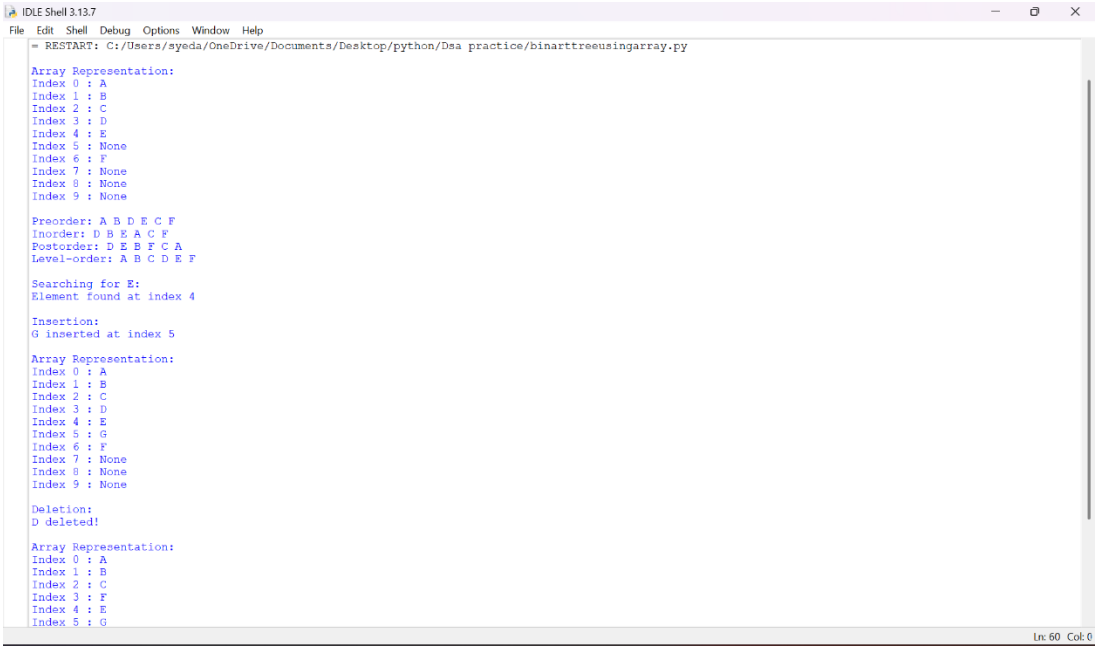

tree.display()
print("\nPreorder after deletion: ", end="")
tree.preorder()

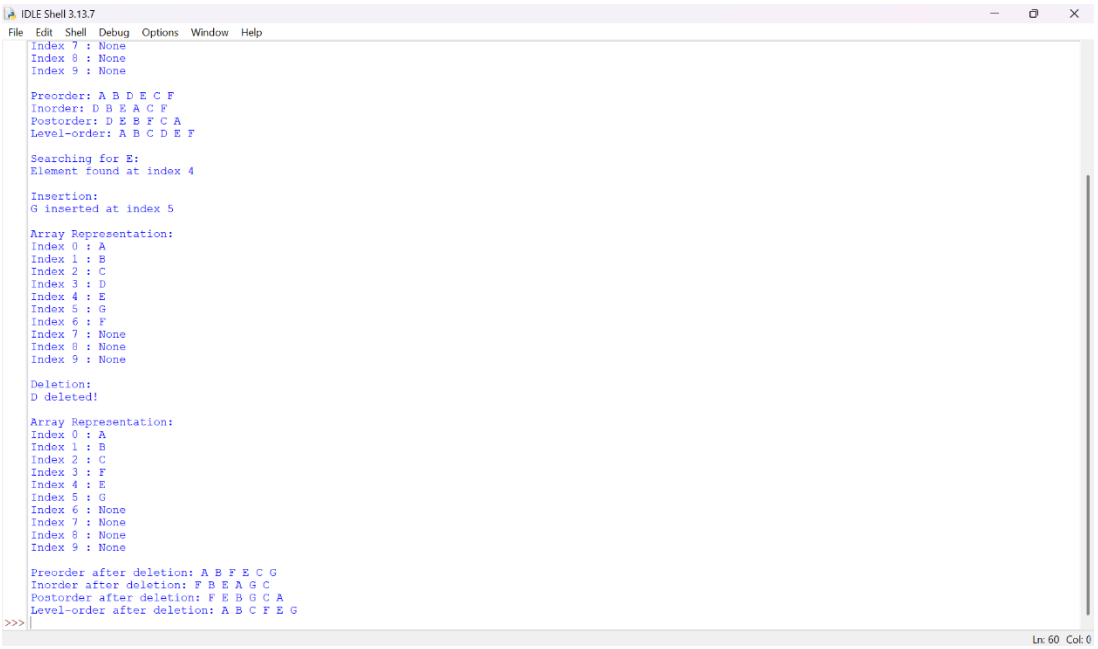

print("\nInorder after deletion: ", end="")
tree.inorder()


print("\nPostorder after deletion: ", end="")
tree.postorder()

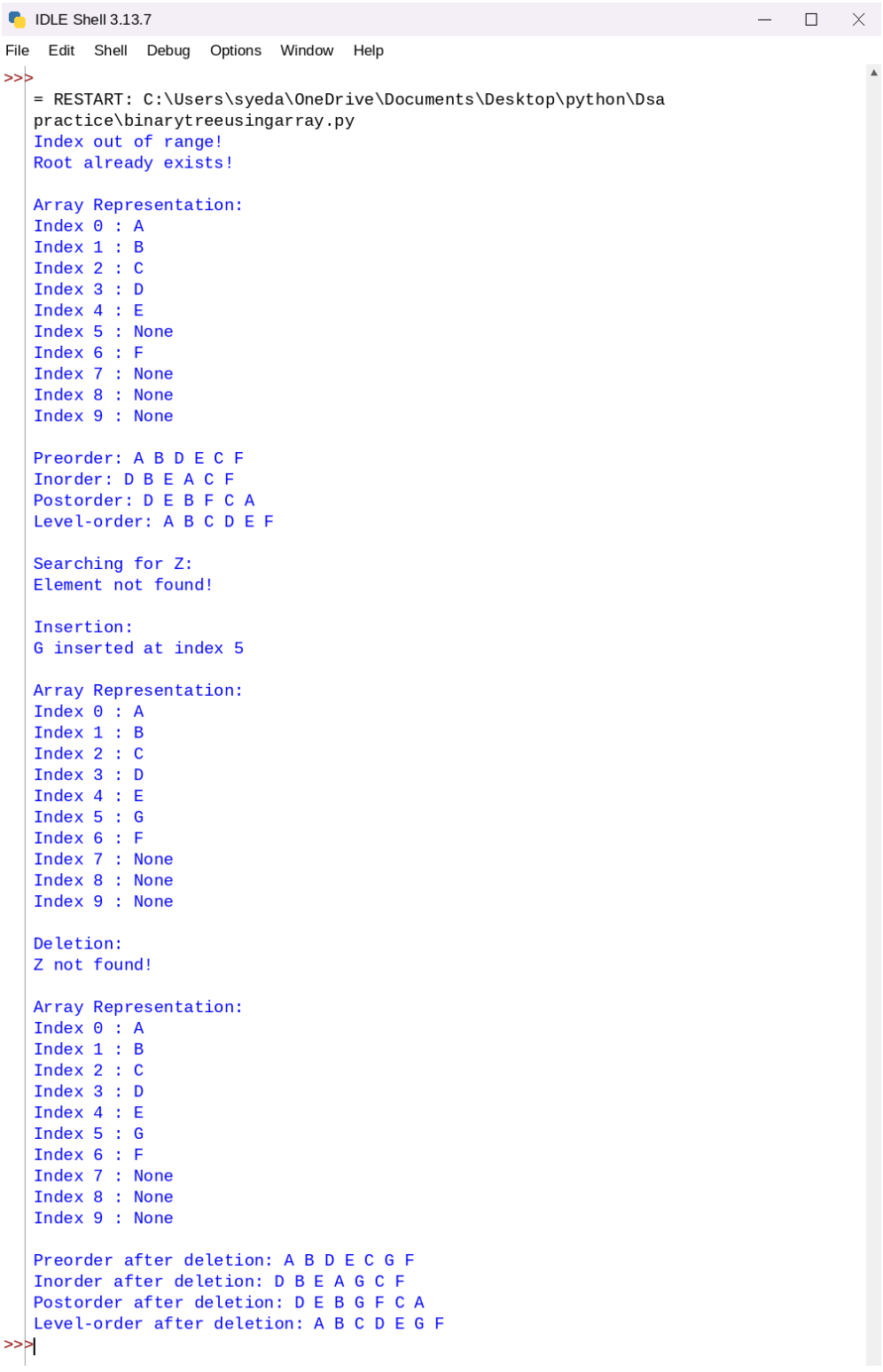

print("\nLevel-order after deletion: ", end="")
tree.level_order()
```

Output:-


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
= RESTART: C:/Users/syeda/OneDrive/Documents/Desktop/python/Dsa practice/binarttreeusingarray.py
Array Representation:
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : D
Index 4 : E
Index 5 : None
Index 6 : F
Index 7 : None
Index 8 : None
Index 9 : None
Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
Searching for E:
Element found at index 4
Insertion:
G inserted at index 5
Array Representation:
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : D
Index 4 : E
Index 5 : G
Index 6 : F
Index 7 : None
Index 8 : None
Index 9 : None
Deletion:
D deleted!
Array Representation:
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : F
Index 4 : E
Index 5 : G
Ln: 60 Col: 0


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Index 7 : None
Index 8 : None
Index 9 : None
Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
Searching for E:
Element found at index 4
Insertion:
G inserted at index 5
Array Representation:
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : D
Index 4 : E
Index 5 : G
Index 6 : F
Index 7 : None
Index 8 : None
Index 9 : None
Deletion:
D deleted!
Array Representation:
Index 0 : A
Index 1 : B
Index 2 : C
Index 3 : F
Index 4 : E
Index 5 : G
Index 6 : None
Index 7 : None
Index 8 : None
Index 9 : None
Preorder after deletion: A B F E C G
Inorder after deletion: F B E A G C
Postorder after deletion: F E B G C A
Level-order after deletion: A B C F E G
>>>
Ln: 60 Col: 0

Negative Case:



1. Binary Tree Using Array

Explanation:

This program implements a **Binary Tree using an Array** and performs operations such as insertion, deletion, searching, and traversal.

1. **Class Initialization:** The class BinaryTreeArray initializes an array of a specified size, with all elements set to None.
2. **Setting the Root:** The function set_root(data) inserts the root node at index 0 if the root does not already exist.
3. **Setting the Left Child:** The function set_left(parent_index, data) inserts the left child at index 2 * parent_index + 1.
4. **Setting the Right Child:** The function set_right(parent_index, data) inserts the right child at index 2 * parent_index + 2.
5. **Insertion:** The function insert(data) searches for the first empty position in the array and inserts the new element. If no space is available, it displays a message.
6. **Searching:** The function search(data) traverses the array and returns the index of the element if found; otherwise, it returns -1.
7. **Deletion:** The function delete(data) searches for the element and replaces it with the last occupied element in the array. The last occupied position is then set to None.
8. **Preorder Traversal:** The function preorder(index) visits the root, left subtree, and right subtree in that order.
9. **Inorder Traversal:** The function inorder(index) visits the left subtree, root, and right subtree in that order.
10. **Postorder Traversal:** The function postorder(index) visits the left subtree, right subtree, and root in that order.
11. **Level-Order Traversal:** The function level_order() displays the elements sequentially from the array, representing level-order traversal for a properly populated array-based tree.
12. **Display:** The function display() prints each array index along with its corresponding value.
13. **Main Program:** The program creates a binary tree, assigns nodes, performs traversal operations, searches for an element, inserts a new element, deletes an element, and displays the updated tree.

Error Table:

S.No.	Error Obtained	Rectification
01.	Array index out of range	Check whether the child index is within the array size.
02.	Insertion when the tree is full	Check for an empty position before inserting.
03.	Element not found during searching	Return -1 if the element does not exist.
04.	Incorrect traversal output	Use the correct order of root, left subtree, and right subtree.
05.	Incorrect deletion result	Find the last occupied position and update the array carefully.
06.	Root already exists	Check whether index 0 is empty before setting the root.

S.No.	Error Obtained	Rectification
07.	Child inserted without a valid parent	Verify that the parent exists before inserting a child.

b)Using linked list:-

```
class Node:


    def __init__(self, data):

        self.data = data

        self.left = None

        self.right = None
class BinaryTree:


    def __init__(self):

        self.root = None

    def create_tree(self):


        self.root = Node('A')

        self.root.left = Node('B')

        self.root.right = Node('C')

        self.root.left.left = Node('D')

        self.root.left.right = Node('E')

        self.root.right.right = Node('F')

    def insert(self, data):


        new_node = Node(data)

        if self.root is None:
```

```
            self.root = new_node
            return

        queue = [self.root]

        while queue:

            node = queue.pop(0)
            if node.left is None:
                node.left = new_node
                return
            else:
                queue.append(node.left)
            if node.right is None:
                node.right = new_node
                return
            else:
                queue.append(node.right)
    def search(self, data):

        if self.root is None:
            return None

        queue = [self.root]
```

```
    while queue:

        node = queue.pop(0)

        if node.data == data:
            return node

        if node.left is not None:
            queue.append(node.left)

        if node.right is not None:
            queue.append(node.right)

    return None
def delete(self, data):

    if self.root is None:
        print("Tree is empty!")
        return
    if (self.root.data == data and
        self.root.left is None and
        self.root.right is None):

        self.root = None
        print(data, "deleted!")
```

```
    return

queue = [self.root]

target = None
deepest = None
parent_of_deepest = None

while queue:

    node = queue.pop(0)
    if node.data == data:
        target = node
    if node.left is not None:
        parent_of_deepest = node
        deepest = node.left
        queue.append(node.left)

    if node.right is not None:
        parent_of_deepest = node
        deepest = node.right
        queue.append(node.right)
if target is None:
    print(data, "not found!")
    return
```

```
        target.data = deepest.data
        if parent_of_deepest.right == deepest:
            parent_of_deepest.right = None
        else:
            parent_of_deepest.left = None

        print(data, "deleted!")
    def preorder(self, node):

        if node is None:
            return

        print(node.data, end=" ")

        self.preorder(node.left)
        self.preorder(node.right)
    def inorder(self, node):

        if node is None:
            return

        self.inorder(node.left)

        print(node.data, end=" ")
```

```
        self.inorder(node.right)
    def postorder(self, node):

        if node is None:
            return

        self.postorder(node.left)
        self.postorder(node.right)

        print(node.data, end=" ")
    def level_order(self):

        if self.root is None:
            return

        queue = [self.root]

        while queue:

            node = queue.pop(0)

            print(node.data, end=" ")
            if node.left is not None:
                queue.append(node.left)
            if node.right is not None:
```

```
                queue.append(node.right)


tree = BinaryTree()
tree.create_tree()
print("Preorder: ", end="")
tree.preorder(tree.root)

print("\nInorder: ", end="")
tree.inorder(tree.root)

print("\nPostorder: ", end="")
tree.postorder(tree.root)

print("\nLevel-order: ", end="")
tree.level_order()
print("\n\nSearching for E:")

result = tree.search('E')

if result is not None:
    print("Element found:", result.data)
else:
    print("Element not found!")

print("\nInsertion:")
```

```
tree.insert('G')

print("Level-order after insertion: ", end="")
tree.level_order()

print("\n\nDeletion:")
tree.delete('B')

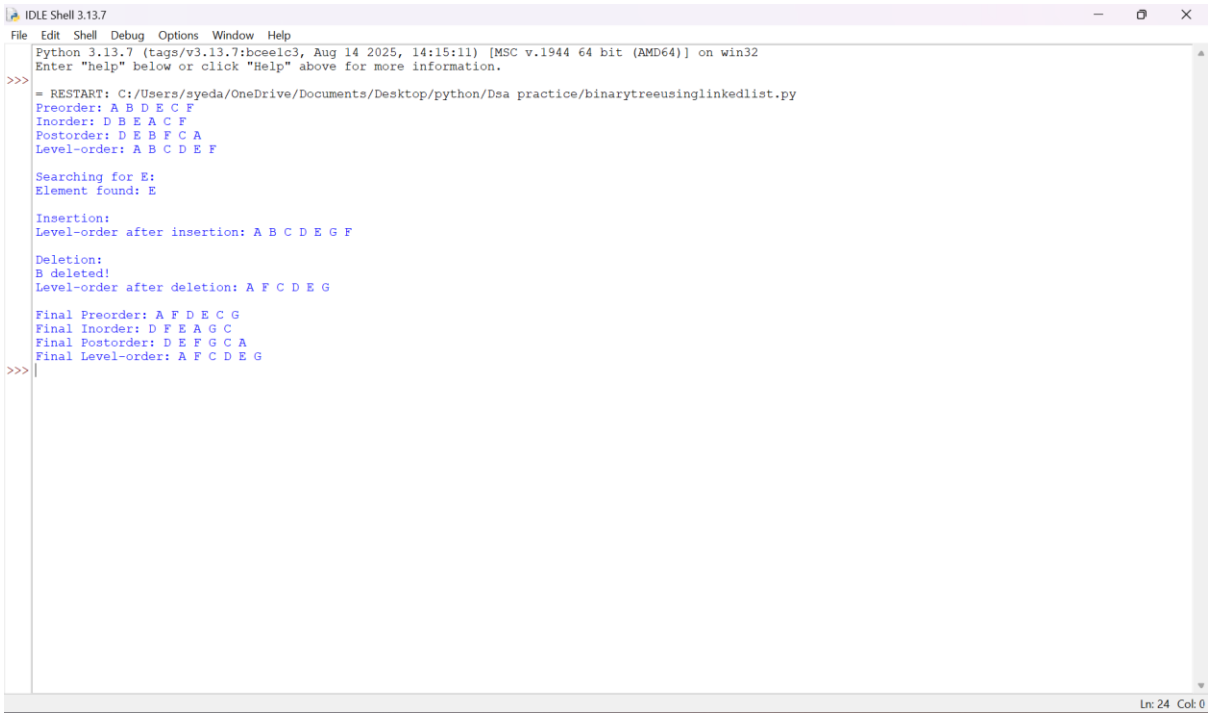
print("Level-order after deletion: ", end="")
tree.level_order()
print("\n\nFinal Preorder: ", end="")
tree.preorder(tree.root)

print("\nFinal Inorder: ", end="")
tree.inorder(tree.root)

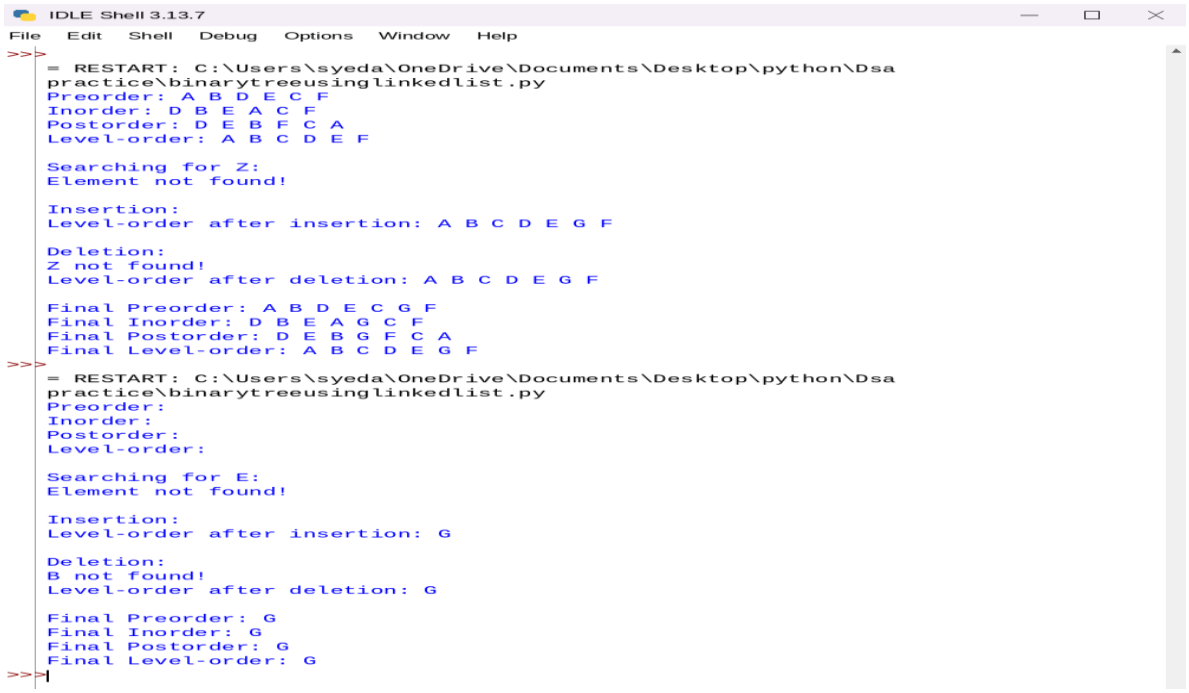
print("\nFinal Postorder: ", end="")
tree.postorder(tree.root)

print("\nFinal Level-order: ", end="")
tree.level_order()
```

Output:-



Negative Case:



2. Binary Tree Using Linked List

Explanation:

This program implements a **Binary Tree using a Linked List**. Each node contains data and references to its left and right children.

1. **Node Class Creation:** The class Node represents a tree node and contains three attributes:
    - data → stores the node value.
    - left → points to the left child.
    - right → points to the right child.
2. **Binary Tree Initialization:** The class BinaryTree maintains a root pointer that refers to the first node of the tree.
3. **Tree Creation:** The function create_tree() creates the binary tree by connecting nodes through their left and right pointers.
4. **Insertion:** The function insert(data) adds a new node to the tree according to the insertion logic defined in the program.
5. **Deletion:** The function delete(data) searches for the specified node and removes it according to the deletion logic implemented in the program.
6. **Searching:** The function search(data) traverses the tree to locate the required element. It returns the node if found; otherwise, it returns None.
7. **Preorder Traversal:** The function preorder(root) visits the root, left subtree, and right subtree.
8. **Inorder Traversal:** The function inorder(root) visits the left subtree, root, and right subtree.
9. **Postorder Traversal:** The function postorder(root) visits the left subtree, right subtree, and root.
10. **Level-Order Traversal:** The function level_order() uses a queue to visit nodes level by level. It adds the left and right children of each visited node to the queue.
11. **Main Program:** The program creates the binary tree, performs all four traversal operations, searches for an element, inserts a new node, deletes a node, and displays the updated traversal results.

Error Table:

S.No.	Error Obtained	Rectification
01.	AttributeError when accessing an empty node	Check whether the node is None before accessing its attributes.
02.	Incorrect left or right child connection	Assign the appropriate left and right pointers.
03.	Element not found during searching	Return None when the element does not exist.
04.	Incorrect traversal output	Follow the correct traversal sequence.
05.	Level-order traversal fails	Initialize the queue with the root and check for empty nodes.
06.	Incorrect deletion of a node	Handle leaf nodes, nodes with children, and root deletion appropriately.
07.	Duplicate or missing nodes during insertion	Follow the defined insertion rules and check existing nodes when necessary.

Week-8

Aim:- To write a Python program to implement a Binary Search Tree (BST) and perform operations such as insertion, deletion, searching, and tree traversals (inorder, preorder, postorder, and level-order).

Program:-

8 Binary search tree:-

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
# Insert a node
def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)
    return root
# Search for a node
def search(root, key):
    if root is None or root.data == key:
        return root
    if key < root.data:
        return search(root.left, key)
```

```
    return search(root.right, key)
# Find minimum value
def find_min(root):
    while root.left:
        root = root.left
    return root



# Delete a node
def delete(root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = delete(root.left, key)

    elif key > root.data:
        root.right = delete(root.right, key)

    else:
        if root.left is None:
            return root.right

        if root.right is None:
            return root.left
```

```
        temp = find_min(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root


# Inorder traversal
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder traversal
def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# Postorder traversal
```

```
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")



# Level-order traversal
def level_order(root):
    if root is None:
        return

    queue = [root]

    while queue:
        node = queue.pop(0)
        print(node.data, end=" ")

        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)
```

```
# Main program
root = None

values = [50, 30, 70, 20, 40, 60, 80]

for value in values:
    root = insert(root, value)

print("Inorder Traversal:")
inorder(root)

print("\nPreorder Traversal:")
preorder(root)

print("\nPostorder Traversal:")
postorder(root)

print("\nLevel-order Traversal:")
level_order(root)

key = 40
print("\n\nSearching for", key)

if search(root, key):
    print("Element found")
```

```
else:
    print("Element not found")

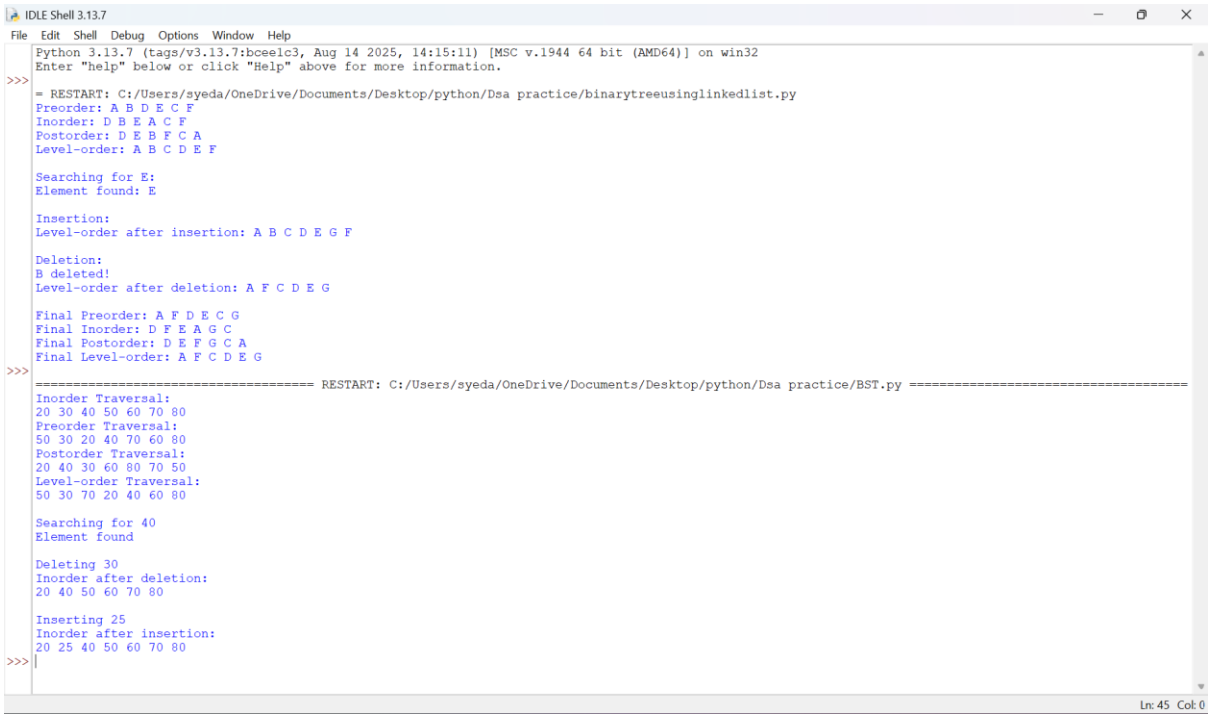

print("\nDeleting 30")
root = delete(root, 30)


print("Inorder after deletion:")
inorder(root)

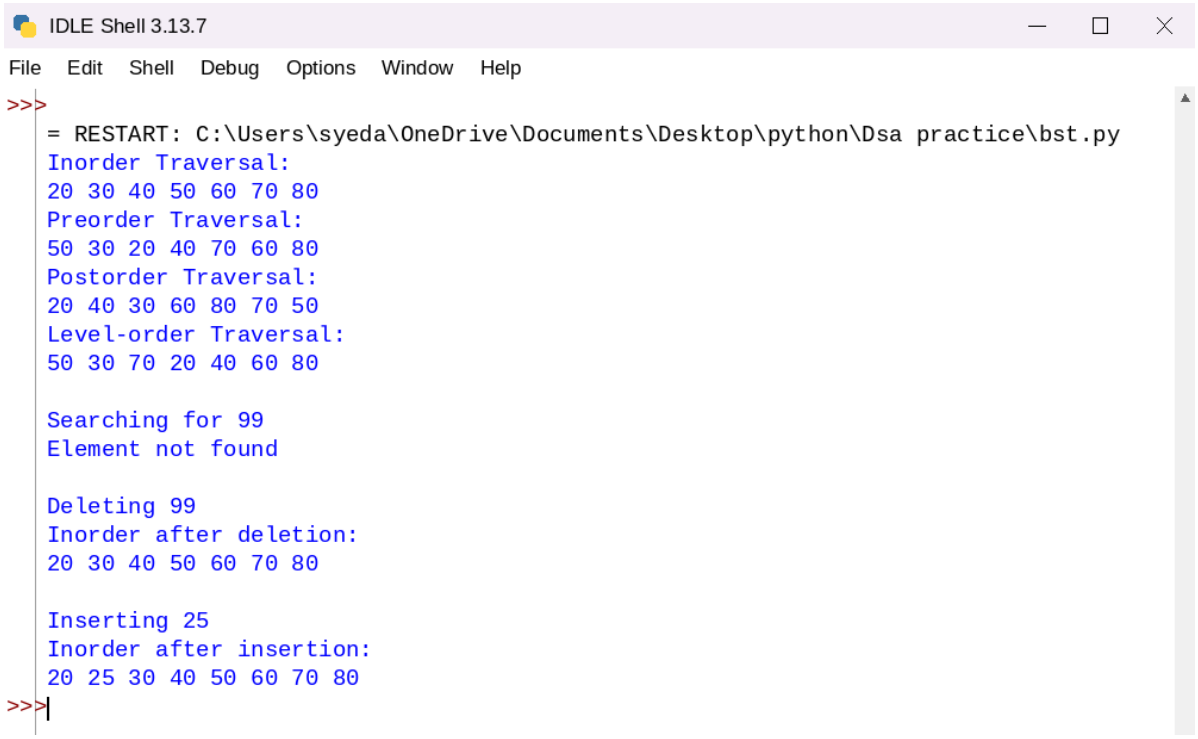

print("\n\nInserting 25")
root = insert(root, 25)


print("Inorder after insertion:")
inorder(root)
print()
```

Output:-



Negative Case:



8. Explanation: Binary Search Tree (BST)

This program implements a **Binary Search Tree (BST)** using Python and performs insertion, deletion, searching, and tree traversal operations.

1. **Node Creation:** The class Node creates a node containing data and references to its left and right children.

2. **Insertion:** The function insert(root, data) inserts a new node. Smaller values are placed in the left subtree, while larger values are placed in the right subtree.
3. **Searching:** The function search(root, key) searches for an element by comparing it with the current node and moving to the left or right subtree accordingly.
4. **Finding Minimum:** The function find_min(root) finds the smallest value by traversing the left subtree until the leftmost node is reached.
5. **Deletion:** The function delete(root, key) removes a node. It handles nodes with no children, one child, or two children while maintaining the BST property.
6. **Inorder Traversal:** The function inorder(root) visits the left subtree, root, and right subtree, displaying values in ascending order.
7. **Preorder Traversal:** The function preorder(root) visits the root, left subtree, and right subtree.
8. **Postorder Traversal:** The function postorder(root) visits the left subtree, right subtree, and root.
9. **Level-Order Traversal:** The function level_order(root) uses a queue to visit nodes level by level.
10. **Main Program:** The program creates a BST using predefined values and performs all four traversals, searches for an element, deletes a node, inserts a new node, and displays the updated traversal results.

8. Error Table – Binary Search Tree (BST)

S. No.	Error	Cause	Correction
1	`AttributeError`	Accessing a node that is `None`.	Check whether the node exists before accessing its attributes.
2	`NameError`	Using an undefined variable or function.	Declare the variable or define the function before using it.
3	`TypeError`	Comparing incompatible data types.	Ensure the values being compared have compatible data types.
4	Incorrect traversal output	Using the wrong order of visiting nodes.	Follow the correct traversal sequence: inorder, preorder, or postorder.
5	Incorrect deletion	Not handling all deletion cases.	Handle nodes with no children, one child, and two children.

Week-9

9)Aim:- To write a Python program to implement an AVL Tree and perform operations such as insertion, deletion, searching, and tree traversals (inorder, preorder, postorder, and level-order), maintaining the balance of the tree using rotations.

Program:-

class Node:

```
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1


# Calculate height
def height(node):
    if node is None:
        return 0
    return node.height


# Calculate balance factor
def balance_factor(node):
    if node is None:
        return 0
    return height(node.left) - height(node.right)


# Right rotation
def right_rotate(y):
    x = y.left
    t2 = x.right
```

```
    x.right = y
    y.left = t2

    y.height = 1 + max(height(y.left), height(y.right))
    x.height = 1 + max(height(x.left), height(x.right))

    return x


# Left rotation
def left_rotate(x):
    y = x.right
    t2 = y.left

    y.left = x
    x.right = t2

    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))

    return y


# Insert a node
```

```
def insert(root, data):
    if root is None:
        return Node(data)

    if data < root.data:
        root.left = insert(root.left, data)

    elif data > root.data:
        root.right = insert(root.right, data)

    else:
        return root

    root.height = 1 + max(height(root.left), height(root.right))

    bf = balance_factor(root)

    # LL Case
    if bf > 1 and data < root.left.data:
        return right_rotate(root)

    # RR Case
    if bf < -1 and data > root.right.data:
        return left_rotate(root)
```

```
    # LR Case
    if bf > 1 and data > root.left.data:
        root.left = left_rotate(root.left)
        return right_rotate(root)

    # RL Case
    if bf < -1 and data < root.right.data:
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root


# Search for a node
def search(root, key):
    if root is None or root.data == key:
        return root

    if key < root.data:
        return search(root.left, key)

    return search(root.right, key)


# Find minimum node
```

```
def find_min(root):
    while root.left:
        root = root.left
    return root



# Delete a node and rebalance
def delete(root, key):
    if root is None:
        return None

    if key < root.data:
        root.left = delete(root.left, key)

    elif key > root.data:
        root.right = delete(root.right, key)

    else:
        if root.left is None:
            return root.right

        elif root.right is None:
            return root.left

        temp = find_min(root.right)
```

```
    root.data = temp.data
    root.right = delete(root.right, temp.data)

if root is None:
    return None

root.height = 1 + max(height(root.left), height(root.right))

bf = balance_factor(root)

# LL Case
if bf > 1 and balance_factor(root.left) >= 0:
    return right_rotate(root)

# LR Case
if bf > 1 and balance_factor(root.left) < 0:
    root.left = left_rotate(root.left)
    return right_rotate(root)

# RR Case
if bf < -1 and balance_factor(root.right) <= 0:
    return left_rotate(root)

# RL Case
if bf < -1 and balance_factor(root.right) > 0:
```

```
        root.right = right_rotate(root.right)
        return left_rotate(root)

    return root


# Inorder traversal
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)


# Preorder traversal
def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)


# Postorder traversal
def postorder(root):
    if root:
```

```
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")


# Level-order traversal
def level_order(root):
    if root is None:
        return

    queue = [root]

    while queue:
        node = queue.pop(0)
        print(node.data, end=" ")

        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)


# Main program
root = None
```

```
values = [10, 20, 30, 40, 50, 25]

for value in values:
    root = insert(root, value)

print("Inorder Traversal:")
inorder(root)

print("\nPreorder Traversal:")
preorder(root)

print("\nPostorder Traversal:")
postorder(root)

print("\nLevel-order Traversal:")
level_order(root)

key = 30
print("\n\nSearching for", key)

if search(root, key):
    print("Element found")
else:
    print("Element not found")
```

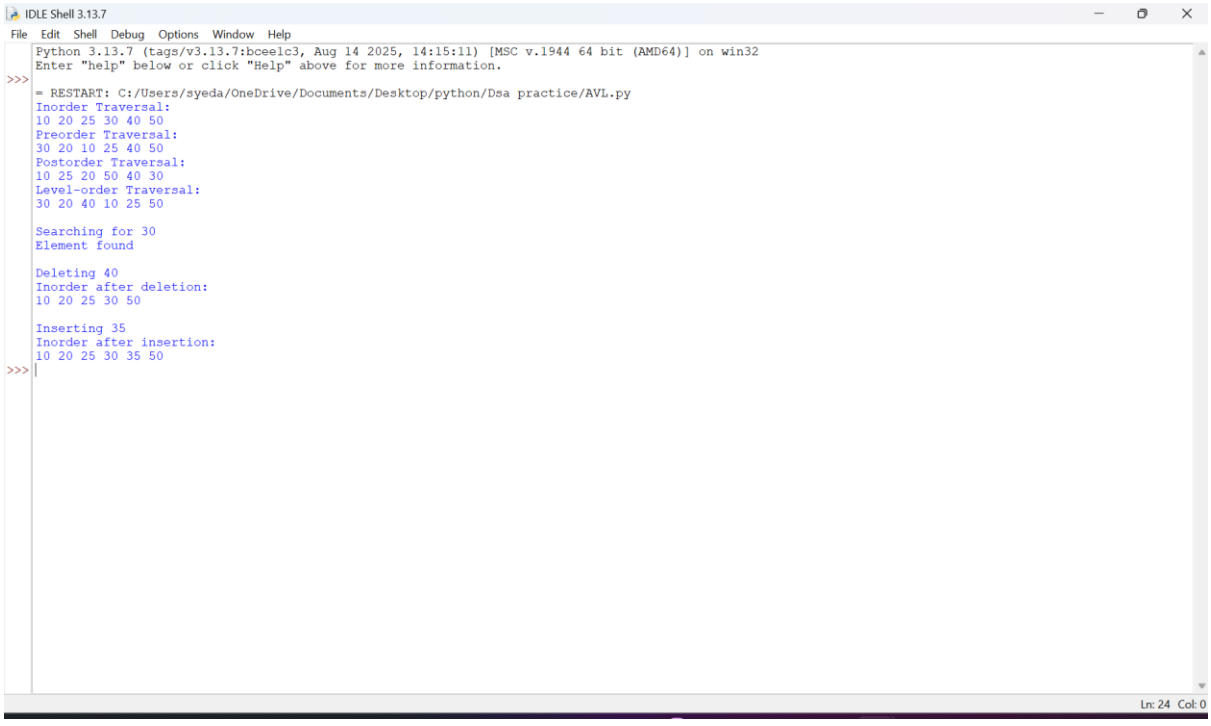
```
print("\nDeleting 40")
root = delete(root, 40)

print("Inorder after deletion:")
inorder(root)

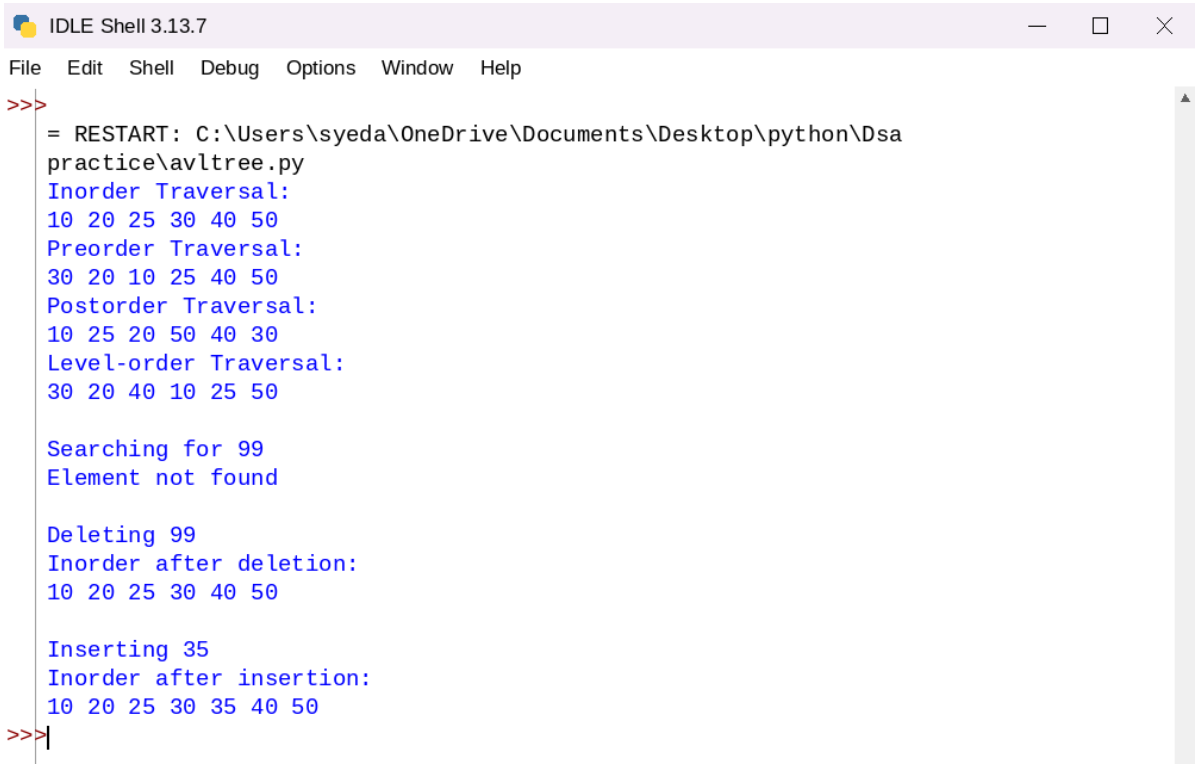
print("\n\nInserting 35")
root = insert(root, 35)

print("Inorder after insertion:")
inorder(root)
print()
```

Output:-



Negative Case:



9. Explanation: AVL Tree

This program implements an **AVL Tree**, a self-balancing Binary Search Tree, using Python. It performs insertion, deletion, searching, and tree traversal operations while maintaining the balance of the tree.

1. **Node Creation:** The class Node creates a node containing data, left and right child references, and a height value initialized to 1.
2. **Height Calculation:** The function height(node) returns the height of a node. If the node is None, it returns 0.
3. **Balance Factor:** The function balance_factor(node) calculates the difference between the heights of the left and right subtrees. An AVL tree maintains a balance factor of -1, 0, or 1 at every node.
4. **Right Rotation:** The function right_rotate(y) performs a right rotation to restore balance when required.
5. **Left Rotation:** The function left_rotate(x) performs a left rotation to restore balance when required.
6. **Insertion:** The function insert(root, data) inserts a node according to BST rules, updates node heights, calculates the balance factor, and performs rotations when necessary.
7. **Balancing Cases:** The insertion function handles four cases:
    - **LL (Left-Left):** Performs a right rotation.
    - **RR (Right-Right):** Performs a left rotation.
    - **LR (Left-Right):** Performs a left rotation followed by a right rotation.
    - **RL (Right-Left):** Performs a right rotation followed by a left rotation.

8. **Searching:** The function search(root, key) searches for an element by comparing values and moving through the appropriate subtree.
9. **Finding Minimum:** The function find_min(root) finds the smallest value by traversing the leftmost nodes.
10. **Deletion:** The function delete(root, key) removes a node while maintaining BST properties. It updates heights and performs the required rotations to restore AVL balance.
11. **Inorder Traversal:** The function inorder(root) visits the left subtree, root, and right subtree, displaying values in ascending order.
12. **Preorder Traversal:** The function preorder(root) visits the root, left subtree, and right subtree.
13. **Postorder Traversal:** The function postorder(root) visits the left subtree, right subtree, and root.
14. **Level-Order Traversal:** The function level_order(root) uses a queue to visit nodes level by level.
15. **Main Program:** The program creates an AVL tree using predefined values and performs all four traversals, searches for an element, deletes a node, inserts a new node, and displays the updated traversal results.

Key difference: A BST does not automatically balance itself, whereas an AVL tree uses rotations after insertion and deletion to maintain balance and provide efficient operations.

9. Error Table – AVL Tree

S. No.	Error	Cause	Correction
1	AttributeError	Accessing attributes of a None node.	Check whether the node is None before accessing its attributes.
2	NameError	Using an undefined function or variable.	Define all required functions and variables before using them.
3	Incorrect height	Not updating the node height after insertion or deletion.	Recalculate the height after modifying the tree.
4	Unbalanced tree	Applying an incorrect rotation.	Identify the LL, RR, LR, or RL case and apply the appropriate rotation.
5	Incorrect deletion	Failing to rebalance the tree after deletion.	Update heights, calculate balance factors, and perform rotations when required.
6	Incorrect traversal output	Visiting nodes in the wrong order.	Follow the correct traversal sequence for each traversal method.

Week-10

10)AIM:- To write a Python program to implement a Hash Table and perform various hashing operations such as insertion, searching, deletion, displaying elements, calculating load factor, handling collisions using Separate Chaining, Linear Probing, Quadratic Probing and Double Hashing, and performing Rehashing.

PROGRAM

The following implementation allows you to select:

- A hashing method.
- A collision resolution technique.
- Different hash table operations.

```
SIZE = 11

EMPTY = None

DELETED = "DELETED"



# Select Hash Function

print("HASH FUNCTIONS")

print("1. Index Mapping")

print("2. Division Method")

print("3. Mid-Square Method")

print("4. Digit Folding Method")

print("5. Multiplication Method")



hf = int(input("Select hash function: "))



# Select Collision Resolution

print("\nCOLLISION RESOLUTION")

print("1. Separate Chaining")
```

```
print("2. Linear Probing")
print("3. Quadratic Probing")
print("4. Double Hashing")

cr = int(input("Select collision method: "))

table = [[] for i in range(SIZE)] if cr == 1 else [EMPTY] * SIZE



# Hash Functions
def hash_function(key):
    if hf == 1:
        return key % SIZE

    elif hf == 2:
        return key % SIZE

    elif hf == 3:
        square = str(key * key)
        width = len(str(SIZE - 1))
        start = max(0, (len(square) - width) // 2)
        return int(square[start:start + width]) % SIZE

    elif hf == 4:
        s = str(key)
```

```
        parts = [int(s[i:i + 3]) for i in range(0, len(s), 3)]
        return sum(parts) % SIZE

    elif hf == 5:
        c = 0.618
        product = key * c
        fraction = product - int(product)
        return int(SIZE * fraction)



def second_hash(key):
    return 7 - (key % 7)



# Find Index
def find_index(key):
    index = hash_function(key)

    for i in range(SIZE):
        if cr == 1:
            return index

        elif cr == 2:
            pos = (index + i) % SIZE
```

```
        elif cr == 3:
            pos = (index + i * i) % SIZE

        else:
            pos = (index + i * second_hash(key)) % SIZE

        if table[pos] is EMPTY:
            return pos

        if table[pos] == DELETED:
            continue

        if table[pos][0] == key:
            return pos

    return -1


# Insert
def insert(key, value):
    index = hash_function(key)

    if cr == 1:
        for i in range(len(table[index])):
            if table[index][i][0] == key:
```

```
            table[index][i] = (key, value)
            print("Value updated.")
            return

    table[index].append((key, value))
    print("Element inserted.")

else:
    pos = find_index(key)

    if pos == -1:
        print("Hash table is full.")
        return

    if table[pos] is not EMPTY and table[pos] != DELETED:
        table[pos] = (key, value)
        print("Value updated.")
    else:
        table[pos] = (key, value)
        print("Element inserted.")

if load_factor() > 0.7:
    rehash()
```

```
# Search
def search(key):
    index = hash_function(key)

    if cr == 1:
        for k, v in table[index]:
            if k == key:
                print("Key found. Value:", v)
                return

    else:
        for i in range(SIZE):
            if cr == 2:
                pos = (index + i) % SIZE
            elif cr == 3:
                pos = (index + i * i) % SIZE
            else:
                pos = (index + i * second_hash(key)) % SIZE

            if table[pos] is EMPTY:
                break

            if table[pos] != DELETED and table[pos][0] == key:
                print("Key found. Value:", table[pos][1])
                return
```

```
    print("Key not found.")



# Delete
def delete(key):
    index = hash_function(key)

    if cr == 1:
        for i in range(len(table[index])):
            if table[index][i][0] == key:
                table[index].pop(i)
                print("Element deleted.")
                return

    else:
        for i in range(SIZE):
            if cr == 2:
                pos = (index + i) % SIZE
            elif cr == 3:
                pos = (index + i * i) % SIZE
            else:
                pos = (index + i * second_hash(key)) % SIZE

            if table[pos] is EMPTY:
```

```
            break

        if table[pos] != DELETED and table[pos][0] == key:
            table[pos] = DELETED
            print("Element deleted.")
            return

    print("Key not found.")


# Display
def display():
    print("\nHASH TABLE")

    for i in range(SIZE):
        print(i, ":", table[i])


# Load Factor
def load_factor():
    if cr == 1:
        count = sum(len(bucket) for bucket in table)
    else:
        count = sum(
            1 for item in table
```

```
            if item is not EMPTY and item != DELETED
        )

    return count / SIZE



# Rehashing
def rehash():
    global SIZE, table

    old_table = table

    SIZE = SIZE * 2 + 1

    if cr == 1:
        table = [[] for i in range(SIZE)]
    else:
        table = [EMPTY] * SIZE

    if cr == 1:
        for bucket in old_table:
            for key, value in bucket:
                insert_without_rehash(key, value)
    else:
        for item in old_table:
```

```
            if item is not EMPTY and item != DELETED:
                insert_without_rehash(item[0], item[1])

    print("Hash table rehashed. New size:", SIZE)


# Insert without triggering another rehash
def insert_without_rehash(key, value):
    index = hash_function(key)

    if cr == 1:
        table[index].append((key, value))

    else:
        for i in range(SIZE):
            if cr == 2:
                pos = (index + i) % SIZE
            elif cr == 3:
                pos = (index + i * i) % SIZE
            else:
                pos = (index + i * second_hash(key)) % SIZE

            if table[pos] is EMPTY:
                table[pos] = (key, value)
                return
```

```
# Main Menu
while True:
    print("\n===== HASH TABLE OPERATIONS =====")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Load Factor")
    print("6. Rehash")
    print("7. Exit")

    choice = int(input("Enter choice: "))

    if choice == 1:
        key = int(input("Enter key: "))
        value = input("Enter value: ")
        insert(key, value)

    elif choice == 2:
        key = int(input("Enter key to search: "))
        search(key)

    elif choice == 3:
```

```
        key = int(input("Enter key to delete: "))
        delete(key)

    elif choice == 4:
        display()

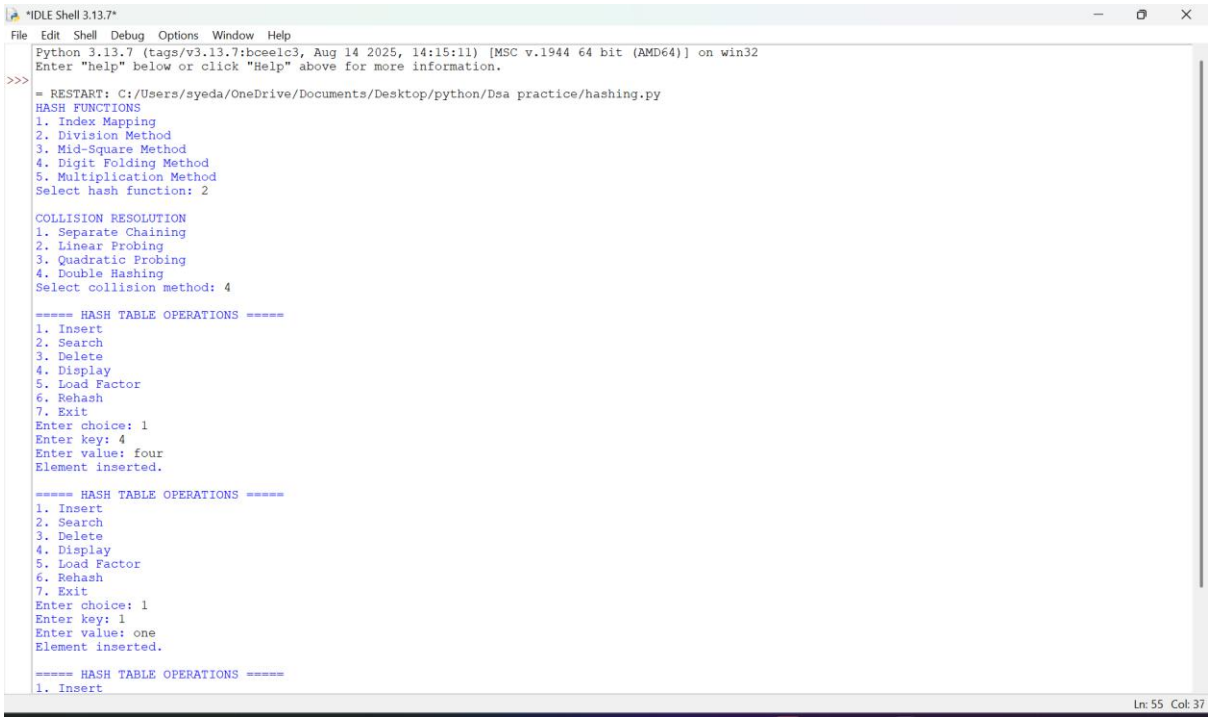
    elif choice == 5:
        print("Load Factor:", load_factor())

    elif choice == 6:
        rehash()

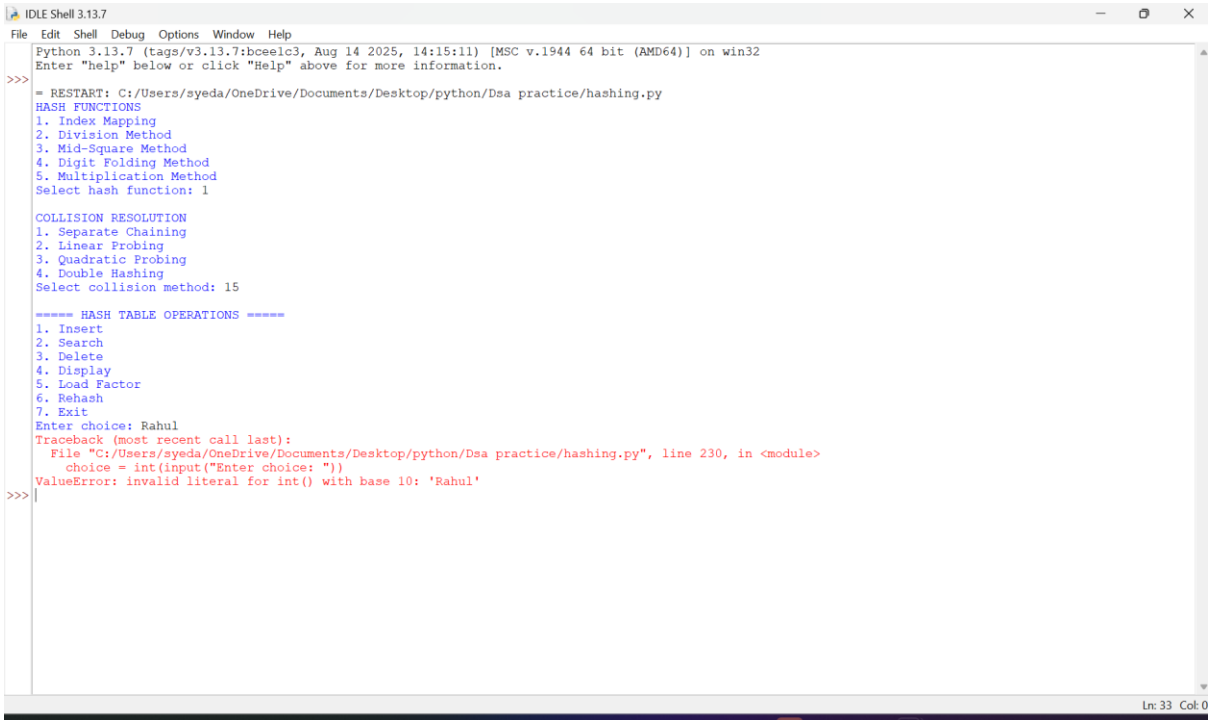
    elif choice == 7:
        print("Program terminated.")
        break

    else:
        print("Invalid choice.")
```

Output:-



Negative case:-



Negative case because:-

Insert key 15 with value Rahul.

Using Division Method: 15 % 11 = 4. The pair is stored at index 4.

Search for key 99.

Its hash index is 99 % 11 = 0.

Index 0 is empty, so the program displays **"Key not found."**

EXPLANATION

1. **Hash Table Creation:** An array-like structure is created with an initial size of 11. Each position represents a slot in the hash table.
2. **Hash Function:** The hash_function() calculates the index where a key should be stored. It supports five methods:
    - Index Mapping
    - Division Method
    - Mid-Square Method
    - Digit Folding Method
    - Multiplication Method
3. **Index Mapping:** Assigns a key to an index using a predefined mapping. It is suitable when keys belong to a small, known range.
4. **Division Method:** Calculates the index using the remainder of the key divided by the table size.

$h(x) = x \% SIZE$

5. **Mid-Square Method:** Squares the key and extracts the middle digits to obtain the hash index.
6. **Digit Folding Method:** Divides the key into smaller parts and adds those parts to generate a hash value.
7. **Multiplication Method:** Multiplies the key by a constant between 0 and 1, extracts the fractional part, and multiplies it by the table size.
8. **Collision Handling:** A collision occurs when two different keys generate the same hash index. The program supports four collision resolution techniques.
9. **Separate Chaining:** Stores multiple key-value pairs in a list associated with the same hash index.
10. **Linear Probing:** Searches for the next available position sequentially when a collision occurs.
11. **Quadratic Probing:** Searches for an available position using the square of the probe number to reduce clustering.
12. **Double Hashing:** Uses a second hash function to calculate the interval between successive positions when collisions occur.
13. **Insertion:** The insert() function calculates the hash index and stores the key-value pair. If the key already exists, its value is updated.
14. **Searching:** The search() function calculates the index and searches for the required key using the selected collision resolution technique.
15. **Deletion:** The delete() function removes the specified key. In open addressing, a deleted marker is used so that searching for other keys continues correctly.
16. **Display:** The display() function prints all the positions and their corresponding values.
17. **Load Factor:** The load_factor() function calculates how full the hash table is.

α = Number of occupied slots / Total number of slots

18. **Rehashing:** The rehash() function creates a larger hash table and inserts all existing elements again after recalculating their indices.
19. **Main Menu:** A while loop allows the user to repeatedly perform the operations until Exit is selected.

Important Formulae:-

Method	Formula
Division	h(x) = x % M
Mid-Square	Square the key and extract middle digits
Digit Folding	Add the separate parts of the key
Multiplication	h(x) = floor(M × fractional(x × C))
Linear Probing	(h(x) + i) % M
Quadratic Probing	(h(x) + i²) % M
Double Hashing	(h1(x) + i × h2(x)) % M
Load Factor	α = n / M

Time complexity:-

Operation	Average Case	Worst Case
Insertion	O(1)	O(n)
Searching	O(1)	O(n)
Deletion	O(1)	O(n)
Display	O(n + M)	O(n + M)
Load Factor	O(n + M)	O(n + M)
Rehashing	O(n)	O(n)

SPACE COMPLEXITY: O(n + M)

Error Table – Week 9: Hashing:-

S.No.	Error	Rectification
1	Searching for a missing key.	Display "Key not found."
2	Deleting a missing key.	Check whether the key exists.

S.No.	Error	Rectification
3	Inserting duplicate keys.	Update the existing value.
4	Collision occurs.	Apply a collision resolution technique.
5	Hash table becomes full.	Perform rehashing.
6	Incorrect hash index.	Ensure the index is within table size.

!<-----THE END----->!